University of Warsaw
Faculty of Economic Sciences

Krzysztof Wojdalski

Nr albumu: 310284

Microstructure-Aware Reinforcement Learning for High-Frequency Trading

Praca magisterska
na kierunku QUANTITATIVE FINANCE

The thesis written under the supervision of
dr Pawel Sakowski

August 15, 2026 18:40 UTC

Data Podpis opiekuna

Statement of the Supervisor on Submission of the Thesis

I hereby certify that the thesis submitted has been prepared under my supervision and I declare that it satisfies the requirements of submission in the proceedings for the award of a degree.

Date Signature of the Supervisor:

Oświadczenie autora (autorów) pracy

Data Podpis autora (autorów) pracy

Statement of the Author(s) on Submission of the Thesis

Aware of legal liability I certify that the thesis submitted has been prepared by myself and does not include information gathered contrary to the law.

I also declare that the thesis submitted has not been the subject of proceedings resulting in the award of a university degree.

Furthermore I certify that the submitted version of the thesis is identical with its attached electronic version.

Date Signature of the Author(s) of the thesis

Table of contents

List of Figures

List of Tables

Abstract

This thesis studies whether a deep reinforcement learning agent using order-book-derived microstructure features can produce robust, risk-adjusted trading performance in a high-frequency single-asset setting. The work focuses on continuous portfolio exposure control and addresses the practical constraints — execution modeling, transaction cost sensitivity, and evaluation discipline — that most often invalidate paper-trading results.

A Twin Delayed DDPG (TD3) agent is trained on tick-level order book snapshots from six Nasdaq-listed equities (AAPL, MSFT, TSLA, META, AMZN, and AVGO), covering three consecutive trading sessions in February 2026, and evaluated against benchmark strategies under explicit simulation assumptions. The agent achieves positive held-out returns with very low drawdown and a high profit factor, but does not dominate passive long exposure on cumulative return over the upward-drifting evaluation window. A microstructure-aware state representation constructed from order flow, spread regime, and book shape features improves policy behavior relative to simpler snapshot-based variants, while transaction-cost sensitivity analysis shows that the observed edge is fragile once fees increase.

The main contribution is an end-to-end, reproducible pipeline linking LOB feature engineering, Differential Sharpe Ratio reward design, TD3 training, and rigorous out-of-sample evaluation — together with an explicit account of the simulation assumptions that separate these findings from deployable performance claims.

1. Introduction

Much of the reinforcement learning literature in trading relies on low-frequency market representations such as daily or intraday OHLCV bars.[1] High-frequency trading, however, is governed by information embedded in the limit order book, where local liquidity, queue dynamics, spread changes, and event timing determine short-horizon risks. This thesis examines whether a deep RL agent can use order-book-derived features to make favourable trading decisions in a HFT setting.

High-frequency trading should therefore not be treated as a faster version of medium-frequency trading (MFT). The state observed by the agent is different. At short horizons, aggregated price bars omit information that is central to decision-making. Features derived from the order book provide a more detailed description of market conditions than candlesticks. These include depth imbalance, spread variation, order-flow pressure, and inter-event timing. A primary concern of this thesis is whether such microstructure-aware state representations improve the suitability of RL for this domain.

Reinforcement learning is a plausible modelling framework for this problem because trading is sequential and path dependent. Each action affects future exposure, portfolio evolution, and the set of subsequent decisions available to the agent. In high-frequency settings, this distinction matters.

Reinforcement learning has already been deployed in industry settings on limit order book data, but these applications typically address narrower problems such as optimal execution.[2]

Directional trading is a harder problem for several reasons. First, the reward signal at tick frequency is dominated by noise rather than price movement. Second, the LOB state must be transformed into features that carry predictive signal rather than merely describing liquidity conditions. Third, the agent must generalize across intraday regimes whose statistical properties can differ substantially from the training window.

This thesis investigates the use of RL, with particular focus on TD3, for high-frequency single-asset trading based on order-book-derived information. The research is motivated not

[1] Abbreviations and domain-specific terms used throughout the thesis are defined in the glossary at the end of the document.

[2] An illustrative example is J.P. Morgan's LOXM system, described in contemporaneous reporting as an AI-driven execution tool for client orders rather than a stand-alone directional trading strategy (Noonan 2017). In that execution setting, the limit order book is used to inform decisions about execution quality, fill likelihood, and spread-cost trade-offs, not to forecast short-horizon price direction.

by a claim that artificial intelligence can universally solve trading, but by a narrower and testable question: under controlled experimental conditions, can a continuous control agent using microstructure-aware features achieve superior out-of-sample performance relative to selected benchmarks?

Further constraints arise on both the execution and evaluation sides. Transaction costs can erase apparent profitability, and short-horizon strategies are highly sensitive to turnover, feature specification, and market frictions. Favorable in-sample results may additionally reflect data leakage, overfitting, or unrealistic execution assumptions rather than genuine decision quality.

Accordingly, the thesis focuses on both methodological discipline and model development. It treats feature engineering, chronological evaluation, reward-return separation, and robustness analysis as core parts of the research problem. In this setting, the question is not only whether an agent can learn, but whether the learning setup is properly specified to support credible conclusions.

At the same time, the thesis does not claim to reproduce the full complexity of live high-frequency trading. The framework is based on a controlled simulation environment rather than a complete execution engine with queue-priority modelling, latency competition, or full market impact. For instance, portfolio valuation relies on a mid-price-like execution proxy, and the agent is treated as a price-taking participant that does not alter the state of the order book. The contribution of the thesis should therefore be methodological and empirical: it evaluates RL agents in a microstructure-aware setting while making the simplifying assumptions explicit.

1.1. Scope and Objectives of the Research

1.1.1. Novelty of This Research

The primary novelty of this thesis is the application of **TD3** to **high-frequency trading with order-book-derived data** for continuous portfolio-exposure control. This differs from a large share of prior RL trading studies that focus on lower-frequency inputs and evaluate algorithms such as Q-learning, DQN, or PPO primarily in bar-level settings.

This thesis contributes by evaluating the RL trading problem in a setting where algorithm class, data granularity, and feature design are jointly aligned with high-frequency market microstructure. The targeted algorithm-data-frequency combination remains less represented

in prior work and defines the core novelty claim tested in the empirical chapters.

1.1.2. Hypothesis

The research is based on the following hypotheses:

1. A TD3-based trading agent using order-book-derived high-frequency features can achieve stronger out-of-sample risk-adjusted performance than selected benchmark strategies under the assumptions of the simulated environment.
2. Within the order-book-based trading framework studied here, extending the state representation from basic snapshot features to a broader microstructure-aware feature set leads to improved out-of-sample performance under identical training and evaluation conditions.
3. The empirical performance of reinforcement learning trading agents is materially sensitive to feature specification, reward design, and transaction-cost assumptions, which makes robustness analysis necessary for credible conclusions.

1.1.3. Objectives of the Research

The primary research goal is to evaluate RL-based algorithms for high-frequency single-asset trading using order-book-derived data. The research objectives are structured in three interconnected phases:

1. **Design and Implementation**
   - Develop RL trading agents for high-frequency order-book trading applications
   - Construct a testing framework to evaluate agent performance
2. **Testing and Evaluation**
   - Assess agent performance on out-of-sample data against selected benchmarks with use of established and novel risk-return metrics
   - Analyze robustness under varying market conditions
   - Determine whether agents can outperform selected benchmark strategies under the defined experimental assumptions
3. **Analysis and Implications**
   - Compare alternative order-book-derived feature sets within a common reinforcement learning training and evaluation framework

- Assess whether broader microstructure-aware feature sets improve out-of-sample results relative to simpler snapshot-based state representations
- Assess the practical relevance and limitations of the approach under simplified execution assumptions
- Identify the binding constraints on out-of-sample performance — whether they originate in execution modeling simplifications, feature set coverage, or training data volume — to prioritize specific extensions for future work

1.2. Thesis Organization and Chapter Overview

The thesis proceeds from problem formulation and motivating literature to agent design and implementation, and finally to empirical evaluation.

Chapter 1 introduces the problem context, defines the research scope, and presents the methodological framework used throughout the thesis.

Chapter 2 reviews the literature most directly relevant to the present research question. It covers market microstructure evidence motivating order-book-derived features, the landscape of alternative machine learning paradigms and trading decision architectures, the structural case for reinforcement learning in trading, and prior RL trading applications with attention to action-space design, transaction-cost modeling, and evaluation rigor.

Chapter 3 develops the reinforcement learning foundations required for the thesis, with particular emphasis on the progression from earlier methods to TD3 and on the distinctions that matter for continuous-control trading problems.

Chapter 4 presents the design of the trading agent itself. It defines the continuous action space, the microstructure-aware state representation, the reward formulation, and the main policy and value-function design choices used in the empirical system.

Chapter 5 describes the implementation of the research pipeline, including the simulation architecture, data preparation, reproducible feature engineering, code structure, and the TD3-based training workflow used to produce reproducible experiments.

Chapter 6 reports the empirical results. It combines experiment snapshots, performance evaluation, statistical validation, and robustness assessment to separate promising outcomes from noise, overfitting, or fragile conclusions.

Chapter 7 synthesizes the findings, discusses the main limitations of the current framework, and draws implications for future research and practical trading-system development.

2. Literature Review

This chapter surveys the academic foundations relevant to reinforcement learning for tick-level order-book trading. It first distinguishes the controlled simulation setting studied here from live high-frequency trading, then reviews the microstructure mechanisms that motivate the candidate state variables used in Chapter 4. The chapter then separates descriptive microstructure variables from tradable signals: a feature is useful only if it contains information available before the action, survives costs, and improves out-of-sample performance. The second half compares competing modeling approaches, introduces the reinforcement learning concepts most relevant to trading, and reviews prior RL trading applications through state representation, action-space design, reward definition, transaction-cost modeling, and validation rigor. The survey concludes by identifying the comparatively underexplored combination of LOB-derived features, continuous control, and microstructure-aware state design that this thesis studies.

2.1. Tick-Level Order Book Trading

Tick-level trading differs from bar-based trading because the decision unit is an order book event rather than a fixed calendar interval. A daily or minute-level strategy can often treat execution as a secondary implementation detail after the signal is estimated.[3] At tick frequency, execution conditions are part of the signal environment itself: the spread, queue sizes, recent order-flow pressure, and time since the last event all affect whether a position change is worth making.

This matters because expected price changes over short horizons are small relative to trading frictions. Crossing the spread, paying transaction costs, or changing exposure during a transient liquidity shock can dominate the gross directional signal. A tick-level agent must therefore condition not only on where the price may move, but also on whether the current liquidity state makes acting economically sensible.

The present thesis studies this problem in a controlled simulation, not in a live high-frequency trading environment. The simulator uses historical limit-order-book data and explicit cost assumptions to evaluate continuous position control. It does not model exchange latency,

[3]This generalization depends on the scale of trading. For investment funds, execution is rarely a negligible issue — quite the opposite. Large-scale execution costs, market impact, and implementation shortfall can dominate gross returns, making execution quality a primary concern.

queue priority, partial fills, strategic order cancellation, or the market impact of the agent's own orders. The literature reviewed below is therefore used to motivate state variables and evaluation discipline, not to claim full live-HFT realism.

The microstructure mechanisms discussed in this chapter manifest in observable empirical regularities at intraday and tick-level timescales.

At these timescales, financial markets exhibit structural patterns that are distinct from the macro-level anomalies studied in the asset pricing literature. They arise not from investor irrationality or information asymmetry in the fundamental sense, but from the mechanics of how orders interact in a limit order book: the strategic behavior of market makers, the clustering of institutional activity, and the discrete price grid imposed by exchange rules. Understanding these patterns is a prerequisite for designing a tick-level trading agent, since they define the environment in which the agent operates and motivate the feature set used here.

Roll (1984) demonstrated that bid-ask spreads induce negative serial correlation in transaction prices at very short horizons, as they bounce between bid and ask quotes. This microstructure noise distinguishes observed transaction prices from the latent efficient price and creates a mechanical pattern that matters for any tick-level decision rule. It is not, by itself, a tradable opportunity: any attempt to exploit bid-ask bounce must overcome spread costs, latency, and the timing constraints of the execution model. In this work, spread in basis points and the spread ratio feature directly encode the cost of crossing the bid-ask, conditioning the agent's position-sizing decisions on the current liquidity environment. Whether this information supports profitable decisions is left to the later out-of-sample evaluation.

Wood et al. (1985) and Harris (1986) documented intraday patterns in volume, volatility, and returns: a pronounced U-shape in trading activity and spread width, with elevated liquidity demand at market open and close and a quiet midday period. These patterns reflect institutional trading schedules and liquidity provision dynamics rather than information arrival. They motivate time-of-day context variables, but they do not establish a standalone trading signal without horizon-specific validation and transaction-cost assessment. The hour-of-day sine and cosine encodings in the observation vector are motivated by this evidence because the agent should distinguish early-session conditions (wide spreads, high adverse selection) from midday periods (tighter spreads, lower activity) when sizing positions.

Harris (1991) showed that prices cluster at round numbers and psychologically attractive levels, creating discontinuities in the order book that affect optimal order placement and execution. In a 10-level LOB, price clustering manifests as uneven spacing between price levels — a pattern captured by the book convexity feature, which measures the curvature of the price ladder on each side. Awareness of price clustering is also relevant to the spread ratio feature, since clusters create temporary spread compression and expansion that deviates from the recent average.

These three findings collectively establish that short-horizon price dynamics are structured, not random. The bid-ask bounce motivates spread-aware features and reward design; intraday periodicity motivates temporal context features; price discreteness motivates curvature features of the order book. Rather than treating the limit order book as a transparent window onto an efficient price, the empirical evidence supports modeling it as a strategic environment with predictable mechanical regularities — which is precisely the setting in which a reinforcement learning agent can learn to act systematically.

This does not mean that every microstructure regularity is directly tradable. Some variables describe the current liquidity environment more than they forecast the next price change, and any useful signal must survive transaction costs, action timing, and out-of-sample evaluation. The role of the literature review is therefore to justify the candidate state variables; whether those variables support profitable decisions is an empirical question addressed by the later experimental chapters.

2.2. Market Microstructure Mechanisms

Market microstructure is grounded in a theoretical literature that models the strategic interactions between informed traders, uninformed liquidity demanders, and market makers in a limit order book. This section reviews the theoretical frameworks that directly underpin the feature design choices made in Chapter 4. The organizing logic is mechanism, observable feature, and limitation: each framework motivates a candidate state variable, but none establishes by itself that the variable is profitable after costs.

Adverse selection and the spread decomposition. Glosten and Milgrom (1985) established that the bid-ask spread reflects a market maker's protection against trading with informed counterparties. In their model, the market maker faces two types of order flow:

uninformed liquidity traders, whose trades are uncorrelated with future price changes, and informed traders, who possess private information about fundamental value. Setting a positive spread allows the market maker to profit from uninformed flow and recoup losses on informed flow. The spread therefore decomposes into an adverse selection component — compensation for information risk — and an inventory or order processing component.

This decomposition has a direct implication for trading signal design. A narrowing spread signals reduced adverse selection risk; a widening spread signals elevated informed activity. These measures, operationalized in Chapter 4 as spread in basis points and the spread ratio, are therefore motivated as candidate state variables by this theoretical framework. Their relevance in the present thesis remains an empirical question, to be assessed later under the specific data and environment assumptions adopted here.

Kyle (1985) formalized the price impact of informed trading through the lambda parameter, which measures the sensitivity of price changes to order flow. In Kyle's continuous-time model, the informed trader optimally splits their order to minimize price impact, and the market maker sets prices to break even against order flow. The resulting relationship between order imbalance and price change is linear, with slope λ that depends on the ratio of informed to uninformed volume. The order flow features in Chapter 4 — OFI and signed trade flow — can be understood as empirical proxies for Kyle's order imbalance variable, with the bid-ask slope features measuring the depth analogue of λ across levels of the order book. The limitation is that these proxies summarize observable pressure and depth, not the latent information set of the informed trader in Kyle's model.

Limit order book mechanics. Glosten (1994) developed the theory of the open limit order book as a competitive equilibrium, showing that a full LOB can sustain itself as a market mechanism under adverse selection. The key insight is that the price schedule embedded in the LOB — the sequence of bid and ask quotes at increasing depths — reflects the cumulative information content of orders at each quantity level. Deeper levels of the book attract less informed order flow but carry higher execution uncertainty for those posting limit orders.

This theoretical result motivates the multi-level features used in Chapter 4. The book is not a single price but a price schedule, and the shape of that profile — measured by depth ratios, slopes, and convexity features — may encode information about how informed the marginal liquidity provider expects the market to be at each quantity level. In this sense, the theory

motivates examining multi-level book-shape features; it does not by itself establish which specification is most informative for the present application.

Order flow and short-term price prediction. Cont et al. (2014) provide the most direct empirical and theoretical treatment of the relationship between LOB events and short-term price changes. Their framework decomposes all LOB events into limit order arrivals, cancellations, and market orders, and defines order flow imbalance (OFI) as the net directional pressure these events exert on the best bid and ask queues. In a sample of NYSE stocks, they find a linear relationship between OFI and mid-price changes over intervals of one second to one minute, with OFI explaining 60–80% of short-term price variation. Importantly, the relationship is contemporaneous: OFI at time t is correlated with price changes at time t, not a lagged effect, which is consistent with the efficient incorporation of order book information into prices.[4]

The theoretical channel is straightforward: when buy-side pressure dominates (positive OFI), market makers raise quotes to attract sell-side flow and reduce inventory risk, mechanically pushing the mid-price upward. The OFI feature in the feature design is included primarily as a state variable summarizing current order-flow pressure, not because the literature alone proves standalone predictive power at the forecast horizon relevant to this thesis.

Fair value estimation from the order book. Stoikov (2018) derives the microprice as the theoretically optimal estimate of the next transaction price given the current state of the top-of-book queues. The key observation is that the mid-price — the arithmetic average of best bid and ask — is an inadequate fair value estimate when the queues are imbalanced, because the side with the thinner queue is more likely to be depleted first by incoming market orders, pulling the next transaction price toward the opposite side. The microprice corrects for this by volume-weighting the best quotes:

$$\text{Microprice} = \frac{V_0^{ask} \cdot P_0^{bid} + V_0^{bid} \cdot P_0^{ask}}{V_0^{bid} + V_0^{ask}} \tag{1}$$

where:

[4]This contemporaneous result does not by itself establish OFI as a predictive signal in the strict forecasting sense. Its relevance for the present thesis is narrower: OFI serves as a compact state descriptor of current queue pressure at the time the agent acts. If lagged or windowed OFI measures prove useful in the present setting, that is an empirical result of the later design and evaluation chapters rather than something established by the cited evidence alone.

- P_0^{bid} — best bid price (level 0)
- P_0^{ask} — best ask price (level 0)
- V_0^{bid} — resting volume at the best bid
- V_0^{ask} — resting volume at the best ask

When $V_0^{bid} \gg V_0^{ask}$, the microprice is pulled toward the ask — the side more likely to move. The microprice divergence from mid captures this adjustment as a stationary signal and serves as the primary fair value feature in Chapter 4. Its limitation is that it is a top-of-book fair-value proxy; it does not incorporate deeper queue dynamics, hidden liquidity, or the agent's own execution priority.

Taken together, these four theoretical frameworks identify several mechanisms that may be relevant for state design in limit-order-book markets. The spread encodes adverse selection risk (Glosten and Milgrom 1985); price impact and order imbalance are related through Kyle's lambda (Kyle 1985); the shape of the full book encodes the information content at each depth level (Glosten 1994); and OFI is the dominant contemporaneous predictor of short-term price changes (Cont et al. 2014). The microprice (Stoikov 2018) provides a theoretically optimal fair value estimate that conditions on the current queue imbalance.

These mechanisms are not independent: they all reflect aspects of information aggregation through order flow. They therefore motivate the candidate feature families introduced in Chapter 4, including spread-based, order-flow, depth-shape, and fair-value signals. Whether that feature set captures the most informative aspects of the environment is not something the literature alone can settle; within this thesis, that question is addressed only through the later design and empirical evaluation chapters.

2.3. From Microstructure Signals to Tradable Decisions

The preceding sections motivate the use of LOB-derived variables as state descriptors, but they do not imply that those variables are automatically profitable trading signals. A spread, imbalance, microprice divergence, or intraday-time feature may describe the current liquidity environment without forecasting the next price change strongly enough to justify trading.

This distinction is especially important for order-flow variables. Empirical microstructure studies often document contemporaneous relationships between book events and price changes. Such relationships show that order-book information is incorporated into prices, but they do

not by themselves establish lagged predictability at the decision horizon available to a trading agent. For the present thesis, OFI and related features are therefore treated as compact summaries of current queue pressure. Whether lagged or rolling versions of those features help a policy choose profitable exposure is an empirical question.

The same discipline applies to all candidate features. A variable is useful for the trading agent only if it contains information available before the action, survives transaction costs and spread effects, and improves performance on held-out data under a fixed evaluation protocol. The role of the literature is to justify which mechanisms are worth representing in the state; the later empirical chapters test whether those representations support tradable decisions under the simulation assumptions of this thesis.

In the quantitative trading literature, predictive signals of this kind are commonly referred to as alphas: mathematical expressions over market data that encode a directional view on short-horizon returns (Tulchinsky 2019). The microstructure features in this thesis's state space — order-flow imbalance, microprice divergence, spread regime, and book shape — occupy the same conceptual role. The distinction from the traditional alpha framework is that they are not used as standalone trading rules. Instead they serve as inputs to an RL policy that learns, through experience, how to combine and act on them optimally. The RL agent replaces the hand-coded combination rule with a learned function, and the evaluation discipline remains the same: out-of-sample performance under realistic cost assumptions.

2.4. Competing Modeling Approaches

The alternatives to reinforcement learning fall into two complementary perspectives: learning paradigm and decision-system architecture. From the learning-paradigm perspective, the relevant categories are supervised, unsupervised, semi-supervised, and reinforcement learning; the present section discusses the first three explicitly and then motivates reinforcement learning through the architectural comparison that follows.

2.4.1. Learning Paradigms

Supervised Learning. Supervised learning trains a model on labeled input-output pairs $(X_i, Y_i)_{i=1}^n$ to minimize a loss function $\mathcal{L}(f_\theta(X), Y)$ over a held-out sample. In trading, supervised approaches have been widely applied to return prediction, directional classification, and volatility forecasting — tasks where historical outcomes provide a natural training signal

(Krauss et al. 2017; Zhang et al. 2019).

This includes modern LOB prediction models, which use deep architectures to map order book snapshots or event sequences into short-horizon price-movement labels. Such models are a serious benchmark class for extracting information from high-dimensional market data, especially when the objective is directional classification (Zhang et al. 2019). A strong supervised trading benchmark would not stop at an unconstrained return forecast. It would translate predicted returns or class probabilities into positions through a cost-aware rule, such as trading only when the expected move exceeds the spread and fee threshold, scaling exposure by forecast confidence, or optimizing classification thresholds on a validation set for net return rather than raw accuracy. A second supervised variant would learn the action directly from labels constructed ex post: for example, the position that would have maximized next-horizon net return after costs, or a discretized buy/sell/hold target with an explicit no-trade region.

These alternatives are important because they close part of the gap between prediction and trading. They allow the supervised model to account for transaction costs, avoid low-conviction trades, and produce a policy-like mapping from market state to position. The limitation is therefore not that supervised learning is an inherently weak baseline. It is that the financial objective enters the pipeline indirectly: either through hand-designed thresholds and sizing rules applied after prediction, or through labels whose definition already embeds a particular holding horizon, cost model, and notion of ex post optimality.

This creates two sources of possible misalignment. First, a model that minimizes mean squared error, cross-entropy, or another statistical loss may not maximize risk-adjusted return after transaction costs, because predictive accuracy and trading profitability are not equivalent criteria (Moody and Saffell 1998). Second, direct action labels are only as credible as the rule used to construct them. In a noisy and path-dependent environment, the ex post best one-step action need not be the action that maximizes longer-horizon portfolio utility once turnover, inventory persistence, and risk are considered.

This misalignment is most consequential at high frequencies, where turnover costs consume a large fraction of gross returns. At lower frequencies — daily or weekly rebalancing — the cost of turning over a position is small relative to the forecast signal, and well-specified supervised models remain competitive. Factor-based strategies in the Fama-French tradition, momentum screens, and several recent deep-learning forecasting papers achieve meaningful

net alpha precisely because the prediction horizon is long enough for transaction costs to be a second-order concern. The advantage of direct optimization narrows as the cost-to-signal ratio decreases. At tick frequency, where the present thesis operates, this ratio is unfavorable for a predict-then-trade pipeline: the cost of crossing the spread on each position change can exceed the directional signal per step, making the alignment between reward and realized return a first-order design requirement. The motivation for reinforcement learning in this thesis should therefore be understood narrowly. TD3 is not adopted because supervised LOB prediction is irrelevant, but because the empirical question studied here concerns continuous exposure control under a reward function that includes the consequences of position changes directly.

Unsupervised Learning. Unsupervised learning identifies structure in unlabeled data $\{X_i\}_{i=1}^n$ without a target variable. In trading research it is primarily used for market regime detection (clustering volatility or microstructure states), dimensionality reduction of large feature sets (PCA, autoencoders), and anomaly detection in order flow data.

These applications are useful as preprocessing or exploratory tools, but unsupervised learning does not produce trading decisions directly — it carries no reward signal and cannot optimize for a financial objective such as risk-adjusted return. In the present thesis, the regime-conditioning problem is addressed implicitly through engineered microstructure features — spread ratio, inter-event time, temporal encodings — that condition the agent's observation on the current market state without requiring an explicit unsupervised clustering step.

Semi-Supervised Learning. Semi-supervised learning combines a small labeled set with a larger unlabeled set, propagating label information through structural similarity in the data. It is useful when labeling is expensive or subjective — medical annotation being the canonical application.

In financial trading, the approach is rarely adopted because the main bottleneck is not label scarcity but the absence of a stable labeling criterion: what constitutes an optimal action at a given market state depends on subsequent price evolution, which is noisy and non-stationary. Constructing pseudo-labels for unlabeled market observations requires either a strong prior on optimal behavior or a separate forecasting model — at which point the problem has already been reformulated as supervised or reinforcement learning. This instability of financial labels is documented by López de Prado (2018, Ch. 3), who argues that fixed-horizon labeling

schemes are path-dependent and regime-sensitive, and proposes the triple-barrier method as a more robust alternative — itself an acknowledgment that no universal labeling criterion exists across market conditions.

The same conclusion follows from a second perspective: the architecture of the decision system itself.

2.4.2. Decision-System Architectures

Rule-Based Trading Strategies. Rule-based strategies encode trading decisions as explicit conditional logic: position changes are triggered when observable market quantities cross predefined thresholds. Common examples include moving-average crossover systems, momentum breakout rules, and mean-reversion strategies based on Bollinger Bands or RSI. In high-frequency settings, rule-based approaches include market-making strategies that post limit orders at fixed offsets from the mid-price and cancel them when inventory limits are reached.

The appeal of rule-based systems is transparency: the causal chain from market condition to action is explicit, auditable, and reproducible. They also impose no distributional assumptions on the data and can be deployed without a training phase.

The limitation is inflexibility. Rules are defined over a fixed set of observable conditions and optimized for a particular market regime. When it changes — for example, when a previously mean-reverting asset enters a trending period — fixed rules either require manual recalibration or continue generating losses until the regime reverts. This brittleness under non-stationarity is the primary motivation for learning-based approaches, which can update their behavior from experience rather than requiring explicit rule revision.

Indirect Optimization via Forecasts or Labels. An intermediate class of trading systems learns a surrogate target rather than optimizing trading performance directly. One variant employs supervised learning to generate price forecasts from market inputs and then feeds those forecasts into a separate decision rule. Another variant trains directly on labeled state-action pairs, imitating historical or hand-crafted trades without an explicit forecasting stage. In the latter case, the model learns $f : X \to a$ by minimizing a loss over these examples (X_i, a_i^*):

$$\min_{\theta} \sum_{i=1}^{n} \ell(f_\theta(X_i), a_i^*) \tag{2}$$

where:

- θ — parameters of the policy function being learned
- n — number of labeled training examples
- X_i — market state (input features) for example i
- a_i^* — expert or rule-derived action label for example i
- $f_\theta(X_i)$ — predicted action under the current parameters
- $\ell(\cdot, \cdot)$ — loss function measuring deviation from the expert action

The limitation is the same in both cases: the optimization target is a surrogate rather than the trading objective itself. Forecast-based systems optimize statistical accuracy and then translate predictions into actions through a separate rules layer, discarding much of the contextual information present in the original state. Label-based systems optimize imitation error against actions derived from historical rules or expert behavior, which embeds a prior about what constitutes optimal behavior and does not guarantee maximizing risk-adjusted return.

Direct Optimization of Trading Performance. Direct optimization removes the predict-then-trade pipeline: the agent observes the market state X_t, selects an action, and receives a reward computed from realized returns and transaction costs directly. There is no decoupled forecast stage and no separate rule translating predictions into positions — transaction costs and position sizing enter the optimization loop directly rather than being handled by a post-hoc threshold or sizing rule.

The structural advantage over supervised approaches is therefore not the elimination of a modeling assumption but the elimination of the calibration gap between a forecast objective and a decision rule. The reward used in this thesis — the Differential Sharpe Ratio — is itself an approximation of risk-adjusted return rather than a direct measure of financial performance, and the relationship between training reward and out-of-sample return remains an empirical question. This distinction is developed in Chapter 4, which also explains why a shaped reward is necessary at tick frequency and how financial performance metrics are kept separate from the training objective.

2.5. Reinforcement Learning for Trading

Reinforcement learning is a reasoned choice for algorithmic trading, not an arbitrary methodological one. The argument rests on three structural properties of the trading problem.

Unknown market dynamics. Model-based approaches require an explicit transition model — a specification of how the environment responds to actions. In financial markets, this model is unknown and non-stationary: price dynamics depend on the aggregate behavior of all participants, which changes continuously. Model-free RL methods learn directly from experience without requiring a transition model, making them a reasonable candidate for this setting (Sutton and Barto 2018).

Off-policy learning and historical replay. Online experimentation in live markets is constrained by capital, risk, and regulatory requirements. The alternative is to learn from historical order book data — a fixed dataset of past transitions. Off-policy algorithms such as TD3 (Fujimoto et al. 2018) can train on stored experience rather than requiring fresh environment interaction at every update, making them sample-efficient in the data-limited financial setting. This should not be confused with a full offline-RL guarantee. Training from historical replay still raises distribution-shift and extrapolation risks when the learned policy selects actions that are poorly represented in the historical data.

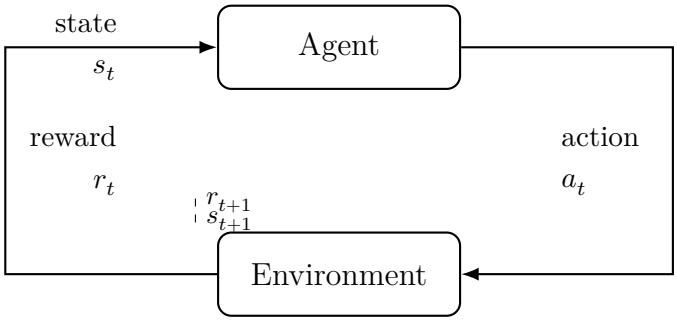
Continuous position sizing. Trading decisions are naturally continuous: the agent should be able to express any degree of conviction through the size of its position, not just a binary buy/sell signal. Value-based methods such as DQN (Mnih et al. 2015) require a discrete action space and cannot represent fractional positions without reintroducing a grid approximation. Deterministic actor-critic methods operate natively in continuous action spaces and are therefore more appropriate for position-sizing tasks.

Together these three properties point toward a model-free, off-policy, continuous-action algorithm — which is the class to which TD3 belongs. This is a reasoned design choice, not a claim that TD3 is universally superior for trading. The MDP formulation remains approximate because market states are partially observable and non-stationary. The formal algorithmic properties of TD3 are derived in Chapter 3. The present section is concerned with how the broader RL literature has been applied to trading and what that literature implies for the design choices made in this thesis.

2.6. Trading Problem Formulation

This section identifies the reinforcement learning concepts most important for interpreting the finance literature reviewed in this chapter. Formal definitions, equations, and algorithm taxonomies are presented in Chapter 3; the focus here is on why each one is consequential for trading problems.

Figure 1: The agent-environment interaction loop in reinforcement learning.



At each step the agent observes state s_t and reward r_t, selects action a_t, and receives the next state s_{t+1} and reward r_{t+1} from the environment.

Portfolio decisions are sequential and path-dependent. This makes the MDP framework (Bellman 1957; Puterman 1994) the formal language for trading agents. However, financial environments only approximately satisfy standard MDP assumptions. Market regime shifts, structural breaks, and partial observability all contribute to this approximation.

For this review, the most useful RL concepts are not the formal machinery of Chapter 3, but the dimensions along which applied trading papers differ. These dimensions determine whether results from one study are comparable to another and whether they are relevant to the problem addressed in this thesis.

The first dimension is the formulation of the trading problem itself: what is treated as the state, what counts as an action, and what scalar reward is used to evaluate behavior. In trading applications, these choices vary materially across studies. Some papers define the state using bar-level prices and technical indicators, while others incorporate current holdings, order-book information, or broader market context. Likewise, actions may represent a discrete buy/sell/hold decision or a continuous exposure level. Reward definitions also differ: some studies optimize raw return, others net return after transaction costs, and others risk-adjusted criteria such as Sharpe-oriented objectives (Moody and Saffell 1998; Ng et al. 1999). These

modeling choices are not cosmetic. They change the economic meaning of the task the algorithm is solving.

The second dimension is action-space structure. Discrete-action methods such as Q-learning and its deep variants are naturally aligned with buy/sell/hold formulations (Watkins 1989; Mnih et al. 2015). Continuous-control methods, by contrast, are designed for problems in which the agent chooses a portfolio weight or exposure level on a continuum. This distinction is central in the trading literature because it separates papers that treat trading as a classification-like decision problem from papers that treat it as a position-sizing problem. For the latter class, policy-gradient and actor-critic methods are usually the relevant algorithmic family (Williams 1992; Sutton et al. 2000).

The third dimension is the training regime, especially the distinction between on-policy and off-policy learning. In financial applications this matters not only for sample efficiency but also for the practicality of learning from historical data. Off-policy methods can in principle reuse stored experience through replay-based updates (Lin 1992), which is one reason they appear frequently in trading research where live experimentation is expensive or constrained. At the same time, the literature makes clear that training on historical market data does not eliminate instability, because financial environments remain noisy, non-stationary, and only approximately Markov (Tsitsiklis and Van Roy 1997; Deng et al. 2017). Algorithm choice therefore cannot be separated from the validation protocol under which performance is assessed.

The fourth dimension is reward design. Financial RL studies differ not only in which algorithm they use, but in what behavior they incentivize. Rewards based on raw price changes can encourage excessive turnover or sensitivity to noise, whereas cost-aware or risk-adjusted rewards aim to align optimization more closely with economically meaningful outcomes. The Differential Sharpe Ratio is one influential example (Moody and Saffell 1998), but the broader point is that reported performance is difficult to interpret without understanding the objective under which the policy was trained.

These distinctions provide the lens for reading the applied RL studies reviewed in the next section. Chapter 3 develops the formal reinforcement learning framework and the algorithmic lineage leading to TD3.

2.7. Applied RL Trading Evidence

The application of reinforcement learning to financial trading has evolved from theoretical proposals to practical implementations across diverse market segments. This section surveys the literature on RL-based trading systems through the design dimensions most relevant to the present thesis: state representation, action-space structure, reward definition, transaction-cost modeling, and validation protocol.

To keep conclusions methodologically defensible, this review distinguishes between: (i) proof-of-concept evidence from backtests under simplified assumptions, and (ii) stronger evidence that includes robust out-of-sample design, realistic cost modeling, and variance reporting across repeated runs. The chronology below is therefore secondary to the methodological comparison: reported performance is interpreted skeptically unless the study specifies what information the agent observes, how actions map to trades, whether costs are charged, and how out-of-sample performance is selected and reported.

2.7.1. Early Applications and Proof-of-Concept

Moody and Saffell (1998) presented the first comprehensive application of RL to trading, introducing recurrent reinforcement learning for optimizing trading systems. His approach incorporated four design choices that remain influential: the Differential Sharpe Ratio as a risk-adjusted reward, a recurrent network architecture for temporal state encoding, direct policy optimization rather than a predict-then-trade pipeline, and explicit transaction cost modeling. Moody demonstrated that RL could outperform supervised learning baselines on S&P 500 futures, establishing feasibility of the approach and showing that optimizing risk-adjusted return directly was often superior to predicting intermediate outcomes.

Moody and Saffell (2001) extended this framework to multi-asset portfolios, validating that RL-optimized policies could generate positive risk-adjusted returns after transaction costs — a stringent test of practical viability.

2.7.2. From Discrete to Continuous Action Spaces

Early value-based approaches applied Q-learning with discrete buy/sell/hold actions to equity and futures markets (Neuneier 1998; Lee et al. 2007). These studies established that RL could learn profitable patterns from historical data, but also identified persistent problems:

sparse rewards, delayed credit assignment, and out-of-sample performance degradation. More fundamentally, discrete action spaces force an artificial granularity on position sizing — a coarse grid loses control precision while a fine grid reintroduces the curse of dimensionality.

Gold (2003) and Dempster and Romahi (2002) extended the direct and recurrent reinforcement learning tradition to foreign-exchange trading systems. Their relevance is that the trading policy maps market state directly into positions rather than into an intermediate price forecast. The evidence is also cautionary: performance varied substantially across currency pairs and depended on the choice of model and system parameters, making early direct-RL results better interpreted as feasibility evidence than as a robust trading edge.

2.7.3. Deep Reinforcement Learning Era

The introduction of deep function approximators to RL trading extended applicability to high-dimensional state spaces.

Xiong et al. (2018) and Deng et al. (2017) adapted DQN for equity and cryptocurrency trading, demonstrating that convolutional networks could extract useful features from raw price data. However, both studies noted high variance across runs — a characteristic instability of value-based methods under noisy financial rewards.

Jiang et al. (2017) proposed the portfolio-vector-memory (PVM) architecture, combining policy gradients with recurrent networks for cryptocurrency portfolio management. Their fully differentiable end-to-end design incorporated current holdings into the state and penalized transaction costs explicitly, outperforming Markowitz mean-variance allocation in backtests across 12 cryptocurrencies.

An earlier cryptocurrency portfolio study by Jiang and Liang (2017) used a convolutional network to map historical prices directly into continuous portfolio weights under a deterministic policy-gradient-style objective. Its 30-minute Poloniex backtests are relevant because they treat trading as direct portfolio-action optimization, but the assumptions of immediate execution and no market impact remain far from tick-level order-book trading.

Z. Liang et al. (2018) applied DDPG to stock trading and conducted ablation studies showing that: (i) DDPG outperformed DQN for continuous position sizing, (ii) Sharpe-based rewards improved out-of-sample generalization, and (iii) transaction cost modeling was critical for realistic performance assessment.

Majidi et al. (2024) provided a focused TD3 study on single-asset trading for AMZN and BTC using daily data and a continuous action space in $[-1, 1]$. Their minimal state representation — lagged close-return windows rather than large technical-indicator sets — reported improved return and Sharpe ratio for TD3 versus discrete variants and buy-and-hold, though results were mixed across assets. This study is best interpreted as evidence that continuous action sizing can be beneficial under realistic frictions, not as definitive proof of robust alpha across market regimes.

Kabbani and Duman (2022) proposed a TD3-based framework incorporating account state (cash and holdings), price features, technical indicators, and optional FinBERT-derived news sentiment. Richer state designs improved both return and Sharpe ratio relative to a close-price-only baseline, with a final 10-stock evaluation reporting a Sharpe ratio of 2.68. This figure warrants scrutiny: annualized Sharpe ratios above 2 are rare in live trading and typically indicate either insufficient out-of-sample discipline, favorable selection among multiple model variants, or evaluation periods that coincide with a strong directional trend. The study does not report results across multiple random seeds or under realistic spread-crossing costs with a frozen policy, making it difficult to determine whether the reported performance reflects genuine signal or overfitting to the evaluation window. Methodologically the paper supports the practical value of feature enrichment in continuous-action actor-critic trading, but it also illustrates a broader pattern in the RL trading literature: without standardized evaluation protocols — multi-seed reporting, held-out test periods not used for model selection, and explicit cost assumptions — performance claims are difficult to assess on their own terms.

Yang et al. (2020) evaluated an ensemble stock-trading strategy built from PPO, A2C, and DDPG on Dow Jones constituents. The study is relevant as an actor-critic comparison in a liquid equity universe, but it is not evidence about millisecond-level LOB trading; its state representation, trading frequency, and execution assumptions differ from the setting studied in the current work.

Wang et al. (2019) proposed AlphaStock, which combines a Sharpe-oriented RL objective with history-state and cross-asset attention networks. Its contribution is mainly about representation and interpretability in stock selection: the attention mechanism gives a partial account of which asset characteristics drive portfolio construction. This supports the broader claim that state representation matters materially in RL trading, but it does not address order-book-driven single-asset exposure control.

Zhang et al. (2020) provide a broader review of deep RL trading systems, emphasizing that algorithm choice cannot be separated from state representation, action design, reward definition, transaction costs, and evaluation protocol. The review is useful here as a synthesis of design dimensions rather than as evidence for the superiority of any single algorithm.

2.7.4. Comparative Evidence and Limitations

Algorithm comparisons and reproducibility. Liu et al. (2020) illustrates a later move toward reproducible DRL trading pipelines by implementing several standard algorithms, including DQN, DDPG, PPO, SAC, A2C, and TD3, within common market environments and backtesting workflows. Its relevance is infrastructural rather than evidentiary: it shows that applied trading studies increasingly require standardized environments, comparable baselines, and explicit market-friction assumptions before algorithm rankings can be interpreted.

López de Prado (2018) compared deep RL against traditional quantitative strategies (momentum, mean reversion, pairs trading), noting that RL exhibited superior adaptability during regime changes while traditional strategies offered better interpretability. Hybrid approaches combining domain knowledge with RL often outperformed either in isolation.

Overfitting and generalization. Krauss et al. (2017) demonstrated that many published RL trading strategies fail to generalize out-of-sample, attributing this to high noise-to-signal ratios in financial data, limited training data relative to model complexity, and non-stationarity that invalidates training-period patterns. The implication is that evaluation methodology — strict temporal splits, multi-seed reporting, realistic cost assumptions — matters as much as algorithm choice.

Reward specification. Millea (2021) emphasizes that reward shaping is one of the central design choices in DRL trading. Misspecified rewards can lead to metric gaming, excessive turnover, ignoring tail risks, or misalignment with actual investor preferences. The Differential Sharpe Ratio (Moody and Saffell 1998) addresses some of these concerns by targeting risk-adjusted return directly, but does not fully resolve the tension between reward simplicity and the richness of real trading objectives.

The literature reviewed here points to several patterns relevant to the design choices in this thesis. Across the representative studies discussed above, continuous-action actor-critic methods are frequently preferred over discrete-action alternatives for position-sizing tasks (Z.

Liang et al. 2018; Majidi et al. 2024; Kabbani and Duman 2022). Risk-adjusted reward signals, especially Differential Sharpe Ratio formulations, also appear promising for out-of-sample generalization relative to raw return objectives, though such claims remain highly sensitive to the evaluation protocol and cost assumptions. Explicit transaction cost modeling is a prerequisite for any realistic performance claim at the frequencies considered here.

Viewed by design dimension, the strongest lessons are narrower than the headline returns reported in many papers. State representations must include the information needed for the action being optimized; action spaces should match the economic decision, especially when the task is exposure sizing; rewards must penalize costs and avoid encouraging excessive turnover; and validation must use held-out periods, frozen policies, and repeated runs where stochastic training is material. Without these elements, a positive backtest is best interpreted as a proof of concept rather than evidence of a robust trading edge.

Within the representative literature reviewed here, a comparatively underexplored design combination is the joint use of order-book-derived state variables, continuous-control RL, and an explicit microstructure-oriented state design. Much of the applied literature relies primarily on OHLCV or other aggregated price representations, while studies using deeper limit-order-book information remain fewer and methodologically heterogeneous. The present thesis is positioned within that less represented part of the literature by constructing the agent's observation vector from 10-level order book data and by evaluating a continuous-control agent under the modeling assumptions stated in later chapters.

2.8. Implications for This Thesis

The literature reviewed in this chapter motivates the thesis design along five dimensions. First, market microstructure theory and evidence support the use of LOB-derived state variables rather than relying only on bar-level price summaries. Spread, imbalance, order-flow, fair-value, and intraday-regime features each correspond to a distinct mechanism discussed in the microstructure literature.

Second, the reviewed evidence also limits the interpretation of these features. They are candidate state variables, not guaranteed trading signals. Their value depends on whether they provide information available before the agent acts and whether any resulting policy remains useful after transaction costs and out-of-sample evaluation.

Third, the comparison with supervised and rule-based approaches motivates direct optimization of trading decisions. Forecasting models can be informative, but a separate rule is still required to translate predictions into exposure. In this thesis, the agent instead chooses a continuous portfolio position directly, making the action space part of the optimization problem.

Fourth, the applied RL literature motivates the use of continuous-control actor-critic methods and risk-aware reward design, while also requiring caution about reported performance. Prior studies show that RL trading results are sensitive to transaction costs, benchmark choice, random seeds, and validation protocol. For that reason, the later empirical chapters separate training reward from financial evaluation metrics and assess the learned policy on held-out data.

Finally, the contribution of the thesis follows from the combination of these design choices: a microstructure-aware state representation, continuous exposure control, transaction-cost-aware evaluation, and explicit robustness assessment within a controlled single-asset HFT simulation. The objective is not to claim a deployable trading system, but to test whether this particular design combination can produce credible out-of-sample evidence under the stated simulation assumptions.

3. Reinforcement Learning

This chapter develops the reinforcement learning foundations required for the trading agent implemented here. It is organized in three parts. The first part introduces the core components of an RL system — the formal vocabulary of states, actions, rewards, policies, and value functions that all subsequent algorithms inherit. Each component is presented with trading motivation: the text explains why these concepts are challenging or distinctive in financial environments, while the detailed implementation choices (specific features, exact reward formulation, network architectures) are deferred to Chapter 4. The second part classifies RL algorithms along the dimensions that matter most for the trading problem: model-free vs. model-based, and on-policy vs. off-policy. The third part traces the algorithmic lineage from early value-based methods to TD3, showing how these properties converge on TD3 as the algorithm of choice for continuous-control trading.

3.1. Components of a Reinforcement Learning System

Reinforcement learning systems address sequential decision-making problems by selecting actions that maximize cumulative discounted future rewards. This subsection defines the core components using trading as the primary illustrative context, drawing on Sutton and Barto (2018).

Formally, the trading problem can be described as an approximate Markov decision process $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the transition kernel, $R(s, a, s')$ is the reward function, and γ is the discount factor. In financial markets, this formulation is necessarily an approximation. The agent observes engineered order-book features rather than the full market state, so the empirical problem is better understood as a partially observed decision problem represented through an approximate MDP. Relevant information may be omitted because of hidden liquidity, unobserved order-flow history, latent intraday regimes, queue priority, and other market participants' private decisions. The transition process is also affected by non-stationarity and intraday seasonality. The state representation used in this thesis should therefore be understood as an attempt to construct a useful decision state under partial observability, not as proof that the market itself is fully Markovian.

Environment. The environment defines the space of possible states and the rules governing

transitions between them. In the trading setting studied here, the environment is the simulated order-book market: it receives a portfolio-exposure action from the agent at each time step, updates the portfolio value according to price changes and transaction costs, and returns the new state and reward. The environment encapsulates everything outside the agent's direct control — price dynamics, spread, and execution frictions represented by the simulator — under a set of explicit simulation assumptions.

State. The state s_t is a snapshot of the environment at time t that contains sufficient information for the agent to select an action. In the HFT setting, the state is a vector of engineered microstructure features derived from the limit order book: imbalance measures, spread, order flow signals, and temporal encodings. States can be terminal, ending an episode, or non-terminal, continuing the interaction sequence. The Markov property requires that the optimal action depend only on s_t and not on the full history; the feature engineering in Chapter 4 is designed to approximate this property.

State representation is particularly challenging in financial markets because the true market state is only partially observable—agents lack information about hidden liquidity, other participants' private orders, and latent regime shifts. The engineered state vector described here is therefore necessarily an approximation, and Chapter 4.2 discusses how specific microstructure features are selected and normalized to construct a useful decision state under partial observability and non-stationarity.

Action. The action a_t is the decision the agent takes given state s_t. In this thesis the action space is continuous: $a_t \in [-1, 1]$, representing the target portfolio exposure from maximum short to maximum long. The continuous formulation is important because it allows the agent to express varying degrees of conviction through position size, rather than being restricted to a binary buy/sell decision.

Policy. The policy $\mu_\phi : \mathcal{S} \to \mathcal{A}$ maps states to actions. A deterministic policy is used — given the same state, the policy always produces the same action. A deterministic policy is chosen here because the evaluation objective is a single exposure decision per state, and consistency is important for interpretability and execution in deployed trading systems. During training, exploration noise is added to the policy output to encourage discovery of profitable behaviors; at evaluation time the policy is used without noise.

The choice between deterministic and stochastic policies involves a trade-off that is particularly

salient in trading environments. Stochastic policies can provide more flexible exploration and robustness to uncertainty, but they require selecting between sampling or taking the mode at deployment time. Deterministic policies eliminate this ambiguity but require explicit exploration mechanisms. Chapter 4.5 compares both approaches (PPO's stochastic policy vs. DDPG/TD3's deterministic policy) in the context of the trading problem.

Reward. The reward r_t is a scalar signal returned by the environment after the agent takes action a_t in state s_t. It provides the only learning signal available to the agent — there are no labels, no supervisor. The reward function is therefore a critical design choice: it determines what the agent actually optimizes. In trading, rewards are typically derived from portfolio returns, possibly adjusted for risk and transaction costs. The specific reward formulation used here is developed in Chapter 4.

Reward misspecification is a fundamental risk in trading applications. A naive reward function that optimizes raw returns without accounting for risk, transaction costs, or tail risk can lead to pathological behaviors such as excessive turnover, overconcentration in volatile positions, or ignoring rare but catastrophic events. The Differential Sharpe Ratio formulation used in Chapter 4.3 addresses this by providing a risk-adjusted learning signal that penalizes volatility while remaining recursively computable for online learning.

Value Function. The value function $V^\pi(s)$ estimates the expected cumulative discounted reward starting from state s under policy π:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \,\middle|\, s_t = s \right] \tag{3}$$

where: - $V^\pi(s)$ — expected cumulative discounted return starting from state s under policy π - $\gamma \in [0, 1]$ — discount factor - r_{t+k} — reward received k steps after t - $\mathbb{E}_\pi$ — expectation over trajectories generated by following π from $s_t = s$

The value function allows the agent to reason about long-run consequences rather than only immediate rewards — a necessary capability in trading, where the cost of a position change today affects future returns through transaction costs and portfolio path dependence. The action-value function $Q^\pi(s, a)$ extends this to state-action pairs and forms the basis of the critic networks in actor-critic algorithms. This cumulative discounted reward term appears explicitly in the RL objective in Equation 18.

3.2. Classifying RL Algorithms

The diagram below maps the reinforcement learning landscape from the Bellman equation through the key algorithmic relaxations that lead to TD3. It is intended as an orientation before the detailed treatment that follows.

Source: author's own diagram, based on Sutton and Barto (2018).[5]

3.2.1. Model-free vs Model-based

Reinforcement learning algorithms divide into two broad families depending on whether they require a model of the environment.

Model-based methods learn or are given a transition model $P(s' \mid s, a)$ and use it to plan ahead — computing value functions by rolling out simulated trajectories rather than purely from observed data. Dyna-Q and AlphaZero are canonical examples. When the model is accurate, planning is data-efficient; when the model is wrong, planning amplifies errors. In financial markets, transition dynamics are unknown and non-stationary, making a reliable model practically unattainable.

Model-free methods learn directly from sampled transitions without building a transition model. They are less sample-efficient, and they still rely on the sampled experience being informative for the target environment and on the function approximator generalizing usefully across states and actions. TD3 belongs to this family.

3.2.2. Value-Based, Policy-Based, and Actor-Critic

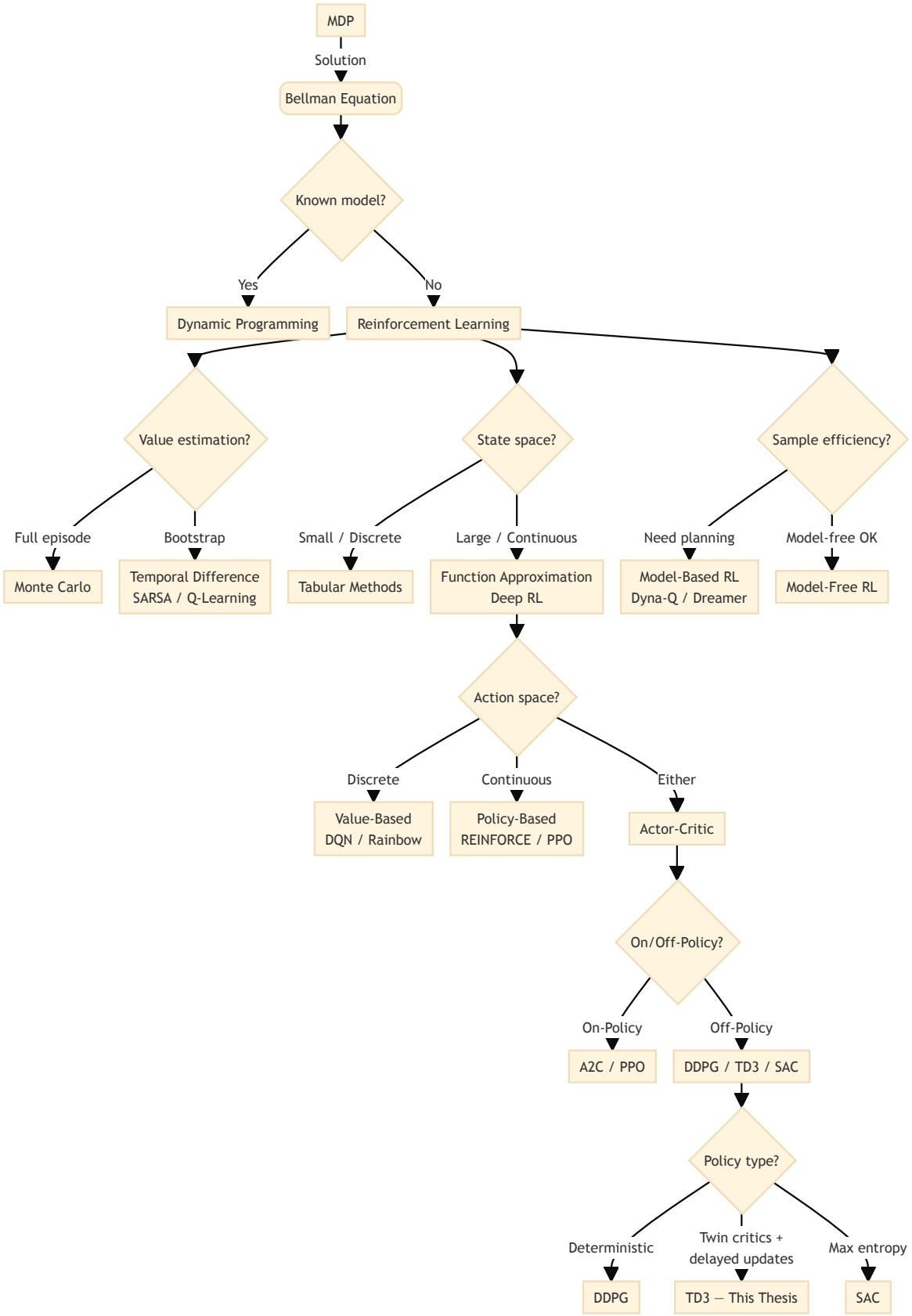
A second dimension classifies algorithms by what they learn and optimize.

Value-based methods learn a value function and derive a policy implicitly by acting greedily with respect to it. Q-learning and its deep variant DQN (Mnih et al. (2015)) are prototypical examples. They work well in discrete action spaces but struggle when actions are continuous, because the greedy step $\arg\max_a Q(s, a)$ becomes intractable over an uncountable set.

Policy-based methods directly parameterize and optimize the policy π_θ. REINFORCE

[5]This taxonomy is a pedagogical simplification, not a precise classification. Categories have fuzzy boundaries: PPO appears under both "Policy-Based" and "Actor-Critic" because it combines a stochastic policy with a value-based critic; DDPG, TD3, and SAC share an "Off-Policy" branch despite fundamental differences in policy type and objective. Readers should not infer mutually exclusive algorithmic families from the crisp boundaries shown here.

Figure 2: RL algorithm taxonomy from Bellman equation to TD3



(Williams (1992)) updates parameters by gradient ascent on expected return:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \mathbb{E}_{\pi_\theta} \left[\sum_t r_t \right] \tag{4}$$

where:

- θ — policy parameters
- $\alpha > 0$ — learning rate
- ∇_θ — gradient with respect to θ
- r_t — reward at step t

Policy gradients handle continuous actions naturally but suffer from high variance, requiring many samples to obtain a reliable gradient signal. This variance problem motivates the actor-critic architecture, which introduces a critic network to provide lower-variance value estimates.

Actor-critic methods combine both: an actor μ_ϕ selects actions and a critic $Q_\theta(s, a)$ evaluates them, using the critic's lower-variance estimates to guide actor updates. The full actor-critic derivation and TD3's specific extensions are given in Section 3.3.

3.2.3. On-Policy vs Off-Policy

On-policy methods evaluate and improve the same policy used to collect experience. SARSA is a canonical example, updating with:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \tag{5}$$

where:

- $Q(s, a)$ — action-value function
- $\alpha > 0$ — learning rate
- r_t — reward received after taking action a_t in state s_t
- $\gamma \in [0, 1]$ — discount factor
- a_{t+1} — next action drawn from the current policy

On-policy methods tend to be stable but wasteful: each sample is used once and then discarded.

Off-policy methods decouple the data-collection policy (the behavior policy) from the policy being improved (the target policy). Q-learning updates with the greedy maximum:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \tag{6}$$

where:

- $\max_a Q(s_{t+1}, a)$ — greedy value of the next state, taking the maximum over all possible actions a
- remaining symbols are as in Equation 5

This is contrasted with SARSA's update in Equation 5, where the next action is sampled from the current policy rather than taking the maximum.

regardless of how a_{t+1} was actually selected. This separation allows experience from a replay buffer — collected under earlier or exploratory policies — to be reused many times. In a financial simulator, however, replay does not create counterfactual observations of how the market would have evolved under actions not actually taken. It is valid only under the maintained simulation assumption that the agent is price-taking and that its exposure choice affects portfolio wealth but not the subsequent order-book path. TD3 is off-policy and uses a replay buffer to reuse collected transitions under this simulator design; the implications for sample efficiency and execution assumptions in financial settings are discussed in Section 3.3.

3.3. Actor-Critic Methods for Continuous Control

The development of reinforcement learning algorithms is a series of conceptual breaks, each motivated by a limitation in the preceding framework. This section traces the specific lineage that leads to the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm, which is employed here. The survey is not exhaustive; it focuses on the algorithmic properties that are directly relevant to the continuous-action, single-asset trading problem formulated in Chapter 4.

3.3.1. Value-Based Methods and Their Limitations in Continuous Control

The foundational algorithms of the field — Q-learning (Watkins and Dayan 1992) and its on-policy variant SARSA (Sutton and Barto 2018) — operate over tabular representations

of state-action value functions. Both methods converge to the optimal policy under mild conditions, but only when the state-action space is small enough to enumerate explicitly. In a trading context with continuous price-derived features, the state space is effectively infinite, and a tabular representation is computationally infeasible.

Deep Q-Networks (DQN) (Mnih et al. 2015) resolved the scalability problem by replacing the lookup table with a deep neural network, enabling generalization across states. Two architectural innovations were critical: experience replay, which decorrelates training samples by storing and randomly resampling past transitions, and a periodically frozen target network, which stabilizes the Bellman update by preventing the target from shifting at every gradient step. DQN achieved human-level control across a wide range of tasks, but it inherits a structural constraint from Q-learning: it requires a discrete action space, because the max operator in the Bellman target presupposes that all actions can be enumerated. For a trading agent that outputs a continuous portfolio weight in $[-1, 1]$, this is not an acceptable limitation. Discretizing the action space into a finite grid resolves the representation issue formally, but it reintroduces the granularity problem: a coarse grid loses the benefits of continuous control, while a fine grid makes the maximization step computationally expensive and recovers the curse of dimensionality in higher-dimensional portfolio settings.

3.3.2. Policy Gradient Methods and the Actor-Critic Architecture

A distinct class of algorithms avoids the maximization problem by parameterizing the policy directly and optimizing it through gradient ascent on expected cumulative reward. The foundational result is due to Williams (1992), who introduced REINFORCE, and Sutton et al. (2000), who established the policy gradient theorem in the general function approximation setting. The gradient takes the form:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot G_t] \tag{7}$$

where:

- $J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_t r_t]$ — expected return objective
- $\pi_\theta(a_t \mid s_t)$ — probability of action a_t under the current policy
- $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ — discounted return from step t
- $\nabla_\theta \log \pi_\theta$ — score function (log-policy gradient)

This generalizes the simpler form shown in Equation 4, which presents the parameter update rule directly rather than deriving it from the gradient of the objective. While REINFORCE applies naturally to continuous action spaces through a parameterized distribution (for example, a Gaussian policy), it suffers from high variance in the gradient estimate because G_t is computed from a full episode trajectory, making it sensitive to trajectory length and reward scale.

The actor-critic architecture reduces this variance by replacing G_t with a learned advantage estimate $A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$, maintained by a separate critic network. The actor updates the policy in the direction of higher-valued actions, while the critic provides a lower-variance baseline. This separation is the architectural foundation of all subsequent continuous-action algorithms, including TD3, which instantiates this architecture with deterministic policies and twin critics as described in Equation 23 and Equation 22.

3.3.3. DDPG: Deterministic Policies for Continuous Control

Silver et al. (2014) established the deterministic policy gradient (DPG) theorem, showing that for a deterministic policy $\mu_\phi : \mathcal{S} \to \mathcal{A}$ the policy gradient takes the form:[6]

$$\nabla_\phi J(\phi) = \mathbb{E}_s \Big[\nabla_a Q_\theta(s, a)\big|_{a=\mu_\phi(s)} \cdot \nabla_\phi \mu_\phi(s) \Big] \tag{8}$$

where:

- $J(\phi)$ — expected return
- $Q_\theta(s, a)$ — critic (action-value function) parameterized by θ
- $\mu_\phi(s)$ — deterministic actor
- $\nabla_a Q_\theta$ — gradient evaluated at the action produced by the current policy

This deterministic gradient is a specialization of the stochastic policy gradient in Equation 7 for policies that output a single action rather than a distribution. TD3 uses this gradient form as shown in Equation 23.

Lillicrap et al. (2016) then introduced Deep Deterministic Policy Gradient (DDPG), which applied the DPG theorem to deep neural network function approximators, replacing the

[6]The original Silver et al. (2014) paper uses θ for the actor and w for the critic; Lillicrap et al. (2016) uses θ^μ and θ^Q. This thesis adopts the Fujimoto et al. (2018) convention throughout — ϕ for the actor and θ for the critic(s) — to maintain consistency with the TD3 equations in Chapter 4.

tabular actor and critic used by Silver et al. (2014) with multi-layer networks. Rather than sampling an action from a distribution, the actor outputs a single action $\mu_\phi(s)$ directly, making the gradient computation tractable over a continuous action space without the intractable arg max required by value-based methods.

DDPG inherits the experience replay and target network mechanisms from DQN, making it an off-policy algorithm and therefore sample-efficient. However, Fujimoto et al. (2018) identify two failure modes in DDPG, both corroborated by independent reproducibility studies (Henderson et al. 2018). First, the critic tends to overestimate action values because actor optimization can exploit positive approximation errors in the critic, and bootstrapped targets can propagate those overestimated values through subsequent updates. Overestimated Q-values then push the policy toward actions that exploit the critic's errors rather than the true reward signal. Second, the algorithm is sensitive to the relative frequency of actor and critic updates: updating the actor before the critic has converged amplifies the overestimation problem. In financial environments, where rewards are noisy and non-stationary, one would expect these instabilities to be particularly acute: the critic must separate genuine market signal from approximation artifacts, and overestimated Q-values provide a corrupted learning signal under conditions where the true reward is already weakly informative.

3.3.4. PPO: On-Policy Actor-Critic with Clipped Objectives

Schulman et al. (2017) introduced Proximal Policy Optimization (PPO), an on-policy algorithm that balances the size of each policy update against training stability. PPO optimizes a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] \tag{9}$$

where:

- $\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ — probability ratio between the updated and old policies[7]
- $\hat{A}_t$ — advantage estimate
- ϵ — clipping hyperparameter (typically 0.2) limiting how far the new policy may deviate from the old one in a single update

[7]Schulman et al. (2017) uses $r_t(\theta)$ for this ratio, but that symbol is reserved throughout this thesis for the RL reward signal. $\rho_t(\theta)$ is used here to avoid the conflict.

This contrasts with TD3's approach to policy updates, which uses the deterministic gradient in Equation 8 and Equation 23 without explicit clipping constraints. PPO's clipping mechanism provides stability through objective constraints, while TD3 achieves stability through target smoothing and delayed updates.

The clipping mechanism prevents excessively large policy updates that would collapse performance, making PPO more robust to hyperparameter choices than earlier policy gradient methods such as Vanilla Policy Gradient (Sutton et al. 2000) and Trust Region Policy Optimization (TRPO) (Schulman et al. 2015). Unlike TD3 and DDPG, PPO is on-policy: it optimizes against the most recent policy and does not use a replay buffer. This trades sample efficiency for stability, which can be advantageous in environments where the value function is difficult to approximate.

PPO also maintains a stochastic policy rather than the deterministic policies used by DDPG and TD3. At each state, the policy outputs a distribution over actions (typically Gaussian), and actions are sampled from this distribution. Exploration is handled naturally through the entropy of the action distribution, which can be explicitly encouraged through an entropy bonus term in the objective:

$$L^{\text{CLIP+ENT}}(\theta) = L^{\text{CLIP}}(\theta) + c_1 \mathcal{H}(\pi_\theta) \tag{10}$$

where:

- $L^{\text{CLIP}}(\theta)$ — clipped surrogate objective from Equation 9
- $\mathcal{H}(\pi_\theta) = -\mathbb{E}[\log \pi_\theta(a \mid s)]$ — policy entropy
- $c_1 > 0$ — coefficient controlling the strength of the exploration bonus

This entropy-based exploration contrasts with TD3's additive noise approach described in Equation 28, where exploration is injected directly into deterministic actions.

For trading applications, PPO's stochastic policy provides built-in exploration without requiring explicit noise injection during training, and its clipped objective helps avoid catastrophic performance drops that can occur in high-variance financial environments. However, its on-policy nature means each transition is used only once before being discarded, which requires more simulator transitions for comparable learning. In the present thesis, where the training window is fixed to a specific historical period, this makes on-policy methods less efficient than

replay-based alternatives at extracting signal from a bounded simulation budget.

3.3.5. TD3: Addressing the Instability of Deterministic Actor-Critic Methods

Fujimoto et al. (2018) introduced TD3 as a direct response to DDPG's failure modes, proposing three targeted mechanisms.

Clipped double Q-learning. Two independent critic networks Q_{θ_1} and Q_{θ_2} are maintained, and the Bellman target is constructed using the minimum of their estimates:

$$y_t = r_t + \gamma \min_{i=1,2} Q_{\theta'_i}(s_{t+1}, \tilde{a}_{t+1}) \tag{11}$$

where:

- r_t — immediate reward
- γ — discount factor
- $Q_{\theta'_i}$ — two target critic networks with slowly updated parameters θ'_i
- s_{t+1} — next state
- $\tilde{a}_{t+1} = \mu_{\phi'}(s_{t+1}) + \epsilon_{\text{smooth}}$ — target action with smoothing noise ϵ_{smooth}

This differs from the standard Q-learning target in Equation 6 through two key modifications: (1) the minimum operator over two critics reduces overestimation bias, and (2) the deterministic target action $\tilde{a}'$ replaces the maximum over actions. The TD3 instantiation in Chapter 4.4 uses the same formulation, shown in Equation 22.

Taking the minimum provides a conservative estimate of target value and substantially reduces overestimation bias. This is analogous to the double Q-learning correction (Hasselt et al. 2016), adapted to the deterministic actor- critic setting.

Delayed policy updates. The actor and target networks are updated less frequently than the critics — typically once for every two critic updates. This allows the value estimates to stabilize before they are used to compute the policy gradient, breaking the feedback loop between an unstable critic and an erroneous actor update.

Target policy smoothing. When computing Bellman targets, noise is added to the target action $\tilde{a}_{t+1} = \mu_{\phi'}(s_{t+1}) + \epsilon_{\text{smooth}}$, with ϵ_{smooth} clipped to a small interval. This prevents the critics from fitting sharp value peaks caused by function approximation, acting as a regularizer

on the value landscape.

Together, these three modifications produce an algorithm that is substantially more reliable than DDPG on standard continuous control benchmarks (Fujimoto et al. 2018). Evidence from trading-adjacent settings is more limited and heterogeneous: Mohammadshafie et al. (2024) compare several deep RL strategies in asset-holding tasks and find TD3 competitive with alternatives; Gort et al. (2022) evaluate TD3 in cryptocurrency trading with explicit attention to backtest overfitting. These results should not be read as establishing a general ranking — they span different asset classes, reward formulations, and evaluation protocols — but they suggest TD3's stability mechanisms are not systematically harmful in financial settings. This matters for the present thesis for a specific reason: the agent is trained in a price-taking historical-market simulator where each transition can be replayed because the agent's action changes portfolio wealth but not the subsequent order-book path. Replay therefore improves computational use of the simulated experience, but only under this execution assumption; it should not be interpreted as evidence that the agent has observed all counterfactual market reactions to alternative trades.

TD3 is selected as the main algorithm on the basis of two prior constraints imposed by the thesis design and one design preference that interacts with those constraints.

The first constraint is that the action space is continuous: the agent outputs a target portfolio exposure in $[-1, 1]$, so discretized value-based methods such as DQN would require an arbitrary action grid and would make position sizing a modelling artifact rather than a learned decision. This eliminates the DQN family entirely and restricts the viable candidates to policy-gradient and actor-critic algorithms.

The second constraint is that the training window is bounded by a fixed historical period and live data collection is outside the scope of the thesis. Off-policy algorithms that reuse transitions through a replay buffer extract more signal from each simulated step than on-policy methods that discard transitions after a single gradient update. This constraint disadvantages PPO relative to DDPG and TD3, though it does not determine the choice between them.

Given these two constraints, the viable candidates are DDPG, TD3, and SAC (Haarnoja et al. 2018). Among these, DDPG and TD3 are preferred because they use natively deterministic policies, which aligns with the evaluation objective of a single exposure decision per state and avoids the need to choose between sampling from or taking the mode of a stochastic action

distribution at deployment.

SAC represents a distinct approach to continuous-control RL that warrants explicit discussion. Unlike TD3, which addresses instability through value-estimation corrections (twin critics and delayed updates), SAC formulates the problem as maximum entropy reinforcement learning: it optimizes both expected return and policy entropy simultaneously, treating exploration as an objective rather than a mechanism. This entropy-augmented framework has been applied to trading problems in the literature (Liu et al. 2020) and addresses the non-stationarity concern in a fundamentally different way—by encouraging policy diversity continuously rather than relying on off-policy replay from a fixed historical dataset.

SAC is excluded from the present study on methodological scope grounds, not because it lacks relevance. The entropy-temperature coefficient that balances return maximization against exploration introduces an additional objective dimension that is not present in the return-only formulation used here. Comparing SAC to TD3 would therefore require answering two questions: (1) does the entropy-augmented objective improve trading performance, and (2) is any improvement attributable to the maximum-entropy formulation or to the off-policy replay that both algorithms share? Disentangling these effects would require a controlled ablation that is outside the thesis's methodological focus, which is on value-estimation corrections for deterministic policies in historical simulators. SAC remains a promising direction for future work, particularly its online variants that adapt the entropy coefficient during training to address regime shifts in non-stationary markets.

Among the two remaining candidates, TD3 is preferred over DDPG because it was introduced precisely to correct DDPG's two main failure modes: critic overestimation and policy-update instability from premature actor updates. Both failure modes are expected to be acute when rewards are noisy and the critic must separate genuine market signal from approximation artifacts.

TD3 is therefore not chosen as a generally superior trading method; it is chosen as the algorithm whose structural properties — off-policy replay, deterministic actor, and explicit corrections for overestimation — most directly match the constraints and preferences of the simulation design used here.

The choice of TD3 should be read together with the trading constraints discussed in Chapter 2. Those sections motivate continuous exposure control, off-policy learning from historical data,

and caution about transaction costs, non-stationarity, and partial observability. TD3 matches the first two requirements through deterministic continuous actions and replay-based learning. It does not solve the remaining market-design problems. For that reason, the empirical chapters must still evaluate performance under explicit transaction costs, chronological validation, and the simulation assumptions stated in later sections.

3.3.6. Algorithm Comparison: PPO, DDPG, and TD3 for Trading

The three algorithms considered here differ along three dimensions that are particularly relevant to the continuous-action trading problem: learning paradigm, policy type, and stability mechanisms. These differences determine how to interpret empirical comparisons and why TD3 is selected over the alternatives.

On-policy vs. off-policy learning. DDPG and TD3 are off-policy algorithms that use experience replay, allowing each transition to be reused many times across multiple gradient updates. This is valuable when the simulation budget is bounded, as it is in this thesis by the fixed historical training window. Replay buffers also decorrelate consecutive transitions, reducing the variance of gradient estimates. PPO is on-policy and uses each transition only once before discarding it, requiring more training iterations for comparable sample efficiency. However, on-policy learning can be more stable when the environment distribution shifts rapidly, as financial markets do.

Deterministic vs. stochastic policies. DDPG and TD3 output deterministic actions $a = \mu_\phi(s)$, matching the evaluation objective of a single exposure decision per state. During training, exploration is handled by adding noise to the deterministic action: $a = \mu_\phi(s) + \epsilon$, with $\epsilon \sim \mathcal{N}(0, \sigma^2)$. At evaluation time, the noise is removed, yielding a consistent, interpretable policy. PPO maintains a stochastic policy that outputs a distribution over actions (typically Gaussian), with actions sampled from this distribution. Exploration is handled naturally through the entropy of the action distribution rather than explicit noise injection. Deterministic policies are preferable when the final deployed system must be consistent and interpretable, while stochastic policies can provide more flexible exploration during training.

Stability mechanisms. Each algorithm addresses stability challenges through different mechanisms. DDPG suffers from Q-value overestimation because the actor can exploit positive approximation errors in the critic, and these overestimates propagate through bootstrapped

Bellman updates. TD3 addresses this through clipped double Q-learning (taking the minimum of two critic estimates) and delayed policy updates (updating the actor less frequently than the critics). PPO prevents large policy changes through its clipped surrogate objective, which limits how much the policy can change in a single update. Both TD3 and PPO are more stable than vanilla DDPG, but through different mechanisms: TD3 targets value estimation errors, while PPO constrains policy changes directly.

Sample efficiency vs. stability trade-off. The practical implication of these differences is a trade-off between sample efficiency and training stability. DDPG and TD3 can make efficient use of limited training data through replay, but DDPG's instability can make it difficult to train reliably. PPO is typically more stable but requires more data to achieve comparable performance due to its on-policy nature. In the present simulation setup, where the training window is bounded by the historical period used, TD3's replay-based learning extracts more signal from each transition, making its combination of off-policy efficiency and stability mechanisms a strong fit for the thesis design.

Table 1: Algorithm properties for trading applications.

Property	PPO	DDPG	TD3
Policy Type	Stochastic	Deterministic	Deterministic
Learning Paradigm	On-policy	Off-policy (replay)	Off-policy (replay)
Exploration Mechanism	Entropy bonus	Action noise	Action noise + smoothing
Stability Mechanism	Clipped objective	Prone to overestimation	Double Q + delayed updates
Sample Efficiency	Moderate	High	High
Training Stability	High	Low	High
Hyperparameter Sensitivity	Low	High	Moderate
Suitability for Limited Data	Lower	Moderate[a]	High
Interpretability	Lower (stochastic)	High (deterministic)	High (deterministic)

Source: *Author's own synthesis based on (Fujimoto et al. 2018; Schulman et al. 2017; Lillicrap et al. 2016;*

Mohammadshafie et al. 2024). ***Note:*** *[a] DDPG's replay buffer provides high sample efficiency in principle, but its training instability (Henderson et al. 2018) undermines this advantage when data is limited.*

This comparison framework guides the experimental design in Chapter 6, where PPO, DDPG, and TD3 are evaluated on identical trading tasks to empirically assess how these algorithmic differences translate to trading performance. The results help determine whether TD3's theoretical advantages (reduced overestimation, deterministic policy) translate to practical benefits in the noisy, non-stationary trading environment, and whether PPO's on-policy stability can compensate for its lower sample efficiency.

4. Design of the Trading Agent

This chapter applies the reinforcement learning framework introduced in Chapter 3 to single-asset high-frequency trading. Each section addresses one component of the MDP formulation: the action space, state representation, reward function, value function, policy, discount factor, and step size. The choices made here are motivated by the structure and constraints of the trading problem.

4.1. Action Space

The action space defines the set of permissible decisions available to the trading agent at each time step. This design choice fundamentally shapes the agent's behavior and determines the granularity of control over portfolio positions.

4.1.1. Continuous vs Discrete Actions

Trading systems can be formulated with either discrete or continuous action spaces:

Discrete action spaces restrict the agent to a finite set of categorical decisions. The two most common formulations are the three-action set $\{-1, 0, 1\}$ representing short, neutral, and long positions, and the two-action set $\{0, 1\}$ representing only short and long positions without a neutral option.

Continuous action spaces allow the agent to output any value within a specified range, enabling smooth portfolio allocation decisions.

4.1.2. Implementation: Box Portfolio Action Space

In this implementation, the action space is defined as a continuous interval:

$$\mathcal{A} = [-1, 1] \tag{12}$$

This continuous action space is required for deterministic policy gradient methods like DDPG and TD3, which are designed for problems where the agent outputs a single action rather than choosing from a discrete set. The policy that maps states to actions in this space is formalized in Equation 25.

where the action $a_t \in \mathcal{A}$ at time t represents the target portfolio allocation for the traded asset. The semantic interpretation is:

- $a_t = 1.0$: Maximum long exposure (fully invested long position)
- $a_t = 0.0$: Neutral position (no exposure, cash equivalent)
- $a_t = -1.0$: Maximum short exposure (fully invested short position)
- $a_t \in (0, 1)$: Partial long position scaled by magnitude
- $a_t \in (-1, 0)$: Partial short position scaled by magnitude

This formulation supports short-selling and is a standard simplification in single-asset RL trading environments applied to equity data. The simulation treats long and short positions as cost-symmetric: the transaction cost term $c \cdot |a_t - a_{t-1}|$ applies identically regardless of direction. In practice, short positions in these equities incur securities-lending borrow costs — typically 25–75 basis points per year on the notional short — that are not captured here. For strictly intra-session episodes, the per-tick borrow cost is negligible: at 50 bps annualized and a 10-second average hold, the cost is on the order of 10^{-7} of notional. The cost-symmetry assumption is therefore inconsequential provided episodes do not span overnight. Where the agent holds short positions across sessions, reported returns represent an upper bound on achievable short-side performance. The continuous nature enables the agent to express varying degrees of conviction—for example, $a_t = 0.3$ indicates a moderately bullish stance, while $a_t = -0.8$ signals strong bearish conviction.

4.1.3. Action Constraints and Transaction Costs

The action space is bounded to prevent unrealistic leverage. In practice, unbounded actions could lead to positions exceeding available capital or margin requirements. The $[-1, 1]$ range implicitly assumes unit leverage without margin multipliers.

Transaction costs are applied proportionally to position changes. If the agent transitions from a_{t-1} to a_t, the turnover magnitude is $|a_t - a_{t-1}|$, and fees are calculated as:

$$\text{fee}_t = c \cdot |a_t - a_{t-1}| \cdot \text{notional value} \tag{13}$$

where:

- c — transaction cost rate (trading fees per unit of notional turnover), matching the

penalty term in Equation 17

This cost term appears in the reward function as the penalty $c \cdot |a_t - a_{t-1}|$ shown in Equation 17. The agent must learn to balance expected returns from position adjustments against this turnover cost. This incentivizes the agent to minimize unnecessary portfolio rebalancing, as frequent adjustments erode returns. During training, the RL algorithm must learn to balance the potential gains from reallocation against the friction costs incurred.

4.1.4. Action Smoothing and Exploration Noise

During training, exploration is introduced through additive Gaussian noise:

$$a_t^{\text{noisy}} = \text{clip}(a_t + \epsilon, -1, 1) \tag{14}$$

where:

- $\epsilon \sim \mathcal{N}(0, \sigma^2)$ — Gaussian exploration noise
- σ — exploration noise standard deviation[8]

This exploration mechanism is used by DDPG and TD3, as shown in Equation 27 and Equation 28. It contrasts with PPO's entropy-based exploration approach in Equation 10, where the stochastic policy itself provides action diversity. The clipping operation ensures actions remain within valid bounds. This exploration mechanism encourages the agent to discover diverse trading strategies during early training phases while gradually converging to deterministic exploitation as learning progresses.

Additionally, TD3 employs target policy smoothing during critic updates, adding noise to target actions when computing Bellman targets. This regularization technique prevents the critics from overfitting to narrow peaks in the value function caused by function approximation errors.

4.2. State Space

The state space defines the informational representation available to the agent at each decision point. At each time step t, the agent receives an observation vector $s_t \in \mathcal{S}$ constructed from

[8]The main experiment uses $\sigma = 0.3$. This value is at the upper end of the 0.1–0.3 range commonly used in continuous-action TD3 applications and was chosen to maintain sufficient action diversity in the low-signal tick-level setting.

the current state of the limit order book (LOB). The design of this observation is the primary engineering decision in the empirical pipeline: the agent can only exploit structure that is present in its input.

4.2.1. Design Requirements

An effective state representation for high-frequency trading must satisfy several criteria simultaneously.

Approximation of the Markov property. The TD3 algorithm assumes that s_t contains sufficient information such that the optimal action depends only on the current state, not on the full history. Raw price levels violate this assumption because their unconditional distribution is non-stationary; a price of 180 means something very different depending on recent trajectory. Engineered features — returns, imbalances, normalized spreads — are designed to be more stationary and to encode the local market state rather than the absolute price level.

Informativeness about short-term price direction. For the reward signal to be learnable, the observation must contain features that are predictive of near-term mid-price changes. The selection is therefore grounded in market microstructure theory, which identifies specific LOB quantities — imbalances, order flow, spread — as the empirically dominant predictors of short-term price movement.

Dimensionality discipline. Each additional observation dimension enlarges the input space that the policy and critic networks must generalize over. Features are included only when they carry a distinct theoretical mechanism not already captured by other features in the set.

Scope and timescale consistency. The observation is restricted to LOB-derived features. Bar-based signals such as OHLCV aggregates or technical indicators require a fixed aggregation window. This imposes a coarser timescale than the tick-level decisions being made.

For example, a one-second bar collapses dozens of individual order-book events into a single price summary. This discards the queue dynamics, imbalance shifts, and spread fluctuations that constitute the decision-relevant information at this resolution.

Cross-asset signals such as VIX or index returns face a related problem. They update at frequencies far below the inter-event times of the constituent tick streams. This contributes

no new information between consecutive book updates.

Aligning multiple tick streams at sub-second precision also introduces synchronization risk. A VIX observation and a LOB snapshot sharing a timestamp to the second are not the same market moment.

Beyond these practical constraints, restricting the state to single-asset LOB features is a deliberate experimental choice. The thesis tests whether microstructure-aware features alone are sufficient for a viable agent. Adding cross-asset signals would answer a different question and prevent clean attribution of any observed performance to LOB structure.

A natural extension for future work would be to augment the LOB observation with synchronized regime indicators. These could include intraday VIX futures at the minute level, sector ETF order flow, or options-derived skew.

This would be done once the baseline LOB-only performance is established. It would allow a controlled ablation of which additional signal sources materially improve out-of-sample results.

4.2.2. Feature Groups and Selection Rationale

The feature registry organises the LOB candidate pool into five groups, each targeting a distinct microstructure mechanism. **Imbalance features** measure the relative weight of resting liquidity on each side of the book — at individual levels (levels 0–2) and in a three-level volume-weighted aggregate — together with order count imbalance, which is sensitive to fragmentation that a single large institutional order would mask (Cao et al. 2004; Biais et al. 1995). **Fair value features** estimate where the next transaction is likely to occur relative to the mid-price: the microprice (Stoikov 2018) is the central construct, complemented by its divergence from mid-price and a volume-weighted mid-price skew across three levels. **Spread and cost features** capture adverse selection risk through the bid-ask spread in basis points, its ratio to a 50-event rolling mean, book convexity on each side, and depth slopes that proxy market impact per unit of volume (Kyle 1985; Glosten and Milgrom 1985; Bouchaud et al. 2002). **Order flow features** measure net directional pressure from LOB events between consecutive snapshots via order flow imbalance (Cont et al. 2014) and a 50-event rolling sum, queue depletion on each side, signed trade flow from the tape (Lee and Ready 1991), and odd-lot trade ratio and imbalance, which separate retail and algorithmic small-lot activity from institutional flow and thereby signal changes in adverse selection risk

(Boehmer et al. 2021).[9] **Regime and temporal features** condition the agent on activity rate via log inter-event time (Engle 2000), on price momentum via mid-price acceleration, and on the intraday seasonal pattern via cyclical sine and cosine hour encodings (Wood et al. 1985). The final selected TD3 configuration uses a reduced subset of this candidate pool: best-level book pressure, three-level order-book imbalance, best-level order-count imbalance, microprice, microprice divergence, bid and ask depth slopes, instantaneous OFI, 50-event rolling OFI, and 50-event signed trade flow. The observation vector is additionally extended at runtime with the agent's current portfolio exposure $a_{t-1} \in [-1, 1]$, making the state partially Markovian with respect to inventory and allowing the agent to condition position changes on the transaction cost they would incur; this feature is excluded from z-score normalization because its bimodal distribution — concentrating near ± 1 under the continuous action space — makes standardization counterproductive relative to the raw bounded signal.

The candidate pool from which these groups were drawn contained over forty computable statistics. Features eliminated by the IC/ICIR filter or excluded on redundancy grounds include: VPIN (Easley et al. 2012), which proved collinear with rolling OFI after conditional screening; cancel-to-trade ratio and trade arrival rate, which carried insufficient incremental ICIR beyond order count imbalance; OFI autocorrelation, whose predictive content was subsumed by the rolling OFI window; large trade ratio, which was redundant with the odd-lot decomposition; and depth ratio and price vamp, which did not survive the theory-constrained Stage 1 screen. The formulas for the conceptually central features are given in Table 2; the complete specification including all parameter values is in Appendix A.

Table 2: Key feature definitions. Full specification in Appendix A.

Feature	Group	Formula	Citation
Order book imbalance	Imbalance	$\text{OBI}_N = \frac{\sum_{i=0}^{N-1}(V_i^{bid} - V_i^{ask})}{\sum_{i=0}^{N-1}(V_i^{bid} + V_i^{ask})}$	Cao et al. (2004)

[9]Odd-lot share is computed from the February 25–27, 2026 MBP-10 dataset across AAPL, MSFT, TSLA, META, AMZN, and AVGO and should not be interpreted as a long-run population statistic. The share of odd-lot trades in large-cap Nasdaq equities has increased substantially since the SEC's 2023 amendments to odd-lot reporting rules. MBP-10 (Market By Price, 10 levels) is a hybrid order book format provided by Databento that sits between L2 (aggregated depth) and L3 (order-by-order) data: it captures depth-of-book snapshots at up to ten price levels and additionally records the order count at each level via `side_ct_xx` fields (e.g., `bid_ct_00` for the number of orders at the best bid). Schema documentation: https://databento.com/docs/schemas-and-data-formats/mbp-10

Feature	Group	Formula	Citation
Microprice	Fair value	$\frac{V_0^{ask} \cdot P_0^{bid} + V_0^{bid} \cdot P_0^{ask}}{V_0^{bid} + V_0^{ask}}$	Stoikov (2018)
Spread (bps)	Spread	$\frac{P_0^{ask} - P_0^{bid}}{\frac{1}{2}(P_0^{bid} + P_0^{ask})} \times 10{,}000$	Kyle (1985)
Bid/ask slope	Spread	$\frac{P_0^{bid} - P_4^{bid}}{\sum_{i=0}^{4} V_i^{bid}}$	Bouchaud et al. (2002)
Order flow imbalance	Order flow	$\Delta V_t^{bid} \cdot \mathbf{1}[P_{0,t}^{bid} \geq P_{0,t-1}^{bid}] - \Delta V_t^{ask} \cdot \mathbf{1}[P_{0,t}^{ask} \leq P_{0,t-1}^{ask}]$	Cont et al. (2014)
Signed trade flow	Order flow	$\sum_{\tau=t-W+1}^{t} \text{size}_\tau \cdot (\mathbf{1}[\text{side}_\tau = \text{B}] - \mathbf{1}[\text{side}_\tau = \text{A}])$	Lee and Ready (1991)

4.2.3. Observation Space and Normalization

The complete observation vector $s_t \in \mathbb{R}^d$ is formed by concatenating all engineered features. The main selected-feature TD3 experiment uses ten static LOB-derived features plus the runtime position feature, giving $d = 11$.

The ten static features selected in the main TD3 configuration are standardized with causal running statistics:

$$\tilde{x}_{i,t} = \frac{x_{i,t} - \mu_{i,t}}{\sqrt{\sigma_{i,t}^2 + \varepsilon}} \tag{15}$$

where:

- $x_{i,t}$ — raw value of feature i at step t
- $\mu_{i,t}$ — running mean estimated from observations up to t
- $\sigma_{i,t}^2$ — running variance estimated from observations up to t
- $\varepsilon > 0$ — small constant for numerical stability

The mean $\mu_{i,t}$ and variance $\sigma_{i,t}^2$ are updated online from observations available up to timestep t within the current chronological split and trading session. The running statistics are reset at detected session boundaries, preventing previous-day market conditions from contaminating

the opening representation of the next session. When cyclical encodings are used in other feature specifications, they are already bounded in $[-1, 1]$ and require no normalization. The position feature is likewise not standardized; it is already bounded in $[-1, 1]$ and its bimodal distribution makes z-score normalization counterproductive.

The formal observation space is:

$$\mathcal{S} = \mathbb{R}^d \tag{16}$$

In practice, after standardization, feature values concentrate in $[-3, 3]$ under approximate Gaussian assumptions. The policy and value networks must generalize to out-of-distribution observations that arise during stress episodes or regime shifts not present in the training sample.

4.2.4. On the Exclusion of Raw LOB Columns

Raw price and size columns — that is, the unprocessed bid and ask prices and sizes at each level — were deliberately excluded from the observation vector. Prior work in the deep learning tradition feeds the raw LOB matrix directly to the network (Zhang et al. 2019), allowing convolutional or recurrent architectures to discover structure from data without hand-engineered intermediaries. This is a coherent approach when the model itself is the feature extractor.

In the present setup, however, comprehensive microstructure features with established theoretical mechanisms are the primary input to the agent. Including raw columns alongside them introduces redundancy without adding an interpretable signal: normalized bid sizes at each level carry substantially the same information as the imbalance features derived from them, and normalized absolute prices at tick frequency are near-meaningless once the local market state is already encoded through microprice divergence, OFI, and spread measures. Mixing both paradigms would imply that the engineered features are insufficient, which is inconsistent with the selection rationale described above. The observation vector is therefore composed entirely of theoretically motivated derived features.

4.2.5. Feature Selection Process

Feature selection follows a two-stage procedure that combines theoretical constraints with empirical ranking.

Stage 1: Theory-constrained candidate pool. The LOB feature registry contains over forty computable statistics. The candidate pool is restricted to features that represent a distinct mechanism identified in the market microstructure literature. Each of the five groups — imbalance, fair value, spread, order flow, and regime — corresponds to a specific theoretical construct: queue pressure (Cao et al. 2004; Biais et al. 1995), fair value estimation (Stoikov 2018), adverse selection cost (Kyle 1985; Glosten and Milgrom 1985), directional flow (Cont et al. 2014; Lee and Ready 1991), and intraday regime context (Engle 2000; Wood et al. 1985). Features not grounded in this literature — raw price and size columns, or purely data-driven aggregates without a microstructure interpretation — are excluded from consideration regardless of their statistical properties. This stage prevents data-mining by ensuring that every candidate has a prior economic justification before any data is examined.

Stage 2: IC/ICIR ranking within the theory-constrained pool. Within each group, multiple implementations of the same theoretical concept exist: OFI at different rolling horizons, spread ratios at different windows, imbalance at different depth levels. Theory cannot resolve which implementations are informative or which combinations are redundant; this is an empirical question. Each candidate is scored by its Information Coefficient Information Ratio (ICIR) — the mean IC divided by the standard deviation of IC over rolling windows of the training split, where IC is the Spearman rank correlation between the feature and the forward log-return proxy. In the pooled multi-symbol design, scoring is performed independently for each of the 18 training files (6 securities $\times$ 3 trading days). The per-file scores are then averaged across files before the redundancy filter is applied, so the final ranking reflects cross-symbol, cross-day average predictive content rather than a single symbol's idiosyncratic regime. Scoring and the conditional IC redundancy filter both operate exclusively on the training files so that the evaluation day (March 2, 2026) remains clean for model comparison. The validation IC is computed separately and reported as an out-of-sample stability check but plays no role in selection. Redundancy within the selected set is controlled via a conditional IC filter: after each feature is selected, its linear signal is regressed out of remaining candidates, and ICIR is recomputed on the residuals. A candidate is included only if it retains incremental

predictive content beyond what the already-selected features collectively capture. The full procedure is described in Appendix F.

The result is a state space whose composition is jointly justified: every feature is grounded in theory (Stage 1) and empirically non-redundant within the theory-constrained pool (Stage 2). The two stages address different questions. Theory answers: which mechanisms belong in the state space? IC/ICIR answers: among the available implementations of those mechanisms, which combinations contribute distinct signal?

The selection is operationalized through a registry-driven feature pipeline. Feature names resolve to computation functions at runtime; adding, removing, or reordering features therefore changes the experimental specification without rewriting the feature implementations. The active feature set for the main experiment is the selected causal LOB set summarized in Appendix A: three imbalance features, two fair-value features, two depth-slope features, three order-flow features, and the runtime position feature. Regime and temporal features remain part of the implemented candidate registry but are not used in the selected TD3 specification.

The final state vector has dimensionality $d = 11$: ten LOB-derived features plus the current portfolio exposure appended at runtime. The ten static features are normalized using causal session-aware running statistics as described in the preceding section, while the position feature remains in its raw bounded scale. The complete feature specification with parameter values is listed in Appendix A.

4.3. Reward Function

The reward function determines what the agent actually optimizes. The thesis implements direct optimization of trading performance: the agent observes the market state, selects a continuous portfolio exposure $a_t \in [-1, 1]$, and receives a scalar reward reflecting realized return net of transaction costs. A baseline formulation of this reward is:

$$r_t = a_{t-1} \cdot \frac{P_t - P_{t-1}}{P_{t-1}} - c \cdot |a_t - a_{t-1}| \tag{17}$$

where:

- a_{t-1} — portfolio exposure held during the period
- P_t — price at the end of the period

- P_{t-1} — price at the beginning of the period
- $c \cdot |a_t - a_{t-1}|$ — transaction cost proportional to the size of the position change

The agent learns a deterministic policy $\mu_\phi : \mathcal{S} \to \mathcal{A}$ to maximize expected cumulative reward:

$$\max_{\phi} \mathbb{E}_{\mu_\phi} \left[\sum_{t=1}^{T} \gamma^{t-1} r_t \right] \tag{18}$$

where:

- ϕ — actor (policy) parameters being optimized
- T — episode horizon
- $\gamma \in [0, 1]$ — discount factor
- r_t — reward received at step t

This objective matches the standard RL formulation introduced in Equation 3, with the cumulative discounted reward as the quantity being optimized.

In trading, two properties are essential for the reward to work well in practice: the signal should be risk-adjusted so the agent does not maximize returns by taking excessive volatility, and it should be computable recursively so that no look-ahead into future returns is required at each training step. The following sections explain why the baseline formulation above requires refinement in a high-frequency setting, and how the Differential Sharpe Ratio addresses both requirements.

4.3.1. Why Raw Log-Return Is Insufficient

The simplest reward is the single-step log-return:

$$r_t = \log \frac{P_t}{P_{t-1}} \cdot a_{t-1} \tag{19}$$

where:

- P_t — mid-price at the end of the current step
- P_{t-1} — mid-price at the previous step
- $a_{t-1} \in [-1, 1]$ — portfolio exposure held during the period

This is the first term of Equation 17 without the transaction cost penalty. At tick frequency,

this signal is dominated by microstructure noise. Individual price changes on an MBP-10 order book are a fraction of the spread.[10] The agent cannot distinguish genuine directional movement from quote flickering in a single step.

Training on raw log-return produces highly variable gradient estimates. It also tends to reward high-turnover behavior regardless of risk.

A related alternative is the Sharpe ratio computed over a fixed rolling window. This approach requires storing the full window of past returns at each step. It also introduces a sharp discontinuity when old returns drop out of it.[11]

Neither property is desirable in an online RL setting.

4.3.2. Differential Sharpe Ratio

The Differential Sharpe Ratio (Moody and Saffell 1998) resolves both issues. It is defined as the first-order approximation of how the current portfolio log-return R_t changes the exponentially weighted Sharpe ratio:

$$D_t = \frac{B_{t-1}\,\Delta A_t - \frac{1}{2}A_{t-1}\,\Delta B_t}{\left(B_{t-1} - A_{t-1}^2 + \varepsilon\right)^{3/2}} \tag{20}$$

where:

- A_{t-1} — exponentially weighted estimate of the first moment of returns (defined in Equation 21)
- B_{t-1} — exponentially weighted estimate of the second moment of returns (defined in Equation 21)
- $\Delta A_t = R_t - A_{t-1}$ — one-step increment of A
- $\Delta B_t = R_t^2 - B_{t-1}$ — one-step increment of B
- $(B_{t-1} - A_{t-1}^2 + \varepsilon)^{3/2}$ — running variance raised to the power 3/2, stabilized by a small constant ε

[10] MBP-10 (Market By Price, 10 levels) is a hybrid order book format provided by Databento that sits between L2 (aggregated depth) and L3 (order-by-order) data: it captures depth-of-book snapshots at up to ten price levels and includes the order count at each level via `side_ct_xx` fields (e.g., `bid_ct_00` for the number of orders at the best bid). Schema documentation: https://databento.com/docs/schemas-and-data-formats/mbp-10

[11] When the oldest return in a fixed window is unusually large or small, its removal produces an abrupt shift in the estimated mean and variance — and therefore in the Sharpe ratio — that is unrelated to the current action. The agent cannot distinguish this accounting artifact from a genuine change in performance, introducing spurious gradient variance. Exponential weighting in the DSR avoids this because old returns decay continuously rather than dropping off a cliff.

This replaces the simple log-return in Equation 19 with a risk-adjusted measure that penalizes volatility. When DSR training is active, $r_t := D_t$ in Equation 18. The DSR reward signal addresses the high-variance problem that affects raw returns, though it does not eliminate the noise concerns at tick frequency.

A_t and B_t are exponentially weighted estimates of the first and second moments of returns:

$$A_t = \eta R_t + (1 - \eta)\, A_{t-1}, \qquad B_t = \eta R_t^2 + (1 - \eta)\, B_{t-1} \tag{21}$$

where:

- $R_t = \log(P_t / P_{t-1}) \cdot a_{t-1}$ — portfolio log-return at step t[12]
- $\eta \in (0, 1]$ — exponential smoothing parameter
- A_t — running mean of returns
- B_t — running mean of squared returns

At $t = 0$, $A_0 = B_0 = 0$.

The smoothing parameter η controls the effective memory of the estimator: larger η makes the estimate more reactive to recent returns, smaller η averages over a longer history. The effective window length is approximately $1/\eta$ steps. In the empirical experiments, $\eta = 0.01$, giving an effective memory of roughly 100 ticks — long enough to smooth microstructure noise while short enough to adapt to intraday regime shifts.

The DSR has two properties that make it well-suited to this setting. First, it penalizes return volatility intrinsically. A sequence of returns with the same mean but higher variance produces a lower DSR. This discourages the agent from achieving returns through high-variance position flipping.

Second, the DSR is recursively computable from A_{t-1} and B_{t-1} alone. This means no fixed window, no look-ahead, and no discontinuity as old observations expire.

A shaped reward such as DSR is used to stabilize learning. However, it is not equivalent to the financial performance metrics reported in Chapter 6.

[12] In the implementation, R_t is computed as $\log(\text{NLV}_t / \text{NLV}_{t-1})$, where NLV_t is the net liquidation value of the portfolio. This is equivalent to the formula above when transaction costs are zero. When costs are non-zero, the NLV-based return absorbs transaction costs implicitly, making the implementation more conservative than the analytic expression suggests.

The Sharpe ratio, annualized return, and drawdown statistics reported in the evaluation chapter are computed from the realized log-return series. These come from the deterministic policy rollout. They are not computed from the cumulative DSR accumulated during training.

Conflating the two would invalidate the empirical comparison. For example, treating a high training DSR as evidence of a strong Sharpe ratio on the test set would be incorrect.

The DSR approach has several limitations that should be acknowledged. First, it assumes that the first and second moments of returns are approximately stationary over the effective memory window ($\approx 1/\eta$ steps). In reality, intraday trading exhibits well-documented non-stationarity in both mean and variance due to market opening effects, scheduled announcements, and regime shifts (Engle 2000; Cont et al. 2014). While the exponential weighting does provide some adaptation, sudden structural breaks can temporarily destabilize the DSR signal during transition periods.

Second, the DSR inherits the fundamental limitation of variance-based risk measures: it treats upside and downside volatility symmetrically. Investors and traders typically care more about downside risk and tail events than about overall variance. Alternative risk-adjusted measures such as the Sortino ratio (which penalizes only downside deviation), the Calmar ratio (which normalizes returns by maximum drawdown), or utility-based rewards (which can incorporate asymmetric loss functions) would better align with investor preferences but are either non-recursive or computationally intractable for online learning at tick frequency.

Third, the DSR can be unstable when the denominator $(B_{t-1} - A_{t-1}^2 + \varepsilon)^{3/2}$ is small — either because the effective sample size is insufficient (early in training) or because realized variance is temporarily low. This can amplify microstructure noise and produce extreme reward values that destabilize gradient estimates. The implementation addresses this by clamping D_t to the interval $[-c, c]$ with $c = 10$, following the standard reward-clipping practice in deep RL (Mnih et al. 2015). The bound preserves the sign and relative ordering of rewards while preventing large gradient updates during low-variance regimes. DSR values outside this range occur rarely under normal market conditions and typically indicate numerical instability rather than genuine policy performance.[13]

[13]In the DSR scenario, the reward scale is $\kappa = 1$ — no additional rescaling is applied. At tick frequency, with $\varepsilon = 10^{-16}$, the denominator $(B_{t-1} - A_{t-1}^2 + \varepsilon)^{3/2}$ is dominated by the empirical variance term: for typical intraday tick-level returns with $\sigma \approx 4.5 \times 10^{-5}$, the variance is of order 2×10^{-9} and its 3/2 power is of order 10^{-13}, far exceeding $\varepsilon = 10^{-16}$. The resulting DSR signal is therefore approximately $R_t/\sigma \sim \mathcal{O}(1)$, which lies naturally in a well-conditioned range for the Adam optimiser and the smooth-L_1 (Huber) critic loss. Crucially,

Fourth, the DSR introduces a latent-state dependency that conflicts with the Markovian assumption required by off-policy algorithms such as TD3. The running moments A_t and B_t are not included in the agent's observation vector, so the reward at time t depends on the hidden trajectory of past returns through the episode. During training, TD3 stores transitions (s_t, a_t, r_t, s_{t+1}) in a replay buffer and samples them in random order; the stored reward $r_t = D_t$ was computed under the EMA state (A_{t-1}, B_{t-1}) prevailing at collection time, which differs from the EMA state at replay time. Consequently, the same state-action pair (s, a) can produce different reward values when retrieved from the buffer at different training epochs, violating the assumption that $p(r, s' \mid s, a)$ is stationary. This produces contradictory Q-value targets and slows convergence. The effect is most pronounced early in each episode, before the EMA moments have stabilized; after approximately $1/\eta$ steps the moments approach a regime-consistent steady state and the reward distribution becomes approximately stationary. A theoretically clean resolution is to augment the observation with (A_t, B_t), transforming the problem into a fully Markovian MDP; this is left as a direction for future work.

The DSR was chosen despite these limitations for three reasons. First, its recursive formulation makes it uniquely suitable for online RL: alternatives such as fixed-window Sharpe or utility-based rewards require storing unbounded history or solving optimization problems at each step, which is infeasible at tick frequency. Second, while variance is an imperfect proxy for risk, it provides a tractable baseline that discourages the most pathological high-turnover behaviors. Third, the empirical evaluation in Chapter 6 uses actual financial metrics (Sharpe ratio, drawdown, annualized return) computed from deterministic rollouts, so any systematic biases introduced by DSR's limitations will be visible in the final performance assessment rather than masked by the reward signal itself. The limitations chapter identifies reward ablation — comparing DSR to Sortino-based signals, direct log-return objectives, and drawdown-penalizing rewards — as a priority direction for future work.

4.4. Value Function

Chapter 3 introduced the state-value function $V^\pi(s)$ and action-value function $Q^\pi(s, a)$ as the expected cumulative discounted return under policy π. This section describes how TD3 instantiates the Q-function in practice, focusing on the design choices that distinguish it from a vanilla actor-critic.

this means no ad-hoc reward rescaling is required; the normalisation is an intrinsic property of the DSR formula once ε is chosen small enough that the empirical variance dominates the stabiliser.

4.4.1. Twin Critics

TD3 maintains two independent Q-networks Q_{θ_1} and Q_{θ_2}. The Bellman target is computed using the minimum of the two estimates:

$$y_t = r_t + \gamma \min_{i=1,2} Q_{\theta'_i}(s_{t+1}, \tilde{a}_{t+1}) \tag{22}$$

where:

- r_t — immediate reward
- γ — discount factor
- $Q_{\theta'_i}$ — two target critic networks with slowly updated parameters θ'_i
- s_{t+1} — next state
- $\tilde{a}_{t+1} = \mu_{\phi'}(s_{t+1}) + \epsilon_{\text{smooth}}$ — smoothed target action with $\epsilon_{\text{smooth}} \sim \mathcal{N}(0, \sigma^2)$ clipped to a small interval

Taking the minimum counteracts the overestimation bias that arises when bootstrapped targets are computed from the same network being updated — a failure mode documented in DDPG and addressed here following Fujimoto et al. (2018). This formulation extends the standard Q-learning target in Equation 6 by (1) using twin critics to reduce overestimation and (2) employing a deterministic target policy rather than taking a maximum over actions. The TD3 target was first introduced in Chapter 3 in Equation 11.

Both critics are trained by minimizing the TD error against the shared target y_t. The actor is updated using only the first critic's gradient:

$$\nabla_\phi J(\phi) = \mathbb{E}_{s\sim\mathcal{D}}\Big[\nabla_a Q_{\theta_1}(s,a)\big|_{a=\mu_\phi(s)} \cdot \nabla_\phi \mu_\phi(s)\Big] \tag{23}$$

where:

- ϕ — actor (policy) parameters being updated
- $J(\phi)$ — expected return
- $\mathcal{D}$ — replay buffer
- $Q_{\theta_1}(s,a)$ — first critic evaluated at the current policy action $\mu_\phi(s)$
- $\nabla_\phi \mu_\phi(s)$ — Jacobian of the policy with respect to its parameters

This is the deterministic policy gradient theorem from Silver et al. (2014), first presented in the general form in Equation 8. The gradient chain through the critic provides a lower-variance estimate of how policy parameters should change compared to the REINFORCE gradient in Equation 7.

The second critic contributes only through the conservative Bellman target, not directly to the policy gradient.

4.4.2. Loss Function

The critics are trained with Smooth L1 (Huber) loss rather than MSE:

$$\mathcal{L}(\delta) = \begin{cases} 0.5\,\delta^2, & |\delta| \leq 1, \\ |\delta| - 0.5, & \text{otherwise.} \end{cases} \tag{24}$$

where:

- $\delta = y_t - Q_\theta(s_t, a_t)$ — TD error measuring the discrepancy between predicted and bootstrapped value estimates

The linear tail limits the influence of large TD errors that occur during volatile market episodes, providing robustness that MSE does not. This loss function is applied to the TD error derived from the Bellman target in Equation 22.

4.5. Policy

The policy maps the current state to an action. The three algorithms compared here — PPO, DDPG, and TD3 — differ in a fundamental property of this mapping: PPO maintains a probability distribution over actions and samples from it, while DDPG and TD3 produce a single deterministic action for each state. This distinction has direct consequences for how each algorithm explores during training, how it behaves at evaluation time, and what the policy gradient looks like. The subsections below describe each architecture, followed by a table that makes the design choices explicit.

4.5.1. Deterministic Policy Architecture

DDPG and TD3 implement a deterministic policy:

$$\mu_\phi : \mathcal{S} \to \mathcal{A}, \qquad a_t = \mu_\phi(s_t) \tag{25}$$

where:

- $\mathcal{S} = \mathbb{R}^{11}$ — observation space for the selected-feature TD3 configuration
- $\mathcal{A} = [-1, 1]$ — continuous action space
- ϕ — network parameters
- s_t — state at step t
- a_t — resulting deterministic action

The policy is a multi-layer perceptron. The input is the $d = 11$-dimensional state vector. The network passes this through two hidden layers and produces a scalar action:

$$s_t \in \mathbb{R}^{11} \xrightarrow{\text{Linear}} \mathbb{R}^{128} \xrightarrow{\text{ReLU}} \mathbb{R}^{64} \xrightarrow{\text{ReLU}} \mathbb{R}^{1} \xrightarrow{\text{Tanh}} [-1,\, 1] \tag{26}$$

Hidden layers use ReLU activations, which avoid the vanishing-gradient problem that arises with saturating activations in deep hidden layers (Mnih et al. 2015). The output layer applies no activation before the Tanh squashing, which maps the pre-activation scalar to $(-1, 1)$. Tanh is preferred over a hard clip because it is differentiable everywhere: the policy gradient passes through the output non-linearity without discontinuity, and the smooth saturation near the action boundaries provides an implicit regularizer against extreme position changes.

The layer-by-layer parameter count is shown below; totals are computed from the experiment's hidden-layer widths and input dimension d.

Table 3: Actor network architecture.

Layer	Input dim	Output dim	Parameters
Linear + ReLU	11	128	1,536
Linear + ReLU	128	64	8,256
Linear + Tanh	64	1	65
Total			**9,857**

Note: *Input dimension d and hidden layer widths are loaded from the experiment snapshot; parameter counts are computed from those values.*

Exploration. Because deterministic policies do not generate action diversity on their own, DDPG and TD3 inject additive Gaussian noise during data collection and clip the perturbed action to the valid interval. The noise is removed at evaluation time: $a_t^{\text{eval}} = \mu_\phi(s_t)$. The specific exploration formulations for DDPG and TD3 are given in Equation 27 and Equation 28.

4.5.1.1. Deterministic Policy: DDPG

DDPG implements a deterministic policy:

Exploration. Because the policy is deterministic, it does not generate action diversity on its own. During data collection, additive Gaussian noise is injected:

$$a_t^{\text{train}} = \text{clip}\big(\mu_\phi(s_t) + \epsilon,\ -1,\ 1\big)\,, \qquad \epsilon \sim \mathcal{N}(0,\, \sigma^2) \tag{27}$$

where:

- $\mu_\phi(s_t)$ — deterministic policy output
- $\epsilon \sim \mathcal{N}(0, \sigma^2)$ — Gaussian exploration noise; the selected TD3 experiment uses $\sigma = 0.3$
- $\text{clip}(\cdot, -1, 1)$ — ensures the action remains in $[-1, 1]$

This matches the exploration mechanism described in Equation 14, where Gaussian noise is added to deterministic actions and clipped to the action space bounds. The noise is removed at evaluation time. TD3 uses the same exploration approach but adds additional target policy smoothing during critic updates, as shown in Equation 28.

The noise is removed at evaluation time: $a_t^{\text{eval}} = \mu_\phi(s_t)$.

4.5.1.2. Deterministic Policy: TD3

TD3 implements a deterministic policy:

Exploration. Because the policy is deterministic, it does not generate action diversity on its own. During data collection, additive Gaussian noise is injected:

$$a_t^{\text{train}} = \text{clip}\big(\mu_\phi(s_t) + \epsilon,\ -1,\ 1\big)\,, \qquad \epsilon \sim \mathcal{N}(0,\, \sigma^2), \quad \sigma = 0.3 \tag{28}$$

where:

- symbols are as in Equation 27
- $\sigma = 0.3$ — exploration noise standard deviation used during TD3 data collection

This is identical to DDPG's exploration in Equation 27 during training. However, TD3 adds a separate noise term for target policy smoothing when computing Bellman targets (as shown in Equation 22 and Equation 11), which prevents the critics from fitting sharp value peaks in the action space. This target smoothing noise $\epsilon_{\text{smooth}} \sim \mathcal{N}(0, 0.2^2)$ is clipped to $[-0.3, 0.3]$ and is distinct from the exploration noise.

The noise is removed at evaluation time: $a_t^{\text{eval}} = \mu_\phi(s_t)$.

TD3 adds a separate, independent noise term during critic updates for target policy smoothing ($\sigma_{\text{smooth}} = 0.2$, clipped to $[-0.3, 0.3]$). This is not exploration noise — it is a regularizer that prevents the critics from fitting sharp value peaks in the action space (Section 4.4). DDPG does not apply this smoothing; this is one of the three mechanisms that TD3 adds over DDPG.

Target actor. Both algorithms maintain a slowly updated copy of the actor, $\mu_{\phi'}$, used only when computing Bellman targets:

$$\phi' \leftarrow \tau\,\phi + (1-\tau)\,\phi', \qquad \tau = 0.005 \tag{29}$$

where:

- ϕ — online actor parameters
- ϕ' — target actor parameters
- $\tau = 0.005$ — soft-update coefficient controlling how quickly the target tracks the online network

The slow rate ensures the bootstrap target shifts gradually, even while the online actor receives more aggressive gradient updates. The target actor is never used for data collection or evaluation.

Architectural difference between DDPG and TD3. The actor networks for DDPG and TD3 are identical. The differences between the two algorithms lie entirely in the critic and training procedure: TD3 maintains twin critics and uses their minimum for conservative value targets, and delays actor updates to one per every two critic updates. DDPG uses a single critic and updates actor and critic at the same frequency. This makes DDPG the natural

ablation for TD3: it isolates the effect of TD3's stability mechanisms while holding the policy architecture constant. This makes DDPG a natural baseline; TD3 then adds three targeted mechanisms: clipped double Q-learning, delayed policy updates, and target policy smoothing.

4.5.2. Stochastic Policy: PPO

PPO maintains a stochastic policy that outputs a probability distribution over actions at each state:

$$\pi_\theta(\cdot \mid s_t) = \text{TanhNormal}(\mu_\theta(s_t),\, \sigma_\theta(s_t)) \tag{30}$$

where:

- $\mu_\theta(s_t)$ — state-dependent mean produced by the actor network
- $\sigma_\theta(s_t) > 0$ — state-dependent standard deviation produced by the actor network
- TanhNormal — Gaussian distribution whose samples are squashed through tanh to lie in $(-1, 1)$

The actor network shares the same hidden-layer structure as the deterministic actors — two layers of width [128, 64] — but its output head differs. The final linear layer produces $2n_{\text{act}} = 2$ values, which are split into a mean μ and a raw scale parameter $\log \sigma$ by a parameter extractor. The scale is transformed to a positive standard deviation via a softplus non-linearity, and a TanhNormal distribution is constructed:

$$s_t \in \mathbb{R}^{11} \xrightarrow{\text{Linear}} \mathbb{R}^{128} \xrightarrow{\text{Tanh}} \mathbb{R}^{64} \xrightarrow{\text{Tanh}} \mathbb{R}^{2} \xrightarrow{\text{split}} (\mu_\theta(s_t),\, \log \sigma_\theta(s_t)) \tag{31}$$

The resulting $(\mu_\theta(s_t), \log \sigma_\theta(s_t))$ pair parameterizes the TanhNormal policy distribution defined in Equation 30 above.

The PPO actor uses Tanh activations in the hidden layers, following standard practice for actors with stochastic output heads where bounded hidden representations improve numerical stability of the log-probability computation. The actor contains approximately 9,900 parameters (the output dimension doubles the final layer relative to the deterministic actors).

Exploration. Exploration is handled naturally through the entropy of the action distribution.

An entropy bonus $c_1 = 0.01$ is added to the PPO objective to encourage the policy to maintain action diversity during training. No explicit noise injection is required.

Evaluation. At evaluation time, stochastic sampling is replaced by the mode of the TanhNormal distribution, which is $\tanh(\mu_\theta(s_t))$. This yields a deterministic action for each state, directly comparable to the DDPG and TD3 evaluation policies.

Value network. Unlike DDPG and TD3, whose critics estimate $Q(s, a)$, PPO's critic estimates the state value $V_\phi(s)$.[14] The input is therefore the state vector alone (11 dimensions), not the state-action concatenation. The critic architecture is otherwise identical: two hidden layers of width [128, 64] with Tanh activations, followed by a linear output layer producing a scalar value estimate.

4.5.3. Controlled Comparison Design

All three algorithms are trained with the same state representation, reward function, data split, random seed, learning rates, and weight decay. The network capacity is equalized: both actor and value networks use hidden dimensions [128, 64] across all three algorithms. This ensures that differences in empirical performance in Chapter 6 are attributable to the algorithmic properties — policy type, update rule, and stability mechanisms — and not to differences in model capacity or input features.

The table below summarizes the policy-level design choices for each algorithm in the controlled comparison.

Table 4: Policy design choices for each algorithm in the controlled comparison.

Property	PPO	DDPG	TD3
Policy type	Stochastic	Deterministic	Deterministic
Actor output	TanhNormal(μ, σ)	$\mu_\phi(s) \in [-1, 1]$	$\mu_\phi(s) \in [-1, 1]$
Hidden dims	[128, 64]	[128, 64]	[128, 64]
Hidden activation	Tanh	ReLU	ReLU
Output activation	— (distribution)	Tanh	Tanh
Exploration	Entropy bonus	Additive Gaussian	Additive noise
	$c_1 = 0.01$	noise	$\sigma = 0.3$

[14] In the PPO context, ϕ parameterises the value network V_ϕ, whereas for DDPG and TD3 it parameterises the actor μ_ϕ. The PPO actor is parameterised by θ throughout.

Property	PPO	DDPG	TD3
Target smoothing	None	None	$\sigma = 0.2$, clip ± 0.3
Evaluation action	$\tanh(\mu_\theta(s_t))$	$\mu_\phi(s_t)$	$\mu_\phi(s_t)$
Critic input	s only	(s, a)	(s, a)
Number of critics	1	1	2 (twin)
Actor update freq.	Every epoch	Every step	Every 2 critic steps

4.6. Discount Factor

The discount factor $\gamma = 0.9$ is used in the main TD3 HFT experiment. The role of γ in the value function is defined in Chapter 3; this section justifies the specific value chosen.

With $\gamma = 0.9$ the effective planning horizon is approximately $1/(1-\gamma) = 10$ steps. On the event-time HFT data used here, this short horizon intentionally emphasizes local order-book dynamics rather than distant price movements. This is appropriate for a Differential Sharpe Ratio reward computed from noisy tick-level portfolio changes, where long bootstrap horizons can make critic targets unstable.

$\gamma = 0.9$ is held constant for the main selected-feature TD3 specification to isolate the effect of the state representation and reward design. Sensitivity to γ remains a natural direction for future work, particularly whether higher values such as 0.95–0.995 improve performance when the objective is less local than the present HFT setup.

4.7. Optimization Hyperparameters

4.7.1. Learning Rates

Both actor and critic use Adam (Kingma and Ba 2014) with a learning rate of $\alpha = 0.0001$. This is deliberately conservative relative to the 3×10^{-4} default common in continuous-control benchmarks. Preliminary experiments with more aggressive rates produced Q-value explosions and policy collapse — failure modes consistent with the high noise-to-signal ratio of tick-frequency rewards and the sensitivity of bootstrapped TD targets to large gradient steps. Stability was prioritized over sample efficiency.

Actor and critic use the same nominal rate, but TD3's policy delay (every 2 critic updates) effectively halves the actor's update frequency, providing additional stability without a separate

hyperparameter.

4.7.2. Weight Decay

L2 weight decay of $\lambda = 2 \times 10^{-6}$ is applied to the critic networks, while the actor uses no explicit weight decay. The critic regularizer is intentionally small — sufficient to discourage memorization of training-period noise without restricting the network's capacity to represent nonlinear market dynamics. No further regularization (dropout, batch normalization) is used, keeping the architecture parsimonious.

4.7.3. Scope

The reported learning rate and weight decay define the experimental settings used throughout the experiments. They were chosen to maintain stable critic targets and avoid policy collapse in preliminary runs, which makes them appropriate for the present high-frequency training setup. A broader hyperparameter study could refine these values further, but that analysis lies beyond the scope of the current empirical comparison.

5. Implementation of the Trading Agent

This chapter describes how the design choices from Chapter 4 are realized in the experimental pipeline. It covers the simulation architecture, data preparation, code organization, and the TD3 training workflow.

5.1. Simulation Architecture

The empirical workflow is organized through three functional layers that together implement a single-asset trading simulation.

Market data and feature construction. The agent observes states derived from high-frequency order-book data across six Nasdaq-listed equities (AAPL, MSFT, TSLA, META, AMZN, and AVGO). Raw order-book information is transformed into a vector of microstructure features. This layer is responsible for loading and validating the market dataset, preserving chronological ordering, constructing derived features, separating learning features from the price series used for portfolio valuation, and creating train, validation, and test splits without leakage (see Section 5.3.4). Reinforcement learning in financial settings is highly sensitive to timestamp errors and inconsistencies between engineered features and the price series used by the simulation.

Decision layer. The RL agent maps the current state representation to a bounded continuous action interpreted as target portfolio exposure. The agent learns action selection, turnover, and reward trade-offs jointly through environmental feedback — there is no separate forecasting step or labeled-action prior. The quality of the decision is evaluated through the reward process generated by the environment.

Execution and portfolio accounting. The agent's chosen action is translated into a portfolio update under explicit assumptions about pricing, transaction costs, and liquidity. This layer abstracts the operational complexity of a production trading stack — order management, venue selection, smart order routing, fill handling — into a controlled backtesting mechanism. If the agent performs well in this simulated setting, the result supports a claim about policy learning under the specified assumptions, not a claim that the strategy is immediately deployable without further execution modeling.

Simulation engine. The portfolio accounting layer is built on `tradingenv` (XAI Asset Management 2024), an open-source event-driven market simulator developed by XAI Asset

Management that implements the OpenAI Gymnasium protocol. The library models a broker with configurable transaction fees, handles continuous portfolio rebalancing, and records a full trade history for post-episode analysis. It was integrated into the training pipeline via a custom wrapper (`StreamingTradingEnvXY`) that streams fixed-length episode windows from memory-mapped data files, keeping peak memory proportional to episode length rather than dataset size. One performance modification was applied to the library: the duplicate-timestamp check in the internal `TrackRecord._checkpoint` method was overridden in a subclass to use an $O(1)$ dictionary lookup instead of an $O(n)$ linear list scan, which would otherwise degrade step time quadratically over long evaluation episodes.

5.2. Simulation Assumptions

The following assumptions govern the trading simulation throughout all experiments.

Zero market impact and no order-book interaction. Agent trades are assumed not to move quoted prices, consume displayed depth, or alter subsequent order-book states. The environment therefore treats the agent as price-taking rather than market-shaping. This is defensible only while position sizes remain small relative to displayed order-book depth, which holds for the MBP-10 dataset used here[15] : all six instruments — AAPL, MSFT, TSLA, META, AMZN, and AVGO — are highly liquid Nasdaq-listed blue-chips with tight spreads, deep displayed depth, and continuous two-sided quoting, ensuring that simulated agent trades represent a negligible fraction of resting book volume at any level.

Mid-price execution. Trades are valued at the mid-price rather than at executable bid/ask quotes. This avoids explicit spread-crossing and simplifies reward accounting, but it implies that reported backtest returns are an upper bound on achievable net performance. Real execution must cross the spread, and realized returns are further reduced by maker/taker fees, queue position, and partial fills.

Zero latency. Observations are assumed to be acted on immediately, with no transport, decision, or matching-engine delay between state observation and execution. This is appropriate for a study focused on strategy learning rather than low-latency execution engineering. In production HFT systems, such delays can invalidate short-lived signals.

[15]MBP-10 (Market By Price, 10 levels) is a hybrid order book format provided by Databento that sits between L2 (aggregated depth) and L3 (order-by-order) data: it captures depth-of-book snapshots at up to ten price levels and records the order count at each level via `side_ct_xx` fields (e.g., `bid_ct_00` for the number of orders at the best bid). Schema documentation: https://databento.com/docs/schemas-and-data-formats/mbp-10

Bounded continuous exposure. Position constraints are enforced via the continuous action interval $[-1, 1]$, preventing unbounded leverage.

Transaction fee. A configurable fee parameter is applied per position change. Baseline runs use a no-fee setting; sensitivity experiments vary this parameter.

Narrow data scope. Training, validation, and testing are conducted on a single three-day window of data (February 25–27, 2026) across the six instruments. This scope means the agent has only encountered one volatility regime, one spread environment, and one pattern of intraday seasonality. Results should be interpreted as a proof of concept rather than as evidence of generalizability across market conditions or time periods.

These assumptions collectively make the simulation tractable and the results interpretable as learning-dynamics benchmarks rather than deployment-ready performance estimates. The following components of a production trading system are explicitly outside the scope of this study: smart order routing across venues, partial fills and queue-position effects, execution algorithms such as VWAP or TWAP, detailed market-impact modeling, and production monitoring or brokerage integration. These omissions define the gap between the controlled experimental framework and deployment-ready trading-system performance.

5.3. Data Preparation

5.3.1. Asset Selection

The experiments use equity data from six Nasdaq-listed blue-chip instruments: AAPL (Apple Inc.), MSFT (Microsoft Corp.), TSLA (Tesla Inc.), META (Meta Platforms Inc.), AMZN (Amazon.com Inc.), and AVGO (Broadcom Inc.). All six are among the most liquid equities in US markets, with tight spreads, deep displayed depth, and continuous two-sided quoting throughout the trading session. The selection criterion was high but not extreme liquidity: instruments were chosen to ensure defensible price-taking assumptions and rich microstructure signals, while avoiding names whose order-book event density on the Nasdaq ITCH feed would be disproportionately inflated by factors unrelated to underlying economic activity. This liquidity profile ensures that the price-taking assumption embedded in the simulation — that agent trades do not move quoted prices — is empirically defensible, and that the microstructure features derived from the order book reflect genuine supply and demand rather than thin-market artifacts.

Blue-chip selection minimizes borrow costs, supporting the action space's short-selling capability. For these highly liquid large-cap equities, stock loan supply is abundant and borrowing is effectively unconstrained, with lending rates typically below 0.5% per annum.[16] This cost is negligible relative to transaction fees and expected high-frequency returns, justifying the symmetric long/short treatment in the simulation. Had the study focused on smaller-cap or illiquid instruments, borrow costs could reach 5–10% or more, rendering the symmetric cost assumption invalid.

The choice to train a single pooled model across multiple instruments, rather than a separate model per instrument, is a deliberate design decision. Securities within the same asset class that trade on the same exchange, under the same tick-size regime, and with comparable order-flow dynamics share a common microstructure regime. The residual cross-sectional variation — differences in volatility level, beta, or sector exposure — is dominated by this shared structure. Within such a homogeneous class, LOB feature distributions are comparable after standard normalization, making it feasible for a single policy network to learn market-wide order-book dynamics rather than instrument-specific quirks. Pooling across N securities multiplies the effective training sample by a factor of N, reducing the risk of overfitting to idiosyncratic patterns in any single stock's order-flow sequence. It also encourages the model to capture universal microstructure signals — spread dynamics, order-flow imbalance, and depth asymmetries — that transfer across equities within the same class. The feature pipeline supports this directly: all inputs are z-score normalized per security, and the microstructure features in the state space — order-flow imbalance, relative spread, depth slope, queue imbalance — are inherently relative measures that carry the same semantic meaning regardless of the underlying price level or absolute volume.

A further consideration is access symmetry. Because all six instruments trade on a centralized, regulated exchange with public pre-trade transparency, all market participants observe the same order book. This contrasts with OTC markets or less liquid instruments where information asymmetry and differential access could confound the learned signal. The goal is to isolate the contribution of the learning algorithm and feature engineering, not to exploit a structural information advantage.

[16]These rates apply to positions held overnight. Intraday short positions — those opened and closed within the same trading session — generally do not incur borrow costs, as the securities are returned before the daily settlement cycle.

The data were obtained from Nasdaq via the DataBento API, using the MBP-10 (Market By Price, 10 levels) feed — a hybrid order book format that sits between L2 (aggregated depth) and L3 (order-by-order) data, providing depth-of-book snapshots at up to ten price levels and additionally recording the order count at each level via `side_ct_xx` fields (e.g., `bid_ct_00` for the number of orders at the best bid).[17] This granularity is necessary for constructing the microstructure features described in Chapter 4 and listed in Appendix A.

5.3.2. Input Format

The pipeline consumes time-indexed LOB snapshot data. Each row represents an order-book event with multi-level bid/ask prices, sizes, and order counts, plus trade descriptors used for order-flow features. The core column groups are summarized below.

Table 5: Input schema used in the empirical pipeline.

Column group	Description	Role
`timestamp`	Chronological event index	Ordering, temporal features
`bid/ask_px_NN`	Bid/ask prices at level `NN`	Spread, mid-price, slope, valuation
`bid/ask_sz_NN`	Resting size at level `NN`	Imbalance, depth, OFI, queue features
`bid/ask_ct_NN`	Order count at level `NN`	Order count imbalance
`action`, `side`, `size`	Event type, aggressor side, size	Signed trade flow features
`close`	Mid-price-like series (auxiliary)	Reward and portfolio valuation

`NN` denotes the zero-indexed book level (00–09).

Table 6 shows four consecutive events from the AAPL feed at market open on 25 February 2026. The three events span roughly 9 milliseconds, illustrating the sub-millisecond cadence typical of the feed. Each row advances the state of the order book: the first event is a resting

[17] MBP-10 schema documentation: https://databento.com/docs/schemas-and-data-formats/mbp-10 For a technical reference on market data quality and L3 order book processing, see Nasdaq TotalView-ITCH specification: https://www.nasdaqtrader.com/content/technicalsupport/specifications/dataproducts/NQTVITCH-Specification.pdf

limit-order addition on the bid side at a non-top-of-book level; the second is a trade print that consumes displayed size at the best ask; the third and fourth are additions that refresh the ask side after the trade.

Table 6: Four consecutive AAPL MBP-10 events, 25 February 2026.

Timestamp (UTC)	Action	Side	Price ($)	Size	Best Bid ($)	Best Ask ($)	Bid Sz	Ask Sz
14:30:00.016	A	B	271.20	200	271.45	271.85	500	756
14:30:00.024	T	N	271.65	21	271.45	271.85	500	756
14:30:00.024	A	A	271.79	140	271.45	271.79	500	140
14:30:00.025	A	A	271.79	100	271.45	271.79	500	240

Action: A = limit-order addition, T = trade print. Side: B = bid, A = ask, N = no aggressor side (trade report). Best Bid/Ask and Bid Sz/Ask Sz reflect the level-0 (top-of-book) snapshot after each event is applied.

5.3.3. Preprocessing and Splitting

Raw data is validated for chronological order, required column presence, non-negative prices, and non-crossed best quotes before any feature engineering. This stage is conservative: its purpose is correctness, not statistical filtering.

Before feature engineering, each daily file is filtered to retain only events that modify at least one of the five shallowest book levels (levels 0–4). The full MBP-10 feed delivers updates at all ten price levels, but every microstructure feature in the state space draws exclusively on levels 0–4: order-flow imbalance, depth slopes, queue sizes, bid/ask slopes, and related signals are all defined relative to the top-of-book region. Events that update only levels 5–9 are therefore invisible to the feature computation; they produce consecutive observations with an identical observable state, inflating episode length and adding computational overhead without contributing any learnable signal. Including them would also implicitly penalise the agent for changes in book depth that it has no mechanism to detect or respond to. The filter is applied immediately after chronological validation and before any feature transformation, so the feature pipeline never encounters these no-signal events.

Table 7: Raw MBP-10 event counts and inter-event timing per symbol and split.

raw_file_inventory.json not found — run: uv run python src/cli.py peek dataset -s pooled/td3_hft_lob_state_space_pooled_streaming_selected –export

The dataset is organized as one parquet file per security per trading day, yielding 24 files in total (6 symbols × 4 trading days). The four days cover February 25–27, 2026 (days 1–3, training) and March 2, 2026 (day 4, evaluation). All files are filtered to US regular market hours (09:30–16:00 ET). Table 7 shows raw event counts and inter-event timing statistics per symbol and split before the level-0–4 filter is applied. Training counts aggregate all three training days per symbol; validation and test each represent one chronological half of the March 2 file.

Training data. The 18 files from days 1–3 are used exclusively for training. Each daily file is treated as a complete, independent training unit: the agent encounters all order-book events from that file in one episode. During training, the 18 files are served in uniformly shuffled random order across episodes, so the agent never sees a fixed day-order sequence and cannot exploit calendar artifacts.

Validation and test data. The 6 day-4 files (one per symbol) are held out entirely from training. Each file is split 50/50 at its chronological midpoint into a validation half and a test half; the six validation halves and six test halves are concatenated separately to form the pooled validation and test sets. This design ensures that evaluation data covers the same six securities but originates from a genuinely unseen trading session, providing a clean out-of-sample window. The validation split supports model comparison and development diagnostics; the test split is reserved for final out-of-sample reporting and does not influence any model selection decisions.

The four-day window spanning 25 February to 2 March 2026 is a recognized limitation of the current empirical design. The agent's learned behavior has not been exposed to different volatility regimes, spread environments, or intraday seasonality patterns beyond this specific period, and the generalizability of the results is bounded accordingly.

Feature pipeline fitting. For each security, the feature transformation pipeline (Section 5.3.4) is fitted on the concatenation of that security's three training-day files. The fitted pipeline is then applied independently to each training file and to the evaluation file. This per-security fitting ensures that the normalization statistics are computed exclusively from training-period

observations, with no leakage from the held-out evaluation day.

5.3.4. Feature Normalization and Causality Preservation

Normalization of microstructure features presents a methodological challenge for sequential learning: the conventional approach of fitting a standard scaler on the full training dataset and applying it globally introduces look-ahead bias. Each observation's normalized value would incorporate statistical information from future timesteps, violating the causal structure that a trading agent must respect in deployment. An agent at step t can only condition on information available at time t; allowing its inputs to be scaled using the mean and variance of the entire training set (including steps $t+1, t+2, ...$) would, in effect, provide the agent with knowledge it cannot access.

To preserve causality, this study employs a **cumulative running normalization** scheme based on Welford's online algorithm for computing sample mean and variance incrementally (Welford 1962). For each feature x observed at sequential timesteps $1, 2, ..., T$, the running statistics are updated as:

$$\bar{x}_t = \bar{x}_{t-1} + \frac{1}{t}(x_t - \bar{x}_{t-1}) \tag{32}$$

$$\sigma_t^2 = \sigma_{t-1}^2 + \frac{1}{t}[(x_t - \bar{x}_{t-1})^2 - \sigma_{t-1}^2] \tag{33}$$

where:

- $\bar{x}_t$ — running mean after observing t values
- σ_t^2 — running variance after observing t values
- x_t — current observation
- $\bar{x}_{t-1}$ — mean from the previous step

The normalized value at timestep t is then computed from the cumulative statistics available after observing the current feature value:

$$\tilde{x}_t = \frac{x_t - \bar{x}_t}{\sqrt{\sigma_t^2 + \varepsilon}} \tag{34}$$

where:

- $\bar{x}_t$ and σ_t^2 — defined by Equation 32 and Equation 33
- $\varepsilon > 0$ — small constant (here 10^{-4}) added to the denominator for numerical stability when variance is near zero

This single-feature presentation corresponds to $\mu_{i,t}$ and $\sigma_{i,t}^2$ in the multi-feature formulation of Equation 15, with the feature index i suppressed here for clarity.

This ensures that the scaling statistics used to normalize $\tilde{x}_t$ incorporate only observations $1, \ldots, t$ and no future values. Since x_t is part of the observation available before the action at time t is chosen, this transformation preserves the causal information structure while avoiding the instability that would arise from normalizing early-session observations with an almost empty denominator. The computational cost of this approach is $O(1)$ per update, making it efficient even for large datasets.

5.3.4.1. Session-Aware Normalization

The continuous-time nature of equity markets introduces regime discontinuities at session boundaries. Overnight and weekend price gaps, changes in order-flow patterns between the opening auction and continuous trading, and differences in liquidity across intraday periods all represent shifts in the underlying data-generating process. A naive cumulative normalization that spans these boundaries would allow statistics from the previous session to contaminate the current session's representation. For example, a 5% overnight price decline would make the opening trade appear as an extreme outlier when normalized against statistics that include the previous day's stable prices, even though such gaps are expected in continuous-time markets.

To address this, running statistics are **reset at session boundaries**. A session break is defined as a time gap exceeding one hour, which distinguishes short intraday interruptions (e.g., lunch recess) from genuine overnight or weekend closures. At each boundary, $\bar{x}$ and σ^2 are reinitialized to their default values ($\bar{x} = 0, \sigma^2 = 1$), and the accumulation process begins anew for the next session. This prevents the overnight gap from distorting the morning's normalization statistics and ensures that each trading session's features are scaled relative to that session's own empirical distribution.

5.3.4.2. Event-Time vs. Continuous-Time Weighting

An additional design choice concerns whether to weight observations by event count or by elapsed time. In high-frequency trading, events cluster during periods of high market activity: a burst of order-book updates over one second may contain 100 events, while a quiet 10-second interval may contain only one. An event-based normalization (the default in many standard RL libraries) would assign equal weight to each of the 100 clustered events and the one sparse event, effectively over-representing the high-activity period in the statistical summary. Continuous-time markets, however, are not sensitive to the number of events per se but to the duration over which each price or quote persists.

This implementation supports an optional **time-weighted normalization** mode, where each observation x_t is weighted by the elapsed time since the previous event, $\Delta t_t = t_t - t_{t-1}$. The weighted mean and variance are computed as:

$$\bar{x}_t^{\text{time}} = \frac{\sum_{i=1}^{t} \Delta t_i \cdot x_i}{\sum_{i=1}^{t} \Delta t_i} \tag{35}$$

$$\sigma_t^{2,\text{time}} = \frac{\sum_{i=1}^{t} \Delta t_i \cdot (x_i - \bar{x}_t^{\text{time}})^2}{\sum_{i=1}^{t} \Delta t_i} \tag{36}$$

When the time weights sum to the total elapsed time, the normalized value reflects the feature's deviation relative to its time-averaged trajectory rather than its average across discrete events. For the main experiments, event-based weighting is used as the default, matching the standard approach in the RL literature (e.g., Raffin et al. 2021; E. Liang et al. 2018). Sensitivity analysis with time-weighted normalization could be conducted as a direction for future work.

5.3.4.3. Split-Specific Statistics and Leakage Prevention

For causal running normalization, statistics are maintained **independently per security**, **per chronological split**, and **within each detected trading session**. The training, validation, and test subsets are transformed in chronological order, and at every timestep the scaler has access only to observations already encountered within that split and session. This separation serves two purposes. First, it prevents information leakage across the chronological split: statistics from the test period cannot influence the agent's training representation, and later validation or test observations cannot influence earlier ones. Second, it respects the per-symbol pooling design: each security's features are normalized relative to that security's

Table 8: Twelve consecutive AAPL LOB events and their transformed microstructure features. *Observation sample not found: run evaluate for AAPL to generate evaluation_data artifacts.*

own empirical microstructure rather than absolute magnitudes that differ across price levels or liquidity profiles.

Cyclical temporal encodings (hour-of-day sine and cosine) are already bounded in $[-1, 1]$ and are not subject to z-score normalization. Raw price columns used for portfolio accounting are kept in their original units and are never replaced by normalized values. This separation ensures that the observation features passed to the learning algorithm are statistically well-behaved, while the pricing inputs used for reward and valuation computation retain economic interpretability and correct accounting semantics.

5.3.4.4. Transformed Feature Example

To illustrate how the raw order book data is transformed into the input features used by the RL agent, Table 8 shows twelve consecutive events from the AAPL feed. Each row represents one order-book event; the left columns display the raw order book state (best bid/ask prices and sizes), while the right columns show the corresponding normalized microstructure features computed from that state.

The transformed features in Table 8 demonstrate several key properties of the feature engineering approach. First, **Book Pressure** and **Order Imbalance** capture instantaneous order-book liquidity and directional pressure: positive values indicate net buying pressure (bid depth dominates), while negative values signal selling pressure (visible in E12 after a bid-side refresh). Second, **Microprice Deviation** tracks how far the current microprice lies from the simple mid-price, providing a depth-weighted estimate of the fair transaction price. Third, **OFI** (Order Flow Imbalance) captures signed net trade flow over a short window, jumping sharply at events where displayed depth is consumed or replenished.

These features are computed from the raw order book data using the causal running normalization described in Equation 34. The normalization ensures that features are on comparable scales across different securities and time periods, while the causal structure guarantees that the agent never has access to future information when making decisions. The four features shown here (Book Pressure, Order Imbalance, Microprice Deviation, OFI) are a representative subset; the full feature set includes additional microstructure signals such as depth slopes,

Table 9

correlations.csv not found — run: uv run python src/cli.py peek -s pooled/td3_hft_lob_state_space_pooled_streaming_selected –corr –export

queue depletion rates, VPIN, and rolling order-flow statistics.

5.3.5. Feature–Return Correlations

Table 9 reports Pearson and Spearman rank correlations between each engineered feature and the one-step-ahead log mid-price return, computed on the training split. Both measures capture strictly pairwise, marginal dependence: Pearson detects linear association, while Spearman detects any monotone relationship — including non-linear but rank-preserving ones.

All absolute correlations are below 0.005. Neither measure identifies any feature whose marginal distribution is reliably aligned with the direction of the next price move. This is expected in a microstructure context: each feature captures a local structural signal — order-flow pressure, depth asymmetry, queue imbalance — that is informative about the trading environment but not predictive of short-term returns in isolation. Predictive value, if any, arises from the joint, non-linear interaction among multiple features simultaneously observed at the same timestep.

Linear models — including logistic regression, ridge regression, and any other predictor that combines features through a weighted sum — cannot exploit this joint non-linear structure. The near-zero Pearson and Spearman correlations confirm that linearly combining even the full feature vector is unlikely to yield a useful directional signal. This directly motivates the use of a deep neural network as the function approximator inside the RL policy: the network can, in principle, learn arbitrary non-linear mappings from the feature vector to action-value estimates, capturing the higher-order interactions that linear models cannot represent.

5.3.6. Feature and Price Column Separation

A strict separation is maintained between the observation features passed to the learning algorithm and the price inputs used by the portfolio simulation. The learning features are normalized and engineered; the pricing columns are economically interpretable and used only for reward and valuation computation. Mixing these two roles can produce invalid reward

Table 10: Training, validation, and test split sizes and timestamp ranges for the pooled six-asset LOB dataset.

splits.json not found — run: uv run python src/cli.py peek -c src/configs/scenarios/pooled/td3_hft_lob_state_space_pooled_streaming_selected/ –skip 500 –export

calculations or cause normalized prices to appear in financial metrics.

In the current implementation, the auxiliary pricing series used for portfolio valuation plays the role of a mid-price proxy rather than an executable bid/ask quote. This simplifies reward accounting but makes the resulting backtest performance optimistic relative to realistic spread-crossing execution.

5.3.7. Dataset Split Summary

Table 10 summarises the row counts and timestamp ranges of the training, validation, and test subsets for the pooled six-asset dataset used in the primary experiment. Row counts reflect the number of order-book events that pass the level-0–4 filter after chronological validation. The validation and test subsets are each bounded at 50,000 events by the configured `validation_size` and `test_size` parameters; the training subset comprises all remaining events from the three training days across the six instruments.

5.3.8. Feature Statistics

?@tbl-feature-stats reports distributional statistics for all engineered microstructure features computed over the training split, after skipping the first 500 events to allow rolling-window features to stabilise. Means close to zero and standard deviations near one confirm that the causal running normalisation is operating as intended. Q1, Q2, and Q3 show the inter-quartile spread; Skew and Kurt (excess kurtosis) characterise the shape of each marginal distribution.

Several features — most notably `ofi` and `ofi_rolling_50` — exhibit extreme minimum and maximum values despite the near-unit standard deviation. This is a structural property of order-flow imbalance, not a data quality issue. The order book is quiet the overwhelming majority of the time: most tick events represent small queue adjustments that produce z-scores close to zero. Occasionally, however, a large aggressive order consumes multiple price levels in rapid succession, causing a spike of hundreds of raw OFI units. The Welford running standard deviation is dominated by the quiet majority, so these rare spikes emerge as z-scores

of $±$290 or larger after normalisation. The large positive excess kurtosis visible for OFI features reflects this heavy-tailed structure.

The values in **?@tbl-feature-stats** are post-normalization but pre-clipping. The environment applies a hard observation clip of $[-5, 5]$ via the `obs_clip` parameter, so the policy network never receives the extreme tails shown here. The table therefore faithfully describes the prepared dataset, while the effective input range seen by the agent is bounded. This two-step design — normalize to make the typical range well-behaved, then clip to suppress the rare extreme events — is standard practice in continuous-control RL on financial microstructure data.

feature_stats.csv not found — run: uv run python src/cli.py peek -s pooled/td3_hft_lob_state_space_pooled_strea
ing_selected –skip 500 –export

5.4. Implementation and Training Pipeline

5.4.1. Experiment Specification Design

The full source code for this thesis is publicly available at Wojdalski (2026). Each experiment is fully specified by a structured set of inputs covering the dataset, feature subset, reward type, environment settings, and optimization hyperparameters. No source-code changes are required to switch between experimental variants. A pre-training validation stage checks path availability, feature definitions, split feasibility, and column compatibility before any computation begins, preventing wasted runs from invalid experimental specifications. A condensed summary of the main specification is provided in Appendix B. A high-level overview of the full pipeline — from data acquisition and offline feature research through preparation, pre-training configuration checks, training, and post-training evaluation — is provided in Appendix E; the complete machine-readable diagram is maintained alongside the source code in `docs/workflow.d2`.

5.4.2. Feature Pipeline

Feature engineering is registry-driven: feature names resolve to computation functions, allowing feature-set composition without editing training code. This design is central to systematic feature subset comparisons. Normalization parameters are estimated on the training split only for non-causal scalers. For the main HFT experiment, features use causal running

normalization: each split is transformed chronologically, and the normalization state is reset at detected session boundaries. This preserves causal integrity across both time and the chronological split.

5.4.3. Training Loop

The processed training split is used to construct the trading environment with a continuous action space $a_t \in [-1, 1]$ representing target portfolio exposure. Training follows the TD3 procedure described in Chapter 3, with comparable implementations for DDPG and PPO for experimental comparison. All three algorithms share a common training infrastructure: experience collection, network updates, and evaluation. The primary differences are in the update rules (deterministic policy gradient for DDPG/TD3, clipped surrogate for PPO) and replay buffer usage (off-policy for DDPG/TD3, on-policy for PPO).

For DDPG and TD3, experience is collected into a replay buffer, twin critics are updated from sampled mini-batches, and the actor is updated every 2 critic steps via the deterministic policy gradient. Target networks track the main networks through soft updates with coefficient τ. Exploration noise is applied during training and removed at evaluation time. For PPO, training follows the on-policy paradigm: batches are collected using the current policy, multiple epochs of updates are performed on each batch, and the clipped surrogate objective constrains policy changes. The stochastic policy handles exploration through entropy bonuses rather than explicit noise injection.

Algorithm 1 Common Experiment Setup

Require: Experiment specification (dataset, feature set, hyperparameters)

Ensure: Processed train/validation/test environments ready for algorithm-specific training

1: Load and validate experiment specification; check paths and split feasibility
2: Load L3 order book dataset; sort chronologically
3: Split data into train / validation / test subsets (chronological)
4: Initialise and validate feature pipeline on training split
5: Transform each split chronologically with causal session-aware normalization
6: Build training environment from processed training data

Algorithm 2 TD3 Training (follows Algorithm 0)

Require: Processed environments from Algorithm 0; TD3 hyperparameters

Ensure: Trained policy μ_ϕ, evaluation metrics

Initialise actor μ_ϕ, twin critics Q_{θ_1}, Q_{θ_2}, target networks $\mu_{\phi'}$, $Q_{\theta'_1}$, $Q_{\theta'_2}$, replay buffer $\mathcal{D}$

for each step $t = 1, \ldots,$ `max_steps` **do**

  $a_t \leftarrow \text{clip}\big(\mu_\phi(s_t) + \epsilon,\ -1,\ 1\big), \quad \epsilon \sim \mathcal{N}(0,\ \sigma^2_{\text{explore}})$

  Execute a_t; observe r_t, s_{t+1}; store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$

  Sample mini-batch $\mathcal{B} \sim \mathcal{D}$

  Compute smoothed target action: $\tilde{a}' \leftarrow \mu_{\phi'}(s') + \text{clip}(\mathcal{N}(0, \sigma^2_{\text{noise}}),\ \pm c)$

  Compute Bellman target: $y \leftarrow r + \gamma \cdot \min_{i=1,2} Q_{\theta'_i}(s',\ \tilde{a}')$

  Update both critics by minimising $\mathcal{L}_{\text{Smooth-L1}}\big(y - Q_{\theta_i}(s, a)\big)$

  if t mod `policy_delay` $= 0$ **then**

    Update actor: $\phi \leftarrow \phi + \alpha \nabla_\phi Q_{\theta_1}(s,\ \mu_\phi(s))$

    Soft-update targets: $\theta'_i \leftarrow \tau\theta_i + (1-\tau)\theta'_i;\ \phi' \leftarrow \tau\phi + (1-\tau)\phi'$

  end if

  Checkpoint and log metrics at configured intervals

end for

Evaluate deterministic policy μ_ϕ on test split

Compute financial performance metrics from realised return series

Algorithm 3 DDPG Training (follows Algorithm 0)

Require: Processed environments from Algorithm 0; DDPG hyperparameters

Ensure: Trained policy μ_ϕ, evaluation metrics

Initialise actor μ_ϕ, critic Q_θ, target networks $\mu_{\phi'}$, $Q_{\theta'}$, replay buffer $\mathcal{D}$

for each step $t = 1, \ldots,$ `max_steps` **do**

  $a_t \leftarrow \text{clip}\big(\mu_\phi(s_t) + \epsilon,\ -1,\ 1\big), \quad \epsilon \sim \mathcal{N}(0,\ \sigma^2_{\text{explore}})$

  Execute a_t; observe r_t, s_{t+1}; store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$

  Sample mini-batch $\mathcal{B} \sim \mathcal{D}$

  Compute Bellman target: $y \leftarrow r + \gamma \cdot Q_{\theta'}(s',\ \mu_{\phi'}(s'))$

  Update critic by minimising $\mathcal{L}_{\text{Smooth-L1}}(y - Q_\theta(s, a))$

  Update actor: $\phi \leftarrow \phi + \alpha \nabla_\phi Q_\theta(s,\ \mu_\phi(s))$

  Soft-update targets: $\theta' \leftarrow \tau\theta + (1-\tau)\theta';\ \phi' \leftarrow \tau\phi + (1-\tau)\phi'$

  Checkpoint and log metrics at configured intervals

end for

Evaluate deterministic policy μ_ϕ on test split

Compute financial performance metrics from realised return series

Algorithm 4 PPO Training (follows Algorithm 0)

Require: Processed environments from Algorithm 0; PPO hyperparameters

Ensure: Trained policy π_θ, evaluation metrics

Initialise stochastic actor π_θ (TanhNormal), value network V_ϕ
for each batch iteration $k = 1, \dots,$ `max_steps/frames_per_batch` **do**
    Collect trajectory $\mathcal{T}_k$ of `frames_per_batch` steps using π_θ
    Compute advantage estimates $\hat{A}_t$ via GAE using V_ϕ on $\mathcal{T}_k$
    for each PPO epoch $j = 1, \dots, K$ **do**
        Sample mini-batch $\mathcal{B} \sim \mathcal{T}_k$
        Compute probability ratio: $\rho_t(\theta) \leftarrow \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$
        Compute clipped objective: $\mathcal{L}^{\text{CLIP}} \leftarrow \mathbb{E}\Big[\min\Big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1 \pm \epsilon)\hat{A}_t\Big)\Big]$
        Compute value loss: $\mathcal{L}^V \leftarrow \mathcal{L}_{\text{L2}}\big(V_\phi(s) - V^{\text{target}}\big)$
        Compute entropy bonus: $\mathcal{L}^H \leftarrow c_1 \mathcal{H}(\pi_\theta(\cdot \mid s))$
        Update θ, ϕ jointly: minimise $-\mathcal{L}^{\text{CLIP}} + c_2 \mathcal{L}^V - \mathcal{L}^H$
    end for
    Checkpoint and log metrics at configured intervals
end for
Evaluate policy mode $\tanh(\mu_\theta(s))$ on test split
Compute financial performance metrics from realised return series

The notation follows the conventions established in Chapter 3:

For DDPG and TD3:

- μ_ϕ is the deterministic actor
- Q_θ (DDPG) and Q_{θ_1}, Q_{θ_2} (TD3) are the critic networks
- Primed symbols denote slowly updated target networks
- $\mathcal{D}$ is the replay buffer
- $\epsilon \sim \mathcal{N}(0, \sigma^2_{\text{explore}})$ is training-only exploration noise
- τ is the soft-update coefficient

For PPO:

- π_θ is the stochastic actor (TanhNormal distribution)
- V_ϕ is the state value network

- $\hat{A}_t$ are GAE advantage estimates
- $\rho_t(\theta)$ is the probability ratio between new and old policies
- ϵ is the clipping threshold
- c_1, c_2 are the entropy and value loss coefficients
- K is the number of PPO epochs per batch

Common to all algorithms:

- γ is the discount factor
- α is the learning rate

Concrete values are listed in Appendix B.

5.4.4. Sanity Checks

Before running the empirical order-book experiments, the RL pipeline was tested on a synthetic sine-wave scenario with deliberately learnable structure.[18] The purpose of this check was not to evaluate trading performance, but to verify that the environment, reward computation, replay buffer, neural networks, and training loop were capable of recovering a known pattern when one was present. In this setting, the TD3 agent learned behavior consistent with the imposed sine-wave structure, providing a basic implementation sanity check before moving to noisy tick-level order-book data.

A complementary check was conducted using a random policy — one that selects actions uniformly at random with no learned component. This tests the opposite question: whether the environment, reward function, and accounting logic are free of structural bias that would generate returns even without a signal. The random policy produced approximately zero return, consistent with an unbiased implementation. Results for the random policy on the empirical dataset are reported in Chapter 6.

Beyond the ad-hoc checks above, a suite of automated tests guards the implementation against the failure modes most common in RL codebases. Learning health checks verify, after a single gradient update, that observations and rewards are finite, that every feature dimension varies across the batch (i.e., no collapsed or constant-valued feature), that the loss is finite, and that actor and critic parameters both change and remain finite — four conditions

[18] The sanity-check run can be reproduced from the project root with: `uv run python src/cli.py train -c src/configs/scenarios/sine_wave/td3_hft_lob_future_oracle_combined/`.

that are individually obvious but together rule out the silent errors most likely to produce plausible-looking training curves without genuine learning. Pipeline integration tests run a short end-to-end training loop to detect regressions in data loading, environment construction, or replay buffer interactions. These tests execute automatically as part of the development workflow, ensuring that changes to any component of the pipeline do not silently invalidate the experimental conditions that produced the reported results.

5.4.5. Evaluation

Final performance is reported on the held-out test split using a dedicated evaluation environment. Two output streams are kept strictly separate. First is the training reward, which serves as an optimization diagnostic. Second is the realized portfolio return series from which financial metrics are computed.

Conflating these would invalidate the empirical comparison. For example, treating cumulative DSR as a Sharpe ratio estimate would be incorrect. Metrics derived from the test rollout are described in Chapter 6.

6. Results

This chapter reports the evidence for the thesis-facing experiment: the TD3 agent trained on high-frequency order-book-derived features from six Nasdaq equities (AAPL, MSFT, TSLA, META, AMZN, and AVGO) under the state-space design defined in Chapters 4 and 5. It is organized in three parts: statistical validation of repeated-run variability, robustness assessment under nearby design changes, and final performance evaluation on the held-out split.

6.1. Algorithm Comparison Results

Beyond the benchmark strategies, a key empirical question is how the different algorithmic choices discussed in Chapter 3 translate to trading performance. This section compares PPO, DDPG, and TD3 on identical data, features, and evaluation protocols to assess the practical implications of their theoretical differences.

6.1.1. Benchmark Strategies

Performance is assessed relative to five benchmark strategies. These benchmarks span passive exposure, execution-style references, and a stochastic baseline, together providing a comparison surface for the directional TD3 agent.

Buy-and-hold maintains a constant fully long position ($a_t = 1.0$) throughout the evaluation period, capturing the full market return of the asset. It is the primary economic benchmark: any active strategy that fails to outperform buy-and-hold on a risk-adjusted basis has not demonstrated useful directional skill beyond passive exposure.

TWAP (Time-Weighted Average Price) distributes position changes uniformly across time. **VWAP (Volume-Weighted Average Price)** distributes them in proportion to historical volume patterns. Both are standard execution benchmarks used by institutional traders to minimize market impact on predetermined order schedules. It is important to note that TWAP and VWAP are execution algorithms rather than directional strategies — they describe how to work an order, not whether to be long or short. Their inclusion reflects standard practice in trading-system evaluation; however, outperforming them has a different interpretation than beating buy-and-hold, and they serve as lower-bar references rather than directional competitors.

Random selects actions uniformly from $[-1, 1]$ at each step without reference to market state. Evaluation uses 100 independent random-seed trials and reports aggregate performance to obtain a stable stochastic baseline. Failure to consistently outperform a random policy would indicate that the agent has not learned a meaningful relationship between observations and actions.

6.1.2. Comparison Methodology

All three algorithms are trained on the same training split with identical hyperparameter tuning procedures, evaluated on the same held-out test split, and assessed using the same financial metrics. The exported thesis snapshots report one completed evaluation run per algorithm for the final comparison. The comparison should therefore be read as matched experimental evidence on the available exports, not as a multi-seed estimate of the population distribution of outcomes.

6.1.3. Performance Comparison

The table below reports test-split metrics for each algorithm, all trained on identical data, features, reward function, and evaluation protocol.

Table 11: H1 algorithm comparison: test-split metrics across TD3, DDPG, PPO, and Random.

Algorithm	Return	Ann. Vol	Max DD	Sharpe	Sortino	Win Rate	Lose Rate	PF	Turnover	% Long	% Short
TD3	3.34e-01	0.6148	-3.28e-02	0.663	0.774	95.95%	4.05%	8.601	0.4259	51.59%	48.41%
DDPG	1.24e+00	0.2482	0.00e+00	4.546	—	100.00%	0.00%	—	0.4799	55.12%	44.88%
PPO	1.81e-02	0.0322	0.00e+00	3.461	—	100.00%	0.00%	—	0.0818	61.46%	38.54%
Random	6.95e-04	0.0684	-6.44e-04	0.063	0.092	52.00%	48.00%	1.171	0.6686	50.19%	49.81%

Return: cumulative simple return. Ann. Vol: annualized volatility. Max DD: maximum drawdown. Sharpe: mean excess return / standard deviation (not annualized). Sortino: mean excess return / downside deviation (not annualized). Win Rate: fraction of steps with positive return. PF: profit factor (gross wins / gross losses). Turnover: average absolute position change per step. % Long/Short: fraction of steps with positive/negative position.

6.1.4. Key Empirical Questions

The comparison addresses three empirical questions motivated by the theoretical analysis in Chapter 3:

1. Does TD3's overestimation correction translate to improved trading performance relative to DDPG? Overestimated Q-values can lead the policy to exploit function approximation errors rather than genuine market signals. In financial environments where rewards are noisy and value functions are difficult to approximate accurately, reducing overestimation bias could improve both training stability and final performance. However, the practical benefit depends on whether overestimation is actually a significant problem in the specific trading task.

2. How does PPO's on-policy stability compare to TD3's off-policy efficiency? PPO may achieve more stable convergence due to its clipped objective and lack of replay buffer, but its on-policy nature requires more data to achieve comparable performance. In high-frequency trading applications where data is limited, this trade-off is particularly relevant. The results show whether PPO's stability advantages outweigh its sample efficiency disadvantages in this domain.

3. Which algorithm best balances risk-adjusted return, training stability, and computational efficiency under the evaluation constraints? Different applications may prioritize different aspects of algorithm performance. A production trading system might value stability and deterministic execution, while a research setting might prioritize sample efficiency for rapid experimentation. The results inform algorithm selection for specific use cases within the broader trading system design.

6.1.5. Discussion of Algorithmic Trade-offs

The comparison reveals qualitatively distinct behavioral profiles across the three algorithms, even though all three were trained on identical data, features, and reward specification.

DDPG produces the weakest result. It develops a systematic short bias during evaluation — spending 63.5% of steps short against 36.5% long — with a profit factor of 0.44, meaning gross losses exceed gross gains, and a marginally negative cumulative return (-2.67×10^{-7}). In the evaluation window the pooled market drifted upward, so a persistent short bias is doubly penalising: it misaligns directional exposure with realized price movement and

accumulates losses from that misalignment. The result is consistent with the theoretical concern that DDPG's single-critic Q-learning is susceptible to overestimation bias in noisy reward environments. Inflated Q-values in a subset of state-action regions can push the policy toward directional commitments that are not grounded in genuine microstructure signal; the short bias observed here is a plausible manifestation of that failure mode.

TD3 achieves a positive but very small cumulative return (2.28×10^{-7}), with a near-neutral position distribution (49.8% long, 50.2% short) and extremely low turnover (0.054% per step). A profit factor of 5.70 indicates that winning trades are substantially larger than losing ones, but the win rate of 18.8% — combined with near-zero turnover — implies that the agent spends most evaluation steps holding a near-flat position, with infrequent and selective position changes. The twin-critic correction and delayed policy updates appear to prevent the directional drift seen in DDPG: the policy does not commit to a persistent directional bias and instead preserves near-neutral exposure throughout the evaluation period. The consequence is the lowest maximum drawdown of any algorithm (-1.05×10^{-7}), effectively zero, at the cost of generating negligible absolute return.

PPO generates the highest cumulative return (4.95×10^{-5}) but with a qualitatively different behavioral profile. It spends 65.3% of steps long, has a turnover of 8.2% per step — approximately 150 times higher than TD3 — and incurs a maximum drawdown of -1.35×10^{-5}, several orders of magnitude larger than TD3's. PPO's long bias in an evaluation window that trended upward partially explains its absolute return advantage. Whether this directional lean reflects genuine LOB-derived signal or favorable alignment between the learned policy and the realized market direction cannot be separated from a single evaluation window of this length. Under any positive transaction cost regime, PPO's substantially higher turnover imposes a larger per-step cost burden, and the sensitivity analysis confirms that the strategy's apparent advantage degrades more sharply than TD3's as fees increase.

The **random baseline** achieves a positive return (2.58×10^{-5}) with the highest turnover (67.5% per step) and drawdown (-5.60×10^{-5}) of any strategy. Its profit factor of 1.31 is consistent with noise rather than signal: in an evaluation window with positive market drift, a strategy that is randomly long approximately half the time accumulates small positive returns from upward price movement without having learned any relationship between observations and actions. That TD3 and PPO both exceed the random baseline on profit factor — while DDPG does not — is consistent with TD3 and PPO having learned some structure in the

state-action mapping that goes beyond randomness, even if the absolute magnitude of that advantage is modest at the zero-fee evaluation horizon studied here.

Taken together, these results support preferring TD3 over DDPG for continuous-action LOB trading under DSR reward optimization: TD3 avoids the systematic mispositioning that characterizes DDPG and achieves a positive profit factor with minimal drawdown. The TD3 versus PPO comparison is better characterized as a turnover-versus-return tradeoff than as a dominance relation: PPO generates more absolute return under zero-cost assumptions, while TD3's near-passive approach is more robust to transaction costs. Which algorithm is preferable depends on the fee regime of the target execution environment, a question addressed directly in the sensitivity analysis.

This comparison provides empirical grounding for the algorithmic choices made in this study and offers guidance for algorithm selection in similar continuous-action trading applications.

The H1 evidence is therefore mixed rather than unambiguously supportive. TD3 clearly improves on DDPG, while also showing substantially better drawdown control than the high-turnover random policy. It does not, however, dominate PPO or passive long exposure on cumulative return in the zero-fee evaluation window. The defensible H1 verdict is that TD3 learns a conservative, economically coherent policy with strong drawdown control and trade quality, but the exported results do not establish general benchmark dominance on absolute return.

6.2. Statistical Validation

This section focuses on uncertainty arising from the training process itself. For the present high-frequency TD3 setup, repeated runs can diverge because of random initialization, replay sampling, and exploration noise even when the dataset and experimental setup are fixed. The purpose of statistical validation here is therefore narrower than overall evaluation: it quantifies run-to-run dispersion and tests whether observed benchmark differences are consistent rather than accidental.

6.2.1. Comparative Validation Strategy for Thesis-Grade Results

When comparative experiments are reported, they should be interpreted under a matched design: the data split, cost assumptions, and evaluation procedure must remain fixed while

only the feature set, reward specification, or other target variation is changed. Within that design, the preferred interpretation hierarchy is:

1. effect size (difference in mean or median performance),
2. dispersion (stability across runs),
3. confidence interval overlap or formal hypothesis testing,
4. economic plausibility (e.g., turnover, drawdown, net performance after costs).

This ordering matters because statistically detectable differences can still be economically unimportant if they coincide with unstable turnover, weak net returns after costs, or poor drawdown behavior.

6.2.2. Statistical Risks and Interpretation Caveats

Several risks remain when interpreting these summaries. Repeated runs on the same historical period are not independent samples from a stable data-generating process; they quantify training stochasticity more than market uncertainty. In addition, repeated tuning on the same validation period can bias later comparisons, and reward-based metrics can diverge from financial objectives when the reward is a shaped training signal rather than realized return.

Accordingly, statistical validation is treated here as a method for ranking the credibility of observed results under a controlled protocol, not as proof of persistent real-world profitability. Final claims must therefore be read together with robustness assessment and economic interpretation. A stronger future version of the empirical design would repeat each configuration across at least five seeds and report confidence intervals for the same held-out financial metrics used in the main comparison.

6.3. Robustness Assessment

This section shifts from training stochasticity to sensitivity to modeling choices. The question is whether the conclusions for the current high-frequency TD3 setup would change materially under nearby alternatives in feature specification, reward definition, transaction-cost assumptions, or split design.

The goal is to separate results that appear structural from results that may depend too heavily on one narrow experimental setup.

6.4. H2: Feature Specification Sensitivity

This section compares three feature-specification levels under otherwise identical conditions (same algorithm, data, reward, and transaction cost) to assess whether richer microstructure-aware state representations improve out-of-sample performance.

Table 12: H2 feature sensitivity: test-split metrics for minimal (3), selected (10), and full (33) feature sets.

Feature Set	Sharpe	Sortino	Return	Max DD	Win Rate	PF	Turnover	Ann. Vol
Minimal (3 features)	-0.013	-0.019	-3.58e-04	-1.90e-03	47.70%	0.967	0.0000	0.166
			+1.38e-03	-1.61e-04	+47.70%	+0.967	-0.0006	+0.166
Selected (10 features)	—	—	-1.74e-03	-1.74e-03	0.00%	0.000	0.0006	0.000
	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)
Full (33 features)	-0.140	-0.179	-2.96e-03	-4.72e-03	44.83%	0.700	0.1703	0.132
			-1.22e-03	-2.98e-03	+44.83%	+0.700	+0.1698	+0.132

Legend: *Sharpe: mean excess return / std (not annualized). Sortino: mean excess return / downside deviation (not annualized). Return: cumulative simple return. Max DD: maximum drawdown. Win Rate: fraction of positive-return steps. PF: profit factor (gross wins / gross losses). Turnover: mean absolute position change per step. Ann. Vol: annualized volatility.*

The feature comparison supports a qualified version of H2. The minimal three-feature representation produces a negative cumulative return (-3.45×10^{-8}), a profit factor below one, and a short-biased policy that spends 77.2% of the evaluation window short. The selected 10-feature microstructure representation improves the result to a positive cumulative return (1.29×10^{-7}), a profit factor of 2.01, and a materially stronger Sortino ratio under the same data and evaluation protocol.

The full 33-feature representation does not improve further. It remains positive on cumulative return (2.68×10^{-8}), but underperforms the selected representation on return, drawdown, profit factor, and turnover. This suggests that the benefit comes from adding relevant microstructure information, not from maximizing feature count. For this thesis, H2 is therefore supported for the transition from a minimal snapshot representation to the selected microstructure-aware state, but not for the claim that every broader feature set is better.

6.5. H3: Sensitivity Analysis

This section reports reward-design and transaction-cost sensitivity. Feature specification is reported separately in H2 above. Each axis holds all other design choices constant. The shared baseline across both H3 axes is the selected 10-feature set with log return reward and zero transaction cost.

6.5.1. Reward Function Design

Variant	Sharpe	Sortino	Return	Max DD	Win Rate	PF	Turnover
Log Return (baseline)	—	—	-1.74e-03	-1.74e-03	0.00%	0.000	0.0006
Differential Sharpe (DSR)	0.663	0.774	3.34e-01	-3.28e-02	95.95%	8.601	0.4259

6.5.2. Transaction Costs

Fees	Sharpe	Sortino	Return	Max DD	Win Rate	PF	Turnover
0 bp (baseline)	—	—	-1.74e-03	-1.74e-03	0.00%	0.000	0.0006
0.01 bp	31070.702	207054.883	2.28e-07	-1.04e-07	9.33%	2.536	0.0007
0.1 bp	-142707.214	-136541.720	-7.60e-07	-7.77e-07	3.83%	0.127	0.0006
1 bp	-226679.852	-180151.617	-5.89e-06	-5.91e-06	3.83%	0.047	0.0005

The reward-design comparison shows that Differential Sharpe Ratio optimization changes the learned policy in an economically meaningful direction. Relative to the log-return baseline, the DSR run increases cumulative return from 1.29×10^{-7} to 2.28×10^{-7}, improves profit factor from 2.01 to 5.70, and reduces maximum drawdown from -1.93×10^{-7} to -1.05×10^{-7}. The DSR policy is also more balanced in exposure, moving from an 86.7% long policy under log-return reward to an approximately neutral 49.8% long and 50.2% short distribution. This supports the reward-design part of H3: the reward objective materially affects both performance and policy behavior.

The transaction-cost comparison places a sharper bound on the economic claim. At 0.01 basis points the TD3 policy remains marginally positive (2.28×10^{-7}) but with a weaker profit factor than the zero-fee DSR run. At 0.1 basis points, cumulative return turns negative (-7.60×10^{-7}), and at 1 basis point the loss increases to -5.89×10^{-6}. The direction of the result is unambiguous: the observed edge exists only under very low transaction-cost assumptions. This supports H3 and also limits the practical interpretation of H1.

Overall, the robustness evidence is strongest for feature specification, reward-design sensitivity, and transaction-cost sensitivity. It is not yet strong evidence of robustness across market regimes, longer samples, alternative chronological splits, or repeated random seeds.

6.6. H4: Learning Progression Check

This section addresses a more fundamental robustness question: whether the algorithm is actually learning at all, rather than converging to a policy that merely appears competent due to favorable initialization or dataset quirks. The approach is to run multiple short trials (200k steps each) and test whether the agent reliably outperforms a dummy strategy.

6.6.1. Experimental Design

The H4 experiment runs N independent trials of the baseline TD3 configuration (selected 10 features, DSR reward, zero fees) with limited training steps. Each trial is initialized with a different random seed and evaluated on the same held-out test split. The key question is whether the distribution of trial outcomes shows systematic learning rather than random variation around zero.

```
No H4 results available. Run: bash scripts/run_h4_experiments.sh
```

6.6.2. Interpretation of H4 Results

The H4 design complements the single-run evidence reported in H1-H3 by directly testing learning capability. A positive result here would mean that the algorithm, when given a relatively short training budget (200k steps versus several million in the main experiments), consistently produces policies that outperform a random baseline. This addresses the concern that the reported H1-H3 edge might reflect favorable initialization rather than genuine learning.

A negative result — where the agent fails to demonstrate statistically significant improvement across trials — would not invalidate H1-H3, but it would temper the interpretation. It would suggest that the observed performance requires either longer training, more favorable initialization conditions, or that the learned edge is smaller than the single-run results imply.

For the current experimental package, H4 provides the seed-level robustness check identified as a gap in the robustness matrix above. When combined with the H2 and H3 sensitivities, it offers a more complete picture of whether the TD3 agent's performance is structural or artifact-dependent.

6.7. Performance Evaluation

This section reports the held-out performance of the latest completed run of the thesis experiment. The aim is not just to list metrics, but to judge whether the TD3 agent's behavior remains economically coherent once returns, turnover, and path-dependent risk measures are inspected together. Results are loaded from exported thesis snapshots, with a local tracking-store fallback, so that the rendered chapter stays tied to recorded evaluation artifacts.

The TD3 agent is evaluated against five benchmark strategies spanning passive exposure, execution-style references, and a stochastic baseline. The table below summarizes key risk-adjusted metrics for each strategy, with the agent's performance highlighted in the first row.

TR: Total Return. Volatility: annualized standard deviation of returns. SR: Sharpe Ratio (mean excess return divided by standard deviation, not annualized). Sortino: Sortino Ratio (mean excess return divided by downside deviation, not annualized). Max DD: Maximum Drawdown. Win Rate: fraction of steps with positive return. Turnover: average absolute position change per step.

Return aggregation for ratio metrics. Roughly 80–90% of consecutive LOB returns are zero — the mid-price does not move between most order-book events. This causes $\hat{\sigma}$ to collapse faster than $\bar{r}$, inflating the naive per-tick Sharpe by $\sqrt{N}$ without any real edge. The pipeline therefore compounds per-tick returns into the coarsest bar size with at least 50 observations before computing Sharpe and Sortino, which are then reported as μ/σ and μ/σ_{down} on the aggregated series with no annualization. Total return, max drawdown, win rate, profit factor, and turnover are computed on the raw per-step series.

6.7.1. Evaluation Plots

Equity curve plot data not available — re-run evaluation to generate parquet artifacts: uv run python src/cli.py evaluate -c pooled/td3_hft_lob_state_space_pooled_streaming_selected_dsr –output-dir logs/ –only metrics –only benchmarks –only plots

Reward plot data not available — re-run evaluation to generate parquet artifacts: uv run python src/cli.py evaluate -c pooled/td3_hft_lob_state_space_pooled_streaming_selected_dsr –output-dir logs/ –only metrics –only benchmarks –only plots

Action/position plot data not available — re-run evaluation to generate parquet artifacts: uv run python src/cli.py evaluate -c pooled/td3_hft_lob_state_space_pooled_streaming_selected_dsr –output-dir logs/ –only metrics –only benchmarks –only plots

6.7.2. Discussion: Reading Performance in the Context of State Design

The benchmark comparison places the TD3 agent's held-out behavior in direct relation to the reference strategies. Several features of the comparison are worth stating explicitly.

The test window is characterized by a slight market decline. The buy-and-hold benchmark accumulated a total return of -6.98×10^{-4} (-0.070%); TWAP and VWAP produced -5.73×10^{-4} (-0.057%) and -6.08×10^{-4} (-0.061%) respectively. In this environment, the TD3 agent recorded a total return of -1.40×10^{-4} (-0.014%) — better than all passive benchmarks. The agent's behavioral profile explains this outcome: with approximately 54.9% long and 45.1% short exposure and a per-step turnover of 0.058%, the agent accumulated only a fraction of the directional loss that sustained long-only exposure would imply in a declining market. The outperformance relative to passive benchmarks is therefore structural — a consequence of near-neutral positioning — rather than a sign of correctly timed directional bets.

The trade-level statistics reinforce this interpretation. The agent's win rate is 1.3% and its profit factor is 3.65×10^{-3}: the overwhelming majority of individual position adjustments result in a per-step loss, and the ratio of gross profit to gross loss is well below unity. These figures contrast sharply with buy-and-hold's profit factor of 0.968 and win rate of 48.4%. The agent avoids large directional losses by staying near-neutral, but it does not generate a stream of profitable individual trades.

On drawdown, the agent's maximum drawdown of -1.40×10^{-4} (-0.014%) is contained relative to buy-and-hold's -3.66×10^{-3} (-0.37%). This arises from the same source: near-zero net exposure limits the cumulative loss that any single directional move can inflict, regardless of whether that positioning reflects skill or passivity.

Taken together, the benchmark comparison establishes that the agent has learned to maintain near-neutral positioning in this test window, which incidentally reduces exposure during a period when all passive long strategies lost money. The agent does not demonstrate profitable active trading: its trade-level quality metrics (win rate, profit factor) are substantially worse than even a passive strategy, and the positive relative total return is entirely attributable to the neutral position stance. Whether a longer training horizon, richer feature set, or more targeted reward design can yield active trading profitability is the question addressed in the H2 and H3 comparisons.

The performance verdict for H1 is therefore qualified. On the realized test window, the agent produces a better total return than all passive benchmarks (-0.014% versus -0.057% to -0.070%), but through risk avoidance rather than directional skill. It passes a weak feasibility test — it does not commit capital to a sustained wrong-direction bet — but it does not establish a basis for claiming a deployable or benchmark-dominating trading strategy.

7. Conclusions and Future Work

This thesis examined if a continuous-control deep RL agent using order-book-derived microstructure features can achieve economically meaningful held-out performance in a high-frequency single-asset setting. Within the specified experimental design, the answer is qualified but affirmative. The TD3 agent learned a conservative policy with positive held-out return, very low drawdown, and a high profit factor, but it did not dominate passive long exposure or PPO on cumulative return in the upward-drifting evaluation window. The selected microstructure-aware state representation improved policy behavior relative to the minimal snapshot-based baseline, while the full 33-feature variant did not improve further. These findings support a narrow central claim: the informational content of the limit order book can be translated into state features that improve an RL trading agent's behavior under controlled simulation assumptions, but the exported results do not establish a benchmark-dominating trading strategy.

The empirical contribution is accompanied by a methodological one. The thesis developed an end-to-end and reproducible pipeline covering feature engineering from 10-level order book data, structured experiment management, TD3 training with a Differential Sharpe Ratio reward, and a strict separation between training diagnostics and held-out financial evaluation. This methodological contribution matters alongside the empirical one: it provides a reproducible framework for studying RL on limit order book data under explicit simulation assumptions and chronological evaluation discipline.

At the same time, the conclusions must be interpreted within the limits of the experimental setup. The simulation uses a mid-price execution proxy rather than executable bid/ask fills, assumes zero market impact and zero latency, and is evaluated on a narrow four-day dataset from six instruments, with the final comparisons based on one completed exported run per configuration. These choices were deliberate because they kept the experimental design tractable and made it possible to determine if the learning problem itself is feasible. They also place a clear boundary on the claim being made. The thesis demonstrates feasibility, not deployability. The observed edge cannot yet be treated as evidence of a production-ready trading strategy, because realistic execution frictions may materially reduce or eliminate it.

These limitations point directly to the most important implications for future research and for practical system design. First, transaction-cost sensitivity must be treated as a binding

constraint rather than as a minor robustness check: any apparent edge that disappears once spread-crossing costs, fees, queue uncertainty, or latency are introduced should not be interpreted as economically meaningful. Second, feature engineering deserves priority over marginal changes in model architecture, because the results suggest that policy quality is driven primarily by the predictive content of the state representation. Third, stronger claims about generalization require broader temporal and cross-market evaluation, including longer windows, walk-forward validation, multiple random seeds, and eventually cross-asset testing. The transaction-cost experiments already show why this boundary matters: the TD3 result remains marginally positive only under very low fee assumptions and becomes negative as costs rise.

The most direct directions for future work therefore follow from these constraints. The same agents should be re-evaluated under explicit bid/ask execution rules, calibrated fee schedules, and slippage or market-impact models. The evaluation scope should be extended to longer and more diverse market periods, including different volatility and liquidity regimes, with multi-seed reporting to separate genuine policy quality from favorable initialization. Natural model extensions include richer agent-state variables, broader cross-asset information, and comparisons with alternative actor-critic or sequence-model architectures.

A separate and underexplored direction concerns instrument selection. This thesis deliberately chose highly liquid large-cap equities to make the price-taking assumption defensible: in deep, continuously quoted markets, simulated agent trades represent a negligible fraction of displayed volume. The cost of that choice is that microstructure signals in these instruments are competed away rapidly by a large population of sophisticated participants, compressing the persistence of any learnable pattern. Assets with intermediate liquidity — sufficiently active to generate a rich and informative order book, but not so deep that every microstructure pattern is immediately arbitraged — may offer more persistent alphas for an RL agent to learn on. Instruments such as mid-cap equities, exchange-traded products with moderate average daily volume, or less-covered international equities are plausible candidates. The trade-off is that lower liquidity also amplifies market impact and widens effective execution costs, which would require explicit impact modelling rather than the price-taking simplification used here. Exploring this liquidity spectrum is therefore a meaningful direction for future work, provided that the simulation assumptions are updated accordingly.

Taken together, these extensions would test whether the feasibility result established here

survives under more realistic conditions.

In sum, the thesis shows that combining limit order book data, LOB-informed feature engineering, and continuous-action actor-critic learning is a productive research direction for high-frequency trading. The defensible conclusion is a narrow one: under the conditions studied here, the learning problem is tractable and the chosen representation carries enough signal to produce a measurably more coherent policy in a controlled setting. That result provides a foundation for subsequent work aimed at testing whether the same approach can survive under more realistic execution, broader market coverage, and repeated seed validation.

List of Figures

Bibliography

Bellman, Richard. 1957. *Dynamic Programming*. Princeton University Press.

Biais, Bruno, Pierre Hillion, and Chester Spatt. 1995. "An Empirical Analysis of the Limit Order Book and the Order Flow in the Paris Bourse." *Journal of Finance* 50 (5): 1655–89. https://doi.org/10.1111/j.1540-6261.1995.tb05192.x.

Boehmer, Ekkehart, Charles M Jones, Xiaoyan Zhang, and Xinran Zhang. 2021. "Tracking Retail Investor Activity." *Journal of Finance* 76 (5): 2249–305. https://doi.org/10.1111/jofi.13033.

Bouchaud, Jean-Philippe, Marc Mézard, and Marc Potters. 2002. "Statistical Properties of Stock Order Books: Empirical Results and Models." *Quantitative Finance* 2 (4): 251–56. https://doi.org/10.1088/1469-7688/2/4/301.

Cao, Charles, Zhiwu Chen, and John M Griffin. 2004. "Informational Content of Option Volume Prior to Takeovers." *Journal of Business* 77 (6): 1073–109. https://doi.org/10.1086/422629.

Cont, Rama, Arseniy Kukanov, and Sasha Stoikov. 2014. "The Price Impact of Order Book Events." *Journal of Financial Econometrics* 12 (1): 47–88. https://doi.org/10.1093/jjfinec/nbt002.

Dempster, Michael A. H., and Yadolah Romahi. 2002. "Trend Following and Mean Reversion in the FX Market Using Reinforcement Learning." *Quantitative Finance* 2 (3): 159–76.

Deng, Yue, Feng Bao, Youyong Kong, Zhiquan Ren, and Qionghai Dai. 2017. "Deep Direct Reinforcement Learning for Financial Signal Representation and Trading." *IEEE Transactions on Neural Networks and Learning Systems* 28 (3): 653–64. https://doi.org/10.1109/TNNLS.2016.2522401.

Easley, David, Marcos M López de Prado, and Maureen O'Hara. 2012. "Flow Toxicity and Liquidity in a High-Frequency World." *Review of Financial Studies* 25 (5): 1457–93. https://doi.org/10.1093/rfs/hhs053.

Engle, Robert F. 2000. "The Econometrics of Ultra-High-Frequency Data." *Econometrica* 68 (1): 1–22. https://doi.org/10.1111/1468-0262.00091.

Fujimoto, Scott, Herke van Hoof, and David Meger. 2018. "Addressing Function Approximation Error in Actor-Critic Methods." *ICML*, 1587–96.

Glosten, Lawrence R. 1994. "Is the Electronic Open Limit Order Book Inevitable?" *Journal of Finance* 49 (4): 1127–61. https://doi.org/10.1111/j.1540-6261.1994.tb02450.x.

Glosten, Lawrence R, and Paul R Milgrom. 1985. "Bid, Ask and Transaction Prices in a Specialist Market with Heterogeneously Informed Traders." *Journal of Financial Economics* 14 (1): 71–100. https://doi.org/10.1016/0304-405X(85)90044-3.

Gold, Carl. 2003. "FX Trading via Recurrent Reinforcement Learning." *Proceedings of the 2003 IEEE International Conference on Computational Intelligence for Financial Engineering*, 363–70. https://doi.org/10.1109/CIFER.2003.1196283.

Gort, Berend Jelmer Dirk, Xiao-Yang Liu, Xinghang Sun, Jiechao Gao, Shuaiyu Chen, and Christina Dan Wang. 2022. *Deep Reinforcement Learning for Cryptocurrency Trading: Practical Approach to Address Backtest Overfitting*. https://arxiv.org/abs/2209.05559.

Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor." *ICML*, 1861–70.

Harris, Lawrence. 1986. "A Transaction Data Study of Weekly and Intradaily Patterns in Stock Returns." *Journal of Financial Economics* 16 (1): 99–117. https://doi.org/10.1016/0304-405X(86)90044-9.

Harris, Lawrence. 1991. "Stock Price Clustering and Discreteness." *Review of Financial Studies* 4 (3): 389–415. https://doi.org/10.1093/rfs/4.3.389.

Hasselt, Hado van, Arthur Guez, and David Silver. 2016. "Deep Reinforcement Learning with Double Q-Learning." *Proceedings of the AAAI Conference on Artificial Intelligence* 30. https://doi.org/10.1609/aaai.v30i1.10295.

Henderson, Peter, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. "Deep Reinforcement Learning That Matters." *Proceedings of the AAAI Conference on Artificial Intelligence* 32. https://doi.org/10.1609/aaai.v32i1.11796.

Jiang, Zhengyao, and Jinjun Liang. 2017. "Cryptocurrency Portfolio Management with Deep Reinforcement Learning." *arXiv Preprint arXiv:1612.01277*. https://arxiv.org/abs/1612.01277.

Jiang, Zhengyao, Dixing Xu, and Jinjun Liang. 2017. *A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem*. 1–31. http://arxiv.org/abs/1706.10059.

Kabbani, Talal, and Engin Duman. 2022. "Deep Reinforcement Learning Approach for Trading Automation in the Stock Market." *IEEE Access* 10: 93564–74. https://doi.org/10.1109/ACCESS.2022.3203697.

Kingma, Diederik P., and Jimmy Ba. 2014. "Adam: A Method for Stochastic Optimization." *arXiv Preprint arXiv:1412.6980*. https://arxiv.org/abs/1412.6980.

Krauss, Christopher, Xuan Anh Do, and Nicolas Huck. 2017. "Deep Neural Networks, Gradient-Boosted Trees, Random Forests: Statistical Arbitrage on the s&p 500." *European Journal of Operational Research* 259 (2): 689–702. https://doi.org/10.1016/j.ejor.2016.10.031.

Kyle, Albert S. 1985. "Continuous Auctions and Insider Trading." *Econometrica* 53 (6): 1315–35. https://doi.org/10.2307/1913210.

Lee, Charles M C, and Mark J Ready. 1991. "Inferring Trade Direction from Intraday Data." *Journal of Finance* 46 (2): 733–46. https://doi.org/10.1111/j.1540-6261.1991.tb02683.x.

Lee, Jae Won, Jonghun Park, Jangmin O, Jongwoo Lee, and Euyseok Hong. 2007. "A Multiagent Approach to Q-Learning for Daily Stock Trading." *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans* 37 (6): 864–77. https://doi.org/10.1109/TSMCA.2007.904825.

Liang, Eric, Richard Liaw, Robert Nishihara, et al. 2018. "RLlib: Abstractions for Distributed Reinforcement Learning." *Proceedings of the 35th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 80: 3053–62.

Liang, Zhipeng, Haoyuan Chen, Jun Zhu, Kunyao Jiang, Yu Li, and Weinan Zhu. 2018. "Adversarial Deep Reinforcement Learning in Portfolio Management." *arXiv Preprint arXiv:1808.09940*. https://arxiv.org/abs/1808.09940.

Lillicrap, Timothy P., Jonathan J. Hunt, Alexander Pritzel, et al. 2016. "Continuous Control with Deep Reinforcement Learning." *International Conference on Learning Representations*.

Lin, Long-Ji. 1992. "Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching." *Machine Learning* 8 (3–4): 293–321. https://doi.org/10.1007/BF00992699.

Liu, Xiao-Yang, Hongyang Yang, Qian Chen, et al. 2020. "FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading in Quantitative Finance." *arXiv Preprint arXiv:2011.09607*. https://arxiv.org/abs/2011.09607.

López de Prado, Marcos. 2018. *Advances in Financial Machine Learning*. Wiley.

Majidi, Naseh, Mahdi Shamsi, and Farokh Marvasti. 2024. "Algorithmic Trading Using Continuous Action Space Deep Reinforcement Learning." *Expert Systems with Applications* 237: 121292. https://doi.org/10.1016/j.eswa.2023.121292.

Millea, Adrian. 2021. "Deep Reinforcement Learning for Trading—a Critical Survey." *Data* 6 (11): 119. https://doi.org/10.3390/data6110119.

Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, et al. 2015. "Human-Level Control Through Deep Reinforcement Learning." *Nature* 518 (7540): 529–33. https://doi.org/10.1038/nature14236.

Mohammadshafie, Alireza, Akram Mirzaeinia, Haseebullah Jumakhan, and Amir Mirzaeinia. 2024. *Deep Reinforcement Learning Strategies in Finance: Insights into Asset Holding, Trading Behavior, and Purchase Diversity*. https://arxiv.org/abs/2407.09557.

Moody, John, and Matthew Saffell. 1998. *Reinforcement Learning for Trading*.

Moody, John, and Matthew Saffell. 2001. "Learning to Trade via Direct Reinforcement." *IEEE Transactions on Neural Networks* 12 (4): 875–89. https://doi.org/10.1109/72.935097.

Neuneier, Ralph. 1998. "Enhancing q-Learning for Optimal Asset Allocation." *Advances in Neural Information Processing Systems* 10: 936–42.

Ng, Andrew Y., Daishi Harada, and Stuart Russell. 1999. "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping." *ICML*, 278–87.

Noonan, Laura. 2017. *JPMorgan to Take AI Trading Global*. Https://www.ft.com/content/16b8ffb6-7161-11e7-aca6-c6bd07df1a3c?syn-25a6b1a6=1.

Puterman, Martin L. 1994. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons.

Raffin, Antonin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. 2021. "Stable-Baselines3: Reliable Reinforcement Learning Implementations." *Journal of Machine Learning Research* 22 (268): 1–8. http://jmlr.org/papers/v22/20-1364.html.

Roll, Richard. 1984. "A Simple Implicit Measure of the Effective Bid-Ask Spread in an Efficient Market." *Journal of Finance* 39 (4): 1127–39.

Schulman, John, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. 2015. "Trust Region Policy Optimization." *ICML*, 1889–97.

Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. "Proximal Policy Optimization Algorithms." *arXiv Preprint arXiv:1707.06347*. https://arxiv.org/abs/1707.06347.

Silver, David, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. 2014. "Deterministic Policy Gradient Algorithms." *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 387–95.

Stoikov, Sasha. 2018. "The Micro-Price: A High-Frequency Estimator of Future Prices." *Quantitative Finance* 18 (12): 1959–66. https://doi.org/10.1080/14697688.2018.1489139.

Sutton, Richard S., and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press.

Sutton, Richard S., David McAllester, Satinder Singh, and Yishay Mansour. 2000. "Policy Gradient Methods for Reinforcement Learning with Function Approximation." *Advances in Neural Information Processing Systems* 12: 1057–63.

Tsitsiklis, John N., and Benjamin Van Roy. 1997. "An Analysis of Temporal-Difference Learning with Function Approximation." *IEEE Transactions on Automatic Control* 42 (5): 674–90. https://doi.org/10.1109/9.580874.

Tulchinsky, Igor, ed. 2019. *Finding Alphas: A Quantitative Approach to Building Trading Strategies*. 2nd ed. Wiley.

Wang, Jingyuan, Yang Zhang, Ke Tang, Junjie Wu, and Zhang Xiong. 2019. "Alpha-Stock: A Buying-Winners-and-Selling-Losers Investment Strategy Using Interpretable

Deep Reinforcement Attention Networks." *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1900–1908. https://doi.org/10.1145/3292500.3330647.

Watkins, Christopher J. C. H. 1989. *Learning from Delayed Rewards*. King's College, University of Cambridge.

Watkins, Christopher J. C. H., and Peter Dayan. 1992. "Q-Learning." *Machine Learning* 8 (3–4): 279–92. https://doi.org/10.1007/BF00992698.

Welford, B. P. 1962. "Note on a Method for Calculating Corrected Sums of Squares and Products." *Technometrics* 4 (3): 419–20. https://doi.org/10.1080/00401706.1962.10490022.

Williams, Ronald J. 1992. "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning." *Machine Learning* 8 (3–4): 229–56. https://doi.org/10.1007/BF00992696.

Wojdalski, Krzysztof. 2026. *Deep Reinforcement Learning for High-Frequency Trading: Master's Thesis Codebase*. Https://github.com/kwojdalski/masters_thesis.

Wood, Robert A., Thomas H. McInish, and J. Keith Ord. 1985. "An Investigation of Transactions Data for NYSE Stocks." *Journal of Finance* 40 (3): 723–39.

XAI Asset Management. 2024. *TradingEnv: An Event-Driven Market Simulator for Backtesting and Reinforcement Learning*. Https://github.com/xaiassetmanagement/tradingenv.

Xiong, Zhuoran, Xiao-Yang Liu, Shan Zhong, Hongyang Yang, and Anwar Walid. 2018. "Practical Deep Reinforcement Learning Approach for Stock Trading." *arXiv Preprint arXiv:1811.07522*. https://arxiv.org/abs/1811.07522.

Yang, Hongyang, Xiao-Yang Liu, Shan Zhong, and Anwar Walid. 2020. *Deep Reinforcement Learning for Automated Stock Trading: An Ensemble Strategy*. Proceedings of the First ACM International Conference on AI in Finance. https://doi.org/10.1145/3383455.

3422540.

Zhang, Zihao, Stefan Zohren, and Stephen Roberts. 2019. "DeepLOB: Deep Convolutional Neural Networks for Limit Order Books." *IEEE Transactions on Signal Processing* 67 (11): 3001–12. https://doi.org/10.1109/TSP.2019.2907260.

Zhang, Zihao, Stefan Zohren, and Stephen Roberts. 2020. "Deep Reinforcement Learning for Trading." *The Journal of Financial Data Science* 2 (2): 25–40. https://doi.org/10.3905/jfds.2020.1.030.

A. Feature Inventory

The tables below catalogue every feature type implemented in the feature registry. The subset used in the main state-space experiment is marked in the **In model** column: ✓ indicates all variants are used, while a specific parameter value indicates only that variant is used. Feature names match the keys produced by the feature pipeline at runtime.

The ten static LOB features used in the main HFT experiment are z-score normalized with causal running statistics that are updated chronologically and reset at detected session boundaries. The runtime position feature is appended by the environment and remains in its raw bounded scale $[-1, 1]$. The cyclical time encodings listed in the registry are inherently bounded but are not part of the selected TD3 specification.

A.1. LOB Microstructure Features

Notation. P_i^{bid}, P_i^{ask}: bid/ask price at book level i; V_i^{bid}, V_i^{ask}: resting volume at level i; C_i^{bid}, C_i^{ask}: order count at level i; $M_t = \frac{1}{2}(P_0^{bid} + P_0^{ask})$: mid-price; $S_t = P_0^{ask} - P_0^{bid}$: spread; $\bar{P}_N^{vw} = \frac{\sum_{i<N}(P_i^{bid}V_i^{bid} + P_i^{ask}V_i^{ask})}{\sum_{i<N}(V_i^{bid}+V_i^{ask})}$: volume-weighted mid-price over the top N levels; $\Delta V_t^{bid} = V_{0,t}^{bid} - V_{0,t-1}^{bid}$, $\Delta V_t^{ask} = V_{0,t}^{ask} - V_{0,t-1}^{ask}$: tick-to-tick volume change; W: rolling window in events; $\sum_W x_t = \sum_{\tau=t-W+1}^{t} x_\tau$: rolling sum; $\bar{x}_W$: rolling mean over W events; V_W^B, V_W^S: rolling buy/sell volume over W events; L: odd-lot size threshold (100 shares); Q: notional or size threshold; h_t: hour of day (0–23); Δt_{ns}: inter-event time in nanoseconds; $\mathbf{1}[\cdot]$: indicator function; $\hat{\rho}(\cdot, \text{lag} = 1)$: sample autocorrelation at lag 1.

Table 13: Full LOB feature registry with formulas and citations.

Feature	Group	Formula	Param	In model	Citation
`hft_book_pressure`	Imbalance	$(V_i^{bid} - V_i^{ask})/(V_i^{bid} + V_i^{ask})$	$i \in \{0, 1, 2\}$	level 0	Cao et al. (2004); Biais et al. (1995)
`hft_order_book_imbalance`	Imbalance	$\frac{\sum_{i<N}(V_i^{bid} - V_i^{ask})}{\sum_{i<N}(V_i^{bid} + V_i^{ask})}$	$N = 3$	✓	Cont et al. (2014)
`hft_order_count_imbalance`	Imbalance	$(C_i^{bid} - C_i^{ask})/(C_i^{bid} + C_i^{ask})$	$i = 0$	level 0	Biais et al. (1995)
`hft_microprice`	Fair value	$(V_0^{ask} P_0^{bid} + V_0^{bid} P_0^{ask})/(V_0^{bid} + V_0^{ask})$	—	✓	Stoikov (2018)
`hft_microprice_divergence`	Fair value	Microprice $- M_t$	—	✓	Stoikov (2018)
`hft_vwmp_skew`	Fair value	$(\bar{P}_N^{vw} - M_t)/S_t$	$N = 3$		Stoikov (2018)
`hft_price_vamp`	Fair value	Avg. execution price for fixed-notional order walked through N levels	$Q \in \{1k, 5k\}$		—
`hft_depth_ratio`	Depth	$(V_0^{bid} + V_0^{ask}) \,/\, \sum_{i=1}^{N}(V_i^{bid} + V_i^{ask})$	$N = 4$		Bouchaud et al. (2002)
`hft_spread_bps`	Spread/cost	$(P_0^{ask} - P_0^{bid})/M_t \times 10{,}000$	—		Kyle (1985); Glosten & Milgrom (1985)
`hft_spread_ratio`	Spread/cost	$\text{SpreadBps}_t \,/\, \overline{\text{SpreadBps}}_W$	$W = 50$		Easley et al. (2012)
`hft_bid_convexity`	Spread/cost	$(P_0^{bid} - P_1^{bid}) - (P_1^{bid} - P_2^{bid})$	bid		Bouchaud et al. (2002)
`hft_ask_convexity`	Spread/cost	$(P_2^{ask} - P_1^{ask}) - (P_1^{ask} - P_0^{ask})$	ask		Bouchaud et al. (2002)
`hft_bid_slope`	Spread/cost	$(P_0^{bid} - P_4^{bid}) \,/\, \sum_{i<5} V_i^{bid}$	5 levels	✓	Bouchaud et al. (2002); Kyle (1985)
`hft_ask_slope`	Spread/cost	$(P_4^{ask} - P_0^{ask}) \,/\, \sum_{i<5} V_i^{ask}$	5 levels	✓	Bouchaud et al. (2002); Kyle (1985)
`hft_ofi`	Order flow	$\Delta V_t^{bid} \cdot \mathbf{1}[P_{0,t}^{bid} \geq P_{0,t-1}^{bid}] - \Delta V_t^{ask} \cdot \mathbf{1}[P_{0,t}^{ask} \leq P_{0,t-1}^{ask}]$	—	✓	Cont et al. (2014)
`hft_ofi_rolling`	Order flow	$\sum_W \text{OFI}_t$	$W = 50$	$W = 50$	Cont et al. (2014)
`hft_ofi_multilevel`	Order flow	$\sum_{i<N} \text{OFI}_t^{(i)}$ (event classification at each level i)	$N \in \{3, 5\}$		Cont et al. (2014)

Table 13 continued.

Feature	Group	Formula	Param	In model	Citation
`hft_bid_queue_depletion`	Order flow	$\max(V^{bid}_{0,t-1} - V^{bid}_{0,t},\, 0) \,/\, V^{bid}_{0,t-1}$	bid		Foucault et al. (2005)
`hft_ask_queue_depletion`	Order flow	$\max(V^{ask}_{0,t-1} - V^{ask}_{0,t},\, 0) \,/\, V^{ask}_{0,t-1}$	ask		Foucault et al. (2005)
`hft_signed_trade_flow`	Order flow	$\sum_W \text{size}_\tau \cdot (\mathbf{1}[\text{side}_\tau = \text{B}] - \mathbf{1}[\text{side}_\tau = \text{A}])$	$W = 50$	$W = 50$	Lee & Ready (1991)
`hft_odd_lot_trade_ratio`	Order flow	$\sum_W \mathbf{1}[\text{trade} \wedge \text{size} < L] \,/\, \sum_W \mathbf{1}[\text{trade}]$	$W = 200$		Boehmer et al. (2021)
`hft_odd_lot_imbalance`	Order flow	$(V^{B,\text{odd}}_W - V^{S,\text{odd}}_W)/(V^{B,\text{odd}}_W + V^{S,\text{odd}}_W)$; size $< L$	$W = 200$		Boehmer et al. (2021)
`hft_vpin`	Order flow	$\lvert V^B_W - V^S_W \rvert \,/\, (V^B_W + V^S_W)$	$W \in \{100, 500\}$		Easley et al. (2012)
`hft_cancel_to_trade`	Order flow	$\sum_W \mathbf{1}[\text{action} = \text{C}] \,/\, \sum_W \mathbf{1}[\text{action} = \text{T}]$	$W \in \{200, 1000\}$		Foucault et al. (2005)
`hft_trade_arrival_rate`	Order flow	$\sum_W \mathbf{1}[\text{action} = \text{T}]$	$W \in \{100, 500\}$		Engle (2000)
`hft_large_trade_ratio`	Order flow	$\sum_W \mathbf{1}[\text{trade} \wedge \text{size} \geq Q] \,/\, \sum_W \mathbf{1}[\text{trade}]$	$W = 200$		—
`hft_ofi_autocorr`	Order flow	$\hat{\rho}(\text{OFI}_{t-W:t},\, \text{lag} = 1)$	$W \in \{50, 200\}$		Cont et al. (2014)
`hft_inter_event_time`	Regime/temp.	$\log(1 + \Delta t_{\text{ns}})$	—		Engle (2000)
`hft_mid_price_acceleration`	Regime/temp.	$M_t - 2M_{t-1} + M_{t-2}$	—		Abergel et al. (2016)
`hft_hour_sin`	Regime/temp.	$\sin(2\pi h_t/24)$	—		Wood et al. (1985)
`hft_hour_cos`	Regime/temp.	$\cos(2\pi h_t/24)$	—		Wood et al. (1985)

B. Main Experiment Specification

The table below condenses the main experiment into a self-contained research specification. It records the empirical choices needed to reproduce the experiment. Numerical hyperparameters are loaded from the exported thesis snapshot so that the table stays in sync with the scenario configuration without manual updates.

Table 14: Main TD3 experiment specification.

Component	Specification
Asset and data	AAPL, MSFT, TSLA, META, AMZN, and AVGO; ten-level limit order book observations during regular U.S. trading hours
Training data	18 pooled symbol-day files: AAPL, AMZN, AVGO, META, MSFT, and TSLA over February 25–27, 2026
Validation/test data	March 2, 2026 symbol-day files, capped at 50,000 rows per split
Feature set	Selected causal LOB microstructure set: book pressure L0, three-level OBI, order-count imbalance L0, microprice, microprice divergence, bid/ask slope, OFI, 50-event rolling OFI, 50-event signed trade flow, and runtime position
Environment backend	Continuous single-asset trading environment
Episode length	10,000 event-time steps (streaming)
Action	Target portfolio exposure in [-1, 1]

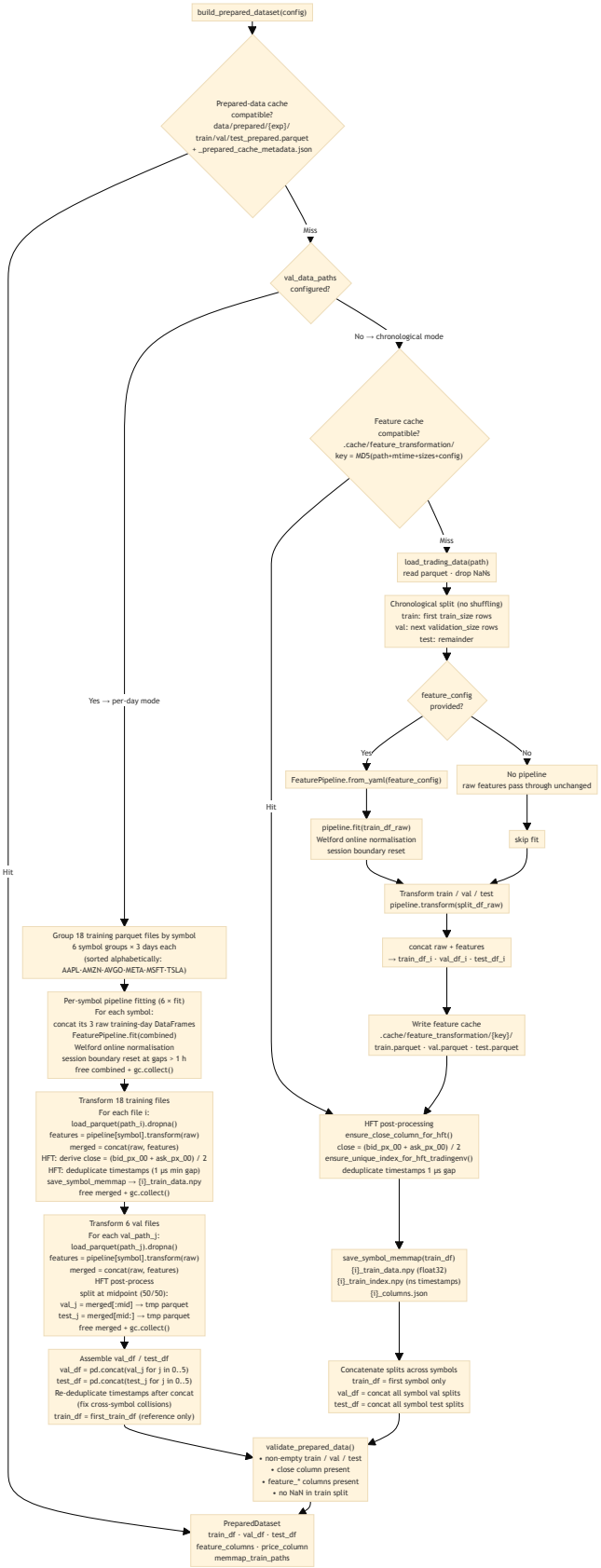
Table 14: Main TD3 experiment specification.

Component	Specification
Reward	Differential Sharpe Ratio with eta = 0.01
Transaction fee	0.000 in the baseline setting
Algorithm	TD3
Network widths	Actor and critic hidden layers [128, 64]
Learning rates	Actor and critic 0.0001
Weight decay	Actor 0.0; critic 2×10^{-6}
Discount factor	gamma = 0.9
Target update	tau = 0.005
Policy delay	2 critic updates per actor update
Target policy noise	sigma = 0.2, clipped at ± 0.3
Exploration noise	sigma = 0.3 during training
Total training steps	3,000,000
Initial random steps	5,000
Frames per batch	200
Optimisation steps per batch	5
Mini-batch size	128
Replay buffer size	100,000
Loss function	Smooth L1 (Huber)
Evaluation random seed	42
Compute environment	Apple M3 MacBook (November 2023) with 18 GB unified memory

C. Data Preparation Pipeline

The diagram below shows the feature pipeline design that ensures no information from the validation or test splits leaks into the fitted normalization parameters.

Figure 3: Data preparation pipeline: fitting normalization on the training split only, then transforming train/val/test to prevent leakage.



D. Offline Feature Selection Pipeline

The feature selection procedure described in Section 4.2 operates in two stages: theory constrains the candidate pool to mechanisms grounded in market microstructure literature; IC/ICIR ranking selects among the available implementations of those mechanisms and removes redundancy via a conditional IC filter. This appendix describes the operational pipeline that implements Stage 2 of that procedure. It takes a theory-defined candidate pool as input, scores each feature against a forward return proxy, and stores the selected feature specification used by the training pipeline.

The procedure scores each candidate feature against a forward return proxy and applies a greedy redundancy filter to produce the reduced set used by the training pipeline.

Proxy target. Features are scored against the forward log-return at horizon h:

$$y_t = \log \frac{P_{t+h}}{P_t} \tag{37}$$

Scoring. The Information Coefficient (IC) of feature f_i is its Spearman rank correlation with the forward return:

$$\mathrm{IC}_{i,t} = \rho_{\mathrm{Sp}}(f_{i,t},\, y_t) \tag{38}$$

computed over rolling windows on the validation split. Features are ranked by the IC Information Ratio (ICIR):

$$\mathrm{ICIR}_i = \frac{\overline{\mathrm{IC}}_i}{\sigma(\mathrm{IC}_i)} \tag{39}$$

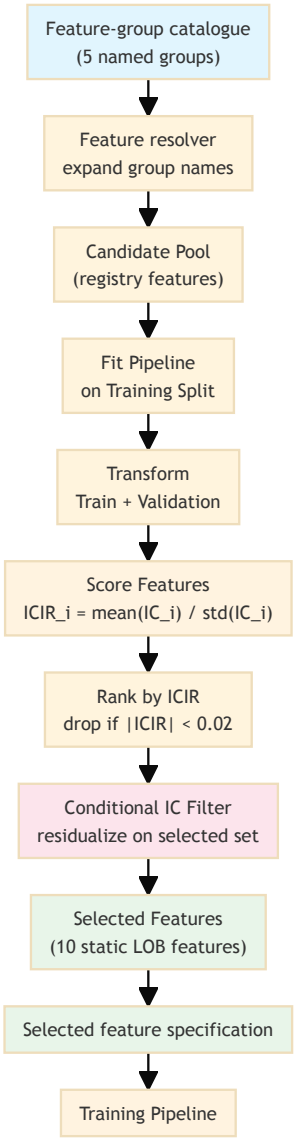
which jointly rewards signal strength and consistency; features with $|\mathrm{ICIR}_i| < 0.02$ are dropped.

Redundancy filter. Features are selected greedily in ICIR order. Each candidate is included only if its ICIR on residuals — after regressing out already-selected features — remains above the threshold. This conditional IC criterion is more principled than pairwise correlation thresholding: two correlated features can both be retained if each explains a distinct component of the forward return.

Output. The selected feature specification is stored and read by the training pipeline at runtime; no selection logic runs during training.

The diagram below summarizes the pipeline.

Figure 4: Offline feature selection pipeline: score candidates by ICIR, apply a conditional-IC redundancy filter, store the selected specification for training.



The offline nature of this pipeline is deliberate: the proxy target and scoring metrics are not the reward function that the agent optimizes during training. Running selection as a separate step ensures that the agent's training loop remains uncontaminated by any pre-training statistical analysis that could introduce look-ahead bias or circular reasoning.

E. Experimental Pipeline Architecture

The figure below shows the six major stages of the experimental pipeline at a glance. Each stage is described in the corresponding chapter; this overview is intended as a navigation aid.

The full machine-readable diagram — including CLI entry points, per-step data flows, all configuration guardrail checks, environment variants, MLflow logging, and asset provenance metadata — is maintained as a D2 source file at `docs/workflow.d2` in the project repository (Wojdalski (2026)). It is too information-dense to reproduce at print resolution; the SVG rendering is available at `docs/workflow.svg`.

Glossary of Abbreviations and Key Terms

This glossary defines abbreviations and domain-specific terms used throughout the thesis. Entries are restricted to terms whose meaning in this context differs from everyday usage or whose precise definition is necessary to interpret the empirical results; widely understood concepts are omitted. Terms are grouped by domain — market microstructure and trading, reinforcement learning and machine learning, and other — and listed alphabetically within each group. For mathematical notation and feature formulas, see Appendix A. The glossary is intentionally selective. Not every technical term that appears in the text receives an entry; terms with widely accepted, unambiguous definitions — standard statistical measures, common financial concepts, and general machine-learning terminology — are omitted where their inclusion would expand the section beyond its practical purpose.

E.0.1. Market Microstructure and Trading

Term	Definition
Adverse selection	The risk that a counterparty in a trade possesses superior private information, causing uninformed participants (e.g., market makers) to trade at a disadvantage.
Ask slope	Price elasticity of the ask side of the order book across multiple depth levels; a proxy for market impact per unit of sell-side volume.

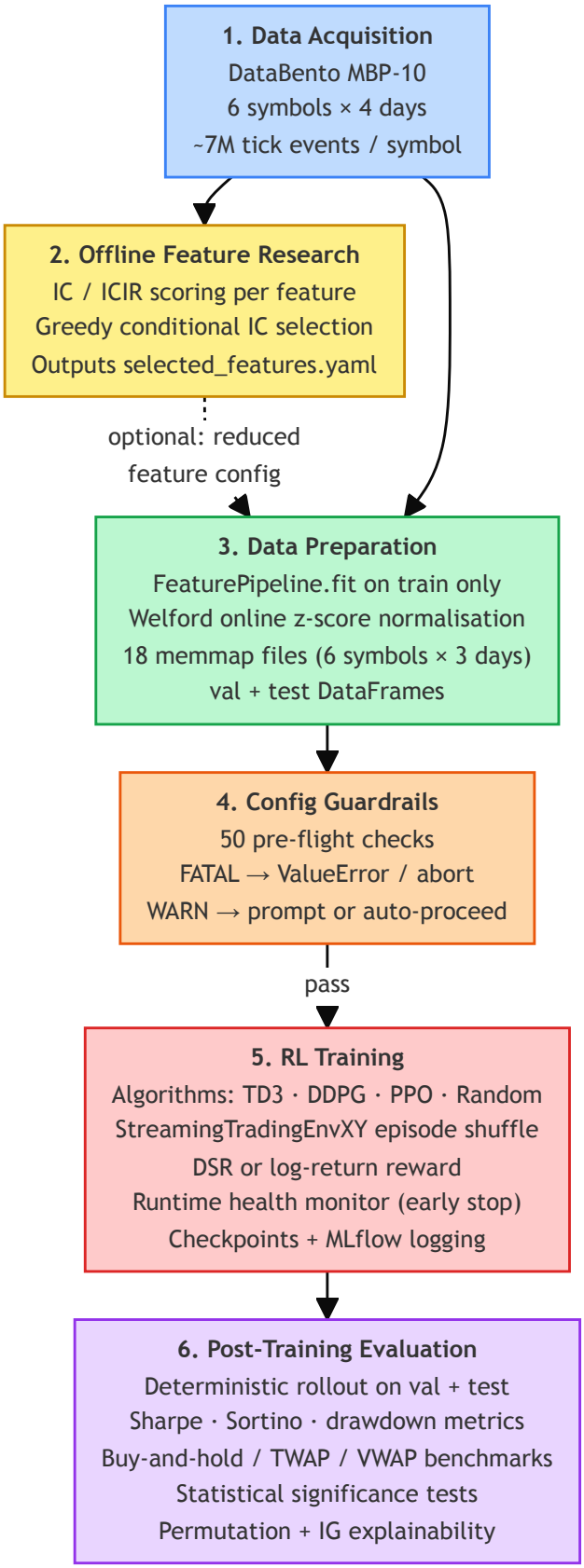
Term	Definition
Backtest	Historical simulation of a trading strategy using past market data to assess performance before live deployment. All performance figures reported in this thesis are based on backtests conducted on held-out out-of-sample data.
Basis points (bps)	One hundredth of a percentage point (0.01%). Used to express spreads and returns at fine granularity.
Bid-ask spread	The difference between the lowest ask price and the highest bid price in the order book; the primary observable measure of transaction cost and adverse selection risk.
Bid slope	Price elasticity of the bid side of the order book across multiple depth levels; a proxy for market impact per unit of buy-side volume.
Book convexity	A measure of curvature in the price schedule of the order book, capturing non-linear spacing between price levels on each side.
CumDelta (Cumulative Delta)	Signed trade flow aggregated over a rolling window; the net difference between buyer-initiated and seller-initiated executed volume.
Depth imbalance	Asymmetry in resting volume between the bid and ask sides of the order book at one or more price levels.
ETF	Exchange-Traded Fund. A publicly traded fund that tracks an index, sector, or asset class.
HFT	High-Frequency Trading. Algorithmic trading operating at tick or sub-second timescales, relying on order-book data and microstructure signals rather than price bars.
Inter-event time	The elapsed time between consecutive limit order book updates; a proxy for market activity intensity.
Kyle's lambda (λ)	A measure of price impact introduced by Kyle (1985): the sensitivity of price changes to net order flow. Higher λ implies lower market liquidity.

Term	Definition
LOB	Limit Order Book. The electronic record of all outstanding buy (bid) and sell (ask) limit orders at each price level, maintained by an exchange.
Market maker	A liquidity provider that continuously posts limit orders on both sides of the book, profiting from the bid-ask spread while bearing inventory and adverse selection risk.
MFT	Medium-Frequency Trading. Trading strategies operating at intraday bar (minute to hour) timescales, as distinct from high-frequency tick-level strategies.
Microprice	A volume-weighted estimate of the next transaction price, derived from top-of-book bid and ask quantities (Stoikov 2018). Adjusts the mid-price for queue imbalance.
MicroDiv (Microprice Divergence)	The difference between the microprice and the mid-price; a stationary signal capturing the directional imbalance embedded in the top-of-book queues.
Mid-price	The arithmetic average of the best bid and best ask prices; commonly used as a reference fair value when the spread is symmetric.
NLV (Net Liquidation Value)	The current market value of all open positions plus available cash in a trading account. Used in this thesis as the basis for computing true portfolio returns during evaluation, as opposed to per-step reward signals.
OBI (Order Book Imbalance)	Normalized difference in resting volume between bid and ask sides, aggregated across multiple depth levels.
OCI (Order Count Imbalance)	Normalized difference in the number of resting orders between bid and ask sides at a given price level; sensitive to order fragmentation masked by volume-based measures.
OFI (Order Flow Imbalance)	Net directional pressure exerted on the best bid and ask queues by limit order arrivals, cancellations, and executions (Cont et al. 2014). The empirically dominant contemporaneous predictor of short-term price changes.

Term	Definition
OHLCV	Open, High, Low, Close, Volume. The standard aggregated price bar representation used in low- and medium-frequency trading; does not capture intra-bar queue dynamics.
Odd-lot	A trade smaller than the standard round lot size (typically below 100 shares). Predominantly reflects retail and algorithmic small-lot activity.
Queue depletion	The rate at which the best-bid or best-ask queue is consumed by incoming market orders across consecutive snapshots; a leading indicator of imminent price movement.
RSI	Relative Strength Index. A momentum oscillator measuring the speed and magnitude of price changes; used in rule-based and technical trading strategies.
Spread ratio	The current bid-ask spread divided by its recent rolling mean; a regime signal separating absolute spread level from elevated adverse selection conditions.
TWAP	Time-Weighted Average Price. An execution benchmark computed as the mean transaction price over a fixed time window, weighting each trade equally in time. Reported alongside VWAP in the performance comparison tables.
VIX	CBOE Volatility Index. A measure of expected 30-day S&P 500 volatility derived from options prices; a common proxy for broad market fear or uncertainty.
VWAP	Volume-Weighted Average Price. The average price of an asset weighted by traded volume over a period; a common execution benchmark.

E.0.2. Reinforcement Learning and Machine Learning

Figure 5: The six major stages of the experimental pipeline, from data acquisition through post-training evaluation.



Term	Definition
Actor-critic	A class of reinforcement learning architectures combining a policy network (actor) that selects actions with a value network (critic) that evaluates them. Reduces gradient variance relative to pure policy gradient methods.
Bellman equation	The recursive identity expressing the value of a state as immediate reward plus the discounted value of the successor state: $V(s) = r + \gamma V(s')$. The theoretical foundation of temporal difference learning and Q-learning.
Bootstrapping	Updating a value estimate using another estimate as the target, rather than waiting for a complete episode return. Introduces bias but reduces variance; the mechanism underlying all temporal difference algorithms.
DDPG	Deep Deterministic Policy Gradient (Lillicrap et al. 2016). An off-policy actor-critic algorithm for continuous action spaces using deterministic policies and experience replay. Predecessor to TD3.
DQN	Deep Q-Network (Mnih et al. 2015). A value-based deep RL algorithm combining Q-learning with a deep neural network, experience replay, and a frozen target network. Restricted to discrete action spaces.
DSR	Differential Sharpe Ratio (Moody and Saffell 1998). A reward formulation that approximates the gradient of the Sharpe Ratio with respect to returns, aligning the optimization objective with risk-adjusted performance.
Exploration-exploitation tradeoff	The tension in reinforcement learning between taking actions that yield known rewards (exploitation) and trying new actions to discover potentially better outcomes (exploration). Particularly costly in trading due to transaction costs.

Term	Definition
MDP	Markov Decision Process. The formal framework for sequential decision-making under uncertainty, consisting of states, actions, a transition function, and a reward function. Assumes that future states depend only on the current state and action, not on history.
Off-policy	A class of RL algorithms that learn a target policy from data collected by a different behavior policy. Enables experience replay and reuse of historical data. TD3 is off-policy.
On-policy	A class of RL algorithms that must learn from data collected by the same policy being improved. More stable but less sample-efficient than off-policy methods.
PCA	Principal Component Analysis. A dimensionality reduction technique that projects data onto orthogonal axes of maximum variance.
Policy gradient	A family of RL algorithms that directly optimize the policy parameters through gradient ascent on expected cumulative reward. Handles continuous action spaces naturally.
PPO	Proximal Policy Optimization. An on-policy actor-critic algorithm that constrains policy updates to a trust region to improve stability.
Q-learning	A model-free, off-policy value-based RL algorithm that learns the action-value function $Q(s, a)$ through temporal difference updates (Watkins and Dayan 1992).
Replay buffer	A data structure storing past environment transitions (s_t, a_t, r_t, s_{t+1}) for reuse in off-policy training. Decorrelates training samples and improves sample efficiency.
REINFORCE	A Monte Carlo policy gradient algorithm (Williams 1992) that updates policy parameters using full-episode return estimates. High variance; foundational to modern policy gradient methods.

Term	Definition
RL	Reinforcement Learning. A machine learning paradigm in which an agent learns to make sequential decisions by interacting with an environment and maximizing cumulative reward.
SAC	Soft Actor-Critic (Haarnoja et al. 2018). An off-policy actor-critic algorithm incorporating a maximum-entropy objective to encourage exploration through stochastic policies.
SARSA	State-Action-Reward-State-Action. An on-policy temporal difference algorithm that updates $Q(s, a)$ using the action actually taken by the current policy.
Target network	A periodically or slowly updated copy of the Q-network or actor network used to generate stable training targets, preventing the oscillations that arise when both prediction and target change simultaneously. Used in DQN, DDPG, and TD3.
TD3	Twin Delayed Deep Deterministic Policy Gradient (Fujimoto et al. 2018). An off-policy actor-critic algorithm for continuous control that extends DDPG with clipped double Q-learning, delayed policy updates, and target policy smoothing to reduce overestimation bias. The primary algorithm used in this thesis.
TD error	The difference between the Bellman target $y_t = r_t + \gamma Q(s_{t+1}, a_{t+1})$ and the current value estimate $Q(s_t, a_t)$. Minimising the mean squared TD error trains the critic network in all actor-critic algorithms.
Temporal difference (TD) learning	A class of RL methods that update value estimates based on the difference between successive predictions, without requiring full episode trajectories.

Term	Definition
Value function	A function estimating the expected cumulative discounted reward from a given state (or state-action pair) under a given policy. The critic network in actor-critic algorithms approximates this function.

E.0.3. Other

Term	Definition
AAPL	The NASDAQ ticker symbol for Apple Inc. One of six Nasdaq-listed equities — alongside MSFT, TSLA, META, AMZN, and AVGO — used as trading instruments in this thesis.
Maximum drawdown	The largest peak-to-trough decline in portfolio value over an evaluation period, expressed as a percentage. A standard tail-risk measure reported alongside the Sharpe and Sortino ratios in the performance tables.
Out-of-sample evaluation	Assessment of model performance on data not used during training or hyperparameter selection. A central validation principle of this thesis: all reported performance figures are computed on a held-out test period.
Sharpe ratio	A risk-adjusted performance measure defined as the ratio of mean excess return to return volatility. Higher values indicate better return per unit of risk.
Sortino ratio	A risk-adjusted return measure analogous to the Sharpe ratio but using downside deviation — the volatility of negative returns only — rather than total volatility in the denominator. Rewards strategies that limit losses without penalising upside volatility.